

Departamento de Electrónica e Computación
Escuela Técnica Superior de Ingeniería



TRABAJO FIN DE MÁSTER
MÁSTER INTERUNIVERSITARIO EN
COMPUTACIÓN DE ALTAS PRESTACIONES

Desarrollo y evaluación de aplicaciones paralelas con OMP4Py

Estudiante: Alejandro Vedo Godines
Director/a/es/as: César Alfredo Piñeiro Pomar
Juan Carlos Pichel Campos

Santiago de Compostela, junio de 2026.

Agradecimientos

A mis tutores, César Alfredo Piñeiro Pomar y Juan Carlos Pichel Campos, por la orientación técnica durante el desarrollo del trabajo y por facilitar el contexto del proyecto OMP4Py. A las personas que han contribuido al desarrollo original de OMP4Py y de los NAS Parallel Benchmarks, cuyo trabajo ha servido como base para esta evaluación.

Resumen

Este trabajo adapta cinco NAS Parallel Benchmarks (EP, CG, IS, MG y FT) al modelo de programación OMP4Py y los evalúa en el supercomputador Finisterrae III con Python 3.15.0b1t *free-threaded*. El conjunto cubre patrones de cómputo distintos: EP es embarazosamente paralelo, CG resuelve un sistema disperso con reducciones, IS ordena enteros, MG aplica operadores de malla multinivel y FT calcula una transformada de Fourier tridimensional.

La evaluación mide corrección, rendimiento absoluto y escalabilidad en cuatro modos de ejecución: puro, híbrido, compilado y compilado tipado. La campaña principal con Python 3.15.0b1t contiene 630 ejecuciones y la comparación entre Python 3.13.13t, 3.14.4t y 3.15.0b1t añade 1440 ejecuciones. Todas obtuvieron verificación SUCCESSFUL.

El resultado más destacado es el impacto del tipado Cython. En la campaña principal, el modo compilado tipado reduce los tiempos de clase W frente al modo compilado sin tipos en factores de $135.42\times$ en CG, $29.45\times$ en EP, $15.89\times$ en FT, $184.67\times$ en IS y $440.51\times$ en MG. El tipado elimina el despacho de objetos Python en los bucles críticos y convierte el modo 3 en la única configuración práctica para cargas grandes.

Con Python 3.15t, la escalabilidad entre hilos depende del benchmark. EP y FT obtienen aceleraciones claras en clase A, con $10.50\times$ y $14.03\times$ a 32 hilos respectivamente. CG, IS y MG no muestran escalabilidad sostenida. CG queda limitado por sus reducciones frecuentes, IS por la fase secuencial de prefijos sobre un kernel muy corto y MG por la granularidad desigual de sus niveles de malla. La comparación entre Python 3.13t, 3.14t y 3.15t muestra que, con la afinidad de hilos corregida, las tres versiones se comportan de forma similar en EP y FT. El resultado indica que la utilidad práctica de OMP4Py depende de la interacción entre el tipado Cython, la configuración del runtime y la estructura del benchmark.

Abstract

This work adapts five NAS Parallel Benchmarks (EP, CG, IS, MG, and FT) to the OMP4Py programming model and evaluates them on the Finisterrae III supercomputer using Python 3.15.0b1t *free-threaded*. The benchmarks cover distinct computational patterns. EP is embarrassingly parallel, CG solves a sparse linear system with reductions, IS sorts integers, MG applies multilevel grid operators, and FT computes a three-dimensional discrete Fourier transform.

The evaluation measures correctness, absolute performance, and scalability across four execution modes. The main Python 3.15.0b1t campaign contains 630 executions, and the comparison across Python 3.13.13t, 3.14.4t, and 3.15.0b1t adds 1440 executions. All runs passed benchmark verification.

The most significant result is the impact of Cython typing. In the main campaign, the typed compiled mode reduces class W times over the untyped compiled mode by factors of $135.42\times$ in CG, $29.45\times$ in EP, $15.89\times$ in FT, $184.67\times$ in IS, and $440.51\times$ in MG. Typing removes Python object dispatch from critical loops and makes mode 3 the only practical configuration for large workloads.

With Python 3.15t, thread scalability depends on the benchmark. EP and FT obtain clear speedups in class A, reaching $10.50\times$ and $14.03\times$ at 32 threads, respectively. CG, IS, and MG do not show sustained scalability. CG is limited by frequent reductions, IS by a sequential prefix phase over a very short kernel, and MG by the uneven granularity of its grid levels. The comparison across Python 3.13t, 3.14t, and 3.15t shows that, once thread affinity is corrected, the three versions behave similarly in EP and FT. Overall, the practical usefulness of OMP4Py depends on the interaction between Cython typing, runtime configuration, and benchmark structure.

Palabras chave:

- OMP4Py
- OpenMP
- Python free-threaded
- NAS Parallel Benchmarks
- Computación de altas prestaciones
- Cython
- Escalabilidad

Keywords:

- OMP4Py
- OpenMP
- Free-threaded Python
- NAS Parallel Benchmarks
- High Performance Computing
- Cython
- Scalability

Índice general

1. Introducción	1
1.1. Motivación	1
1.2. Objetivos	2
1.3. Alcance	3
1.4. Estructura de la memoria	3
2. Contexto tecnológico	5
2.1. Python y el problema del GIL	5
2.2. OMP4Py	5
2.3. NAS Parallel Benchmarks	7
2.4. Trabajo relacionado	8
3. Adaptación de los NAS Parallel Benchmarks a OMP4Py	9
3.1. Benchmarks adaptados	9
3.2. Metodología de adaptación	9
3.3. Decisiones de implementación	10
3.4. Particularidades por benchmark	11
4. Metodología experimental	15
4.1. Objetivo de la evaluación	15
4.2. Entorno de ejecución	15
4.3. Automatización de la campaña	16
4.4. Diseño de la matriz experimental	17
4.5. Justificación de la restricción de modos en clases grandes	19
4.6. Métricas y tratamiento de los resultados	19
5. Resultados	21
5.1. Efecto del modo de ejecución en clase S	21

5.2.	Escalabilidad en clase S	23
5.3.	Clase W: modos compilados	25
5.4.	Clase A: evaluación principal de escalabilidad	27
5.5.	Síntesis global del modo 3	27
5.6.	Curvas por benchmark	30
5.7.	Interpretación por benchmark	33
5.8.	Limitaciones de la medida	34
5.9.	Conclusiones del capítulo	34
6.	Profiling y diagnóstico	37
6.1.	Objetivo del profiling	37
6.2.	Control de entorno y afinidad	38
6.3.	Cuellos de botella corregidos durante el port	38
6.4.	EP y FT: kernels que sí escalan	38
6.5.	CG, IS y MG: paralelismo ineficiente o granularidad insuficiente	41
6.6.	Interpretación conjunta	47
6.7.	Profiling con el sistema de muestreo de Python 3.15	48
6.8.	Implicaciones para OMP4Py	50
7.	Comparación entre versiones de Python <i>free-threaded</i>	51
7.1.	Diseño de la campaña	51
7.2.	Rendimiento secuencial en modo 3	52
7.3.	Efecto del tipado Cython entre versiones	54
7.4.	Escalabilidad en modo 3	54
7.5.	Curvas por benchmark	56
7.6.	Tiempos absolutos con 32 hilos	63
7.7.	Modo 2 frente a modo 3 con varias versiones	65
7.8.	Relación con la campaña principal	66
7.9.	Implicaciones	66
7.10.	Limitaciones	67
7.11.	Conclusiones del capítulo	67
8.	Conclusiones	69
8.1.	Hallazgos principales	69
8.2.	Limitaciones	71
8.3.	Trabajo futuro	71
8.4.	Valoración global	72
	Apéndices	73

A. Interfaz de ejecución de los benchmarks	75
B. Ejemplo de kernel adaptado: EP en modo tipado	77
C. Infraestructura de ejecución en Finisterrae III	79
D. Estructura de los resultados de campaña	81
E. Glosario de acrónimos	83
F. Glosario de términos	85
Bibliografía	87

Índice de figuras

2.1. Flujo de ejecución de un programa OMP4Py. Las directivas se procesan mediante transformación AST en todos los modos. El modo determina si los kernels del usuario se compilan con Cython y qué implementación del runtime ejecuta la distribución de trabajo y la sincronización.	7
4.1. Flujo de la infraestructura de campaña en Finisterrae III: generación de la matriz de combinaciones, envío como array de Slurm, ejecución de cada combinación y consolidación de resultados.	16
5.1. Tiempo NAS con un hilo en clase S para los cuatro modos de ejecución. El eje vertical usa escala logarítmica.	22
5.2. Speedup en clase S, modo 3, Python 3.15t. La línea discontinua marca el umbral de speedup 1.	23
5.3. Tiempo NAS por número de hilos en clase S, modo 3, Python 3.15t. Cada línea corresponde a un benchmark.	24
5.4. Speedup en clase W, modo 3, Python 3.15t.	26
5.5. Tiempo NAS por número de hilos en clase W, modo 3, Python 3.15t. Cada línea corresponde a un benchmark.	26
5.6. Speedup en clase A, modo 3, Python 3.15t.	28
5.7. Tiempo NAS por número de hilos en clase A, modo 3, Python 3.15t. Cada línea corresponde a un benchmark.	28
5.8. Speedup a 32 hilos en modo 3 para las clases S, W y A.	29
5.9. CG: speedup por clase en modo 3, Python 3.15t.	31
5.10. EP: speedup por clase en modo 3, Python 3.15t.	31
5.11. FT: speedup por clase en modo 3, Python 3.15t.	32
5.12. IS: speedup por clase en modo 3, Python 3.15t.	32
5.13. MG: speedup por clase en modo 3, Python 3.15t.	33

6.1. CPUs utilizadas por EP y FT en clase W, modo 3, según <code>perf stat</code>	39
6.2. IPC de EP y FT en clase W, modo 3, según <code>perf stat</code>	40
6.3. Porcentaje de fallos de caché de EP y FT en clase W, modo 3, según <code>perf stat</code>	40
6.4. CPUs utilizadas por CG, IS y MG con Python 3.14t en clase W, modo 3.	43
6.5. IPC de CG, IS y MG con Python 3.14t en clase W, modo 3.	43
6.6. Porcentaje de fallos de caché de CG, IS y MG con Python 3.14t en clase W, modo 3.	44
6.7. CPUs utilizadas por CG, IS y MG con Python 3.15t en clase W, modo 3.	44
6.8. IPC de CG, IS y MG con Python 3.15t en clase W, modo 3.	45
6.9. Porcentaje de fallos de caché de CG, IS y MG con Python 3.15t en clase W, modo 3.	45
6.10. CPUs utilizadas por CG, IS y MG con Python 3.13t en clase W, modo 3.	46
6.11. IPC de CG, IS y MG con Python 3.13t en clase W, modo 3.	46
6.12. Porcentaje de fallos de caché de CG, IS y MG con Python 3.13t en clase W, modo 3.	47
7.1. Speedup a 32 hilos en modo 3 para Python 3.13t, 3.14t y 3.15t.	54
7.2. EP clase S, modo 3: speedup por versión de Python.	58
7.3. EP clase W, modo 3: speedup por versión de Python.	58
7.4. FT clase S, modo 3: speedup por versión de Python.	59
7.5. FT clase W, modo 3: speedup por versión de Python.	59
7.6. CG clase S, modo 3: speedup por versión de Python.	60
7.7. CG clase W, modo 3: speedup por versión de Python.	60
7.8. IS clase S, modo 3: tiempo NAS por versión de Python. El speedup no es fiable porque el tiempo con un hilo aparece redondeado a 0.00 s.	61
7.9. IS clase W, modo 3: speedup por versión de Python.	61
7.10. MG clase S, modo 3: speedup por versión de Python.	62
7.11. MG clase W, modo 3: speedup por versión de Python.	62
7.12. Tiempo NAS con 32 hilos en modo 3 para las tres versiones de Python. El eje vertical usa escala logarítmica.	64

Índice de tablas

3.1.	Estado de los benchmarks NAS en el repositorio.	10
5.1.	Cobertura de las campañas finales ejecutadas en Finisterrae III.	21
5.2.	Mejor tiempo NAS en clase S con un hilo para los cuatro modos de ejecución en Python 3.15t.	22
5.3.	Speedup a 32 hilos respecto a 1 hilo en clase S para cada modo de Python 3.15t.	23
5.4.	Mejor tiempo NAS en clase W con un hilo para los modos compilados de Python 3.15t.	25
5.5.	Speedup a 32 hilos respecto a 1 hilo en clase W para los modos compilados de Python 3.15t.	25
5.6.	Mejor tiempo NAS en clase A, modo 3, Python 3.15t.	27
5.7.	Speedup por número de hilos en clase A, modo 3, Python 3.15t.	27
5.8.	Mejor número de hilos y speedup asociado en modo 3, Python 3.15t.	29
5.9.	Síntesis de tiempos, speedup y Mop/s a 32 hilos en modo 3, Python 3.15t.	30
6.1.	Perfil con <code>perf stat</code> de EP y FT en clase W, modo 3.	39
6.2.	Perfil con <code>perf stat</code> de CG, IS y MG en clase W, modo 3.	42
6.3.	Coste de sincronización medido con el profiler de muestreo de Python 3.15 (<code>profiling.sampling</code> , modo <i>wall</i> , todos los hilos). Porcentaje de las muestras de ejecución paralela que caen en primitivas de espera del runtime OMP4Py (barrera y reducción, <code>threading.Condition.wait</code>), frente al número de hilos. Clase S, modo 1.	49
6.4.	Gestión de hilos con 8 hilos según el profiler de muestreo de Python 3.15 (clase S, modo 1). <i>Hilos trabajadores distintos</i> es el número de identificadores de hilo con muestras de cómputo observados durante la ejecución: EP emplea un único equipo persistente, mientras que CG, FT y MG crean y destruyen un equipo por región paralela, generando cientos o miles de hilos efímeros.	49

7.1.	Mejor tiempo NAS con 1 hilo en modo 3 para cada versión de Python.	52
7.2.	Mejor tiempo NAS por número de hilos en modo 3 para cada versión de Python.	53
7.3.	Efecto del tipado Cython con 1 hilo en la campaña de comparación de versiones.	55
7.4.	Speedup a 32 hilos respecto a 1 hilo en modo 3 para cada versión de Python. .	56
7.5.	Speedup por número de hilos en modo 3 para cada versión de Python.	57
7.6.	Mejor número de hilos en modo 3 para cada versión, clase y benchmark. . . .	63
7.7.	Mejor tiempo NAS con 32 hilos en modo 3 para cada versión de Python. . . .	64
7.8.	Versión más rápida con 32 hilos en modo 3 y margen sobre la segunda mejor. .	65
7.9.	Mejor resultado de modo 2 frente a modo 3 considerando todas las versiones de Python y todos los recuentos de hilos.	66

Introducción

PYTHON ocupa una posición consolidada en la computación científica. Su ecosistema de bibliotecas, la legibilidad del código y la rapidez de prototipado lo han convertido en el lenguaje de referencia para análisis numérico y aprendizaje automático [1]. Sin embargo, la ejecución paralela con múltiples hilos ha estado históricamente limitada por el *Global Interpreter Lock* (GIL), que impide la ejecución simultánea de bytecode Python en un mismo proceso. Las versiones *free-threaded* de Python, disponibles experimentalmente a partir de Python 3.13 [2], eliminan esa restricción y abren la posibilidad de explotar paralelismo de memoria compartida directamente en el intérprete.

OMP4Py es una biblioteca que adapta el modelo de programación OpenMP al entorno Python [3, 4]. Permite expresar paralelismo mediante directivas integradas en el código, con una sintaxis próxima a la de OpenMP en C o C++ [5]. La biblioteca ofrece cuatro modos de ejecución (desde un intérprete puro hasta kernels Cython con tipos nativos), lo que permite comparar el coste de cada capa de abstracción sobre el mismo código fuente.

Este TFM evalúa OMP4Py sobre NAS Parallel Benchmarks adaptados a Python. Los NAS son adecuados para este propósito porque definen problemas conocidos, clases de tamaño estandarizadas y rutinas de verificación propias [6]. El trabajo no consiste en validar implementaciones triviales. Se trata de portar benchmarks completos, conservar su semántica y medir si OMP4Py puede explotarlos de forma paralela y eficiente en un entorno de supercomputación real.

1.1. Motivación

La pregunta central del TFM es si un programador puede expresar paralelismo de estilo OpenMP en Python con OMP4Py, obtener resultados correctos y conseguir aceleración real sobre más de un hilo.

La motivación surge de una tensión habitual en computación científica de altas presta-

ciones [7]. Python es cómodo para desarrollar, pero los kernels numéricos intensivos suelen requerir código nativo o bibliotecas como NumPy [8], que delegan el trabajo paralelo a capas C sin exponer el modelo de programación al desarrollador. OMP4Py propone una vía intermedia: mantiene una interfaz Python y, cuando el rendimiento lo exige, compila los kernels mediante Cython. Este TFM estudia esa vía con benchmarks estandarizados, separando el efecto del modo de ejecución del efecto del paralelismo entre hilos.

Los resultados obtenidos durante la evaluación revelan que ambos factores no tienen el mismo peso. En la campaña principal con Python 3.15t, el tipado Cython (modo 3) reduce los tiempos entre uno y varios órdenes de magnitud frente al modo compilado sin tipos (modo 2), incluso con un único hilo. La escalabilidad entre hilos depende mucho del benchmark: EP y FT alcanzan speedups superiores a $10\times$ en clase A, mientras que CG, IS y MG quedan limitados por sincronización, granularidad o patrones de memoria. La comparación entre versiones muestra que Python 3.13t, 3.14t y 3.15t se comportan de forma similar cuando el entorno de afinidad está correctamente configurado.

1.2. Objetivos

El objetivo general es desarrollar y evaluar aplicaciones paralelas con OMP4Py usando NAS Parallel Benchmarks como conjunto de referencia en el cluster Finisterrae III.

Los objetivos específicos son los siguientes.

- Estudiar el modelo de programación de OMP4Py y sus cuatro modos de ejecución.
- Adaptar cinco NAS Parallel Benchmarks a OMP4Py conservando la verificación oficial de cada uno como criterio de aceptación.
- Construir una infraestructura reproducible para ejecutar los benchmarks en Finisterrae III mediante Slurm.
- Medir rendimiento y escalabilidad con los cuatro modos de ejecución, distintas clases de problema y entre 1 y 32 hilos.
- Comparar el impacto del tipado Cython frente al aumento del número de hilos en cada benchmark.
- Comparar el comportamiento de Python 3.13t, Python 3.14t y Python 3.15t en modo *free-threaded*.
- Identificar mediante profiling los cuellos de botella en los casos donde la escalabilidad sea reducida, usando `perf stat` y el nuevo sistema de muestreo de Python 3.15 para medir el coste de sincronización y el balance de carga.

1.3. Alcance

El trabajo cubre los benchmarks EP, CG, IS, MG y FT. Son los cinco sobre los que se ha introducido paralelismo explícito con directivas OMP4Py y cuya verificación es correcta en todos los modos. Los benchmarks BT, SP y LU quedan fuera del alcance principal: sus estructuras de datos y patrones de sincronización presentan restricciones que exceden el tiempo disponible para este TFM, y sus adaptaciones actuales no incorporan paralelismo.

Las modificaciones a la propia biblioteca OMP4Py también quedan fuera del alcance. La biblioteca se toma como entorno dado. El trabajo se centra en las aplicaciones y en su evaluación experimental.

La campaña experimental principal cubre las clases de problema S, W y A. La clase S permite comparar los cuatro modos de ejecución y validar la corrección en todos los benchmarks. La clase W se centra en los modos compilados y la clase A en el modo 3, que es el único modo práctico para cargas grandes. Además, se ejecuta una campaña de comparación entre Python 3.13t, 3.14t y 3.15t sobre las clases S y W. Todas las ejecuciones finales obtuvieron verificación correcta.

1.4. Estructura de la memoria

El Capítulo 2 describe el contexto tecnológico, incluyendo Python *free-threaded*, OMP4Py y los NAS Parallel Benchmarks. El Capítulo 3 detalla el proceso de adaptación de los cinco benchmarks y las decisiones de implementación más relevantes. El Capítulo 4 define la metodología experimental y la infraestructura de medición. El Capítulo 5 presenta y analiza los resultados de la campaña principal con Python 3.15t. El Capítulo 6 diagnostica con `perf stat` la ocupación real de CPU, el IPC y los límites de escalabilidad, y con el nuevo sistema de muestreo de Python 3.15 los costes de sincronización y el balance de carga. El Capítulo 7 compara Python 3.13t, 3.14t y 3.15t. El Capítulo 8 resume los hallazgos y plantea el trabajo futuro.

Contexto tecnológico

2.1. Python y el problema del GIL

Python gestiona la memoria mediante contadores de referencia. Para evitar condiciones de carrera en esos contadores cuando múltiples hilos acceden al mismo objeto, el intérprete CPython mantiene el GIL, un mutex global que solo permite a un hilo ejecutar bytecode Python en cada instante. La consecuencia práctica es que los programas Python multihilo no pueden aprovechar múltiples núcleos para trabajo intensivo en el intérprete, incluso en sistemas con decenas de cores disponibles.

La restricción no impide toda forma de paralelismo. Extensiones nativas como NumPy [8] liberan el GIL durante operaciones vectoriales, compiladores como Numba [9] generan código máquina para funciones numéricas, y bibliotecas como mpi4py [10] o Dask [11] distribuyen el trabajo mediante procesos. El propio módulo `multiprocessing` evita el problema con procesos independientes y memoria privada. Sin embargo, ninguna de esas vías permite expresar paralelismo de memoria compartida al nivel del código Python, con el modelo de hilos y directivas que caracteriza a OpenMP.

Las versiones *free-threaded* de Python eliminan el GIL. Disponibles de forma experimental desde Python 3.13t [2], sustituyen los contadores de referencia por mecanismos atómicos y biased locking. Esto añade un pequeño sobrecoste en código monohilo, pero hace posible la ejecución paralela real con hilos Python. Este TFM utiliza Python 3.15.0b1t como entorno principal y compara sus resultados con Python 3.13.13t y Python 3.14.4t, compilados también con el GIL desactivado.

2.2. OMP4Py

OMP4Py es una implementación del modelo de programación OpenMP para Python [3, 4]. Su objetivo es permitir que el programador exprese regiones paralelas, distribución de itera-

ciones y sincronización mediante directivas integradas en código Python, con una sintaxis análoga a la de OpenMP en C, C++ o Fortran [5, 12].

Las directivas se expresan como gestores de contexto:

```
with omp("parallel for"):
    for i in range(n):
        ...
```

La biblioteca ofrece cuatro modos de ejecución, seleccionables en tiempo de ejecución mediante el argumento `-m`:

Modo	Nombre	Descripción
0	puro	Runtime puro de OMP4Py en Python; kernels sin compilar.
1	híbrido	Runtime nativo (Cython) si está disponible; kernels sin compilar.
2	compilado	Kernels compilados a Cython; tipos inferidos.
3	compilado tipado	Cython con anotaciones de tipo explícitas y memoryviews.

En los cuatro modos, las directivas se transforman mediante el árbol sintáctico abstracto (AST) del módulo `ast` de Python y el paralelismo se ejecuta con hilos reales de la API `threading`, de modo que en Python *free-threaded* pueden aprovechar múltiples núcleos. La diferencia entre modos está en qué partes del código se compilan (Figura 2.1). En los modos 0 y 1 los kernels del usuario no se compilan. El modo 0 fuerza el runtime puro de OMP4Py, cuyas primitivas de distribución de trabajo y sincronización están implementadas en Python, y actúa como línea base funcional. El modo 1 usa el runtime por defecto, que emplea la implementación nativa del propio runtime, compilada con Cython, cuando está disponible. Los modos 2 y 3 compilan además los kernels del usuario a C mediante Cython [13]. La diferencia entre ellos es que el modo 3 permite anotar variables con tipos Cython (`cython.double`, `cython.int`, `memoryviews`) para eliminar la indirección de los objetos Python en los bucles numéricos.

La compilación en los modos 2 y 3 se realiza en tiempo de ejecución, antes del primer uso de cada kernel. Los kernels compilados se almacenan en caché para reutilizarlos en ejecuciones posteriores.

La diversidad de modos permite aislar tres factores de rendimiento: el coste del runtime Python frente al runtime compilado, el impacto del tipado Cython sobre los bucles numéricos y la ganancia real del paralelismo entre hilos.

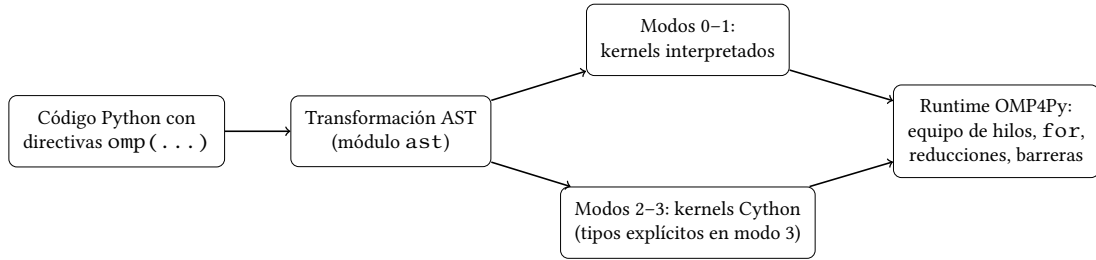


Figura 2.1: Flujo de ejecución de un programa OMP4Py. Las directivas se procesan mediante transformación AST en todos los modos. El modo determina si los kernels del usuario se compilan con Cython y qué implementación del runtime ejecuta la distribución de trabajo y la sincronización.

2.3. NAS Parallel Benchmarks

Los NAS Parallel Benchmarks (NPB) son un conjunto de benchmarks desarrollado por NASA para evaluar sistemas paralelos [6]. Cada benchmark implementa un núcleo computacional representativo de aplicaciones de simulación numérica.

- **EP** (*Embarrassingly Parallel*): generación de pares aleatorios gaussianos mediante el método de Box-Muller. El trabajo útil es embarazosamente paralelo. La única sincronización es una reducción global al final.
- **CG** (*Conjugate Gradient*): estimación del menor autovalor de una matriz dispersa irregular mediante gradiente conjugado. Requiere productos matriz-vector, actualizaciones de vectores y reducciones escalares en cada iteración.
- **IS** (*Integer Sort*): ordenación entera mediante un algoritmo de counting sort por cubos. Exige paralelismo en histogramas, offsets y ordenación local con semántica estricta.
- **MG** (*Multi-Grid*): resolución aproximada de la ecuación de Poisson 3D mediante el método multigrid. Opera sobre mallas en varios niveles de resolución con operaciones de restricción, prolongación e interpolación.
- **FT** (*Fourier Transform*): solución de una EDP mediante transformadas de Fourier 3D complejas. El kernel principal es una FFT aplicada sucesivamente a cada dimensión de un array tridimensional de números complejos.

Cada benchmark define varias clases de problema (S, W, A, B, C, D), asociadas a tamaños de malla o números de iteraciones crecientes. Esta escala permite estudiar el comportamiento de OMP4Py en cargas de distinta granularidad sin cambiar de benchmark.

Los NPB incluyen rutinas de verificación internas que comprueban la exactitud numérica del resultado. Esa verificación es la única condición de aceptación utilizada en este trabajo. Una adaptación que ejecute más rápido pero no verifique correctamente se rechaza.

2.4. Trabajo relacionado

El paralelismo en Python se ha abordado tradicionalmente por vías que evitan el GIL en lugar de eliminarlo. Las extensiones nativas como NumPy [8] liberan el GIL durante operaciones vectoriales, y compiladores como Numba [9] o Cython [13] generan código máquina para funciones numéricas. En todos los casos el modelo de programación paralela queda oculto en la capa nativa y no se expresa al nivel del código Python. La excepción más próxima a OMP4Py es PyOMP [14], que añade directivas OpenMP a funciones compiladas con Numba. Su alcance queda, sin embargo, restringido al subconjunto del lenguaje que Numba puede compilar, mientras que OMP4Py opera sobre código Python estándar y delega en Cython la compilación opcional de los kernels. El paralelismo de procesos (`multiprocessing`, `mpi4py` [10] o Dask [11]) escala a múltiples nodos, pero con memoria privada y un coste de comunicación que no corresponde al modelo de memoria compartida de OpenMP [5, 12].

La eliminación del GIL en las versiones *free-threaded* [2] abre una vía distinta: ejecutar hilos Python reales sobre memoria compartida. OMP4Py [3, 4] se sitúa en esa vía al trasladar el modelo de directivas de OpenMP al código Python. Este trabajo se diferencia de los estudios que presentan la propia biblioteca en que su objeto no es OMP4Py en sí, sino su evaluación sobre aplicaciones completas y a lo largo de varias versiones del intérprete.

Como cargas de trabajo se emplean los NAS Parallel Benchmarks [6], ampliamente utilizados para evaluar sistemas paralelos. Existen portados previos de los NPB a Python a partir de las versiones seriales en C++ [15], que sirven de base serial para la adaptación descrita en el Capítulo 3. Hasta donde alcanza este trabajo, la evaluación de OMP4Py sobre kernels NPB completos, con sus cuatro modos de ejecución y la comparación entre versiones *free-threaded* de Python, no se había abordado de forma sistemática.

Adaptación de los NAS Parallel Benchmarks a OMP4Py

3.1. Benchmarks adaptados

Se adaptaron cinco NAS Parallel Benchmarks al modelo de programación OMP4Py: EP, CG, IS, MG y FT. En todos los casos se conservaron la clase del problema, el número de iteraciones y la verificación oficial del benchmark como criterio único de aceptación. Los tres benchmarks restantes del conjunto NPB (BT, SP y LU) quedan diferidos. Sus estructuras de datos y patrones de sincronización presentan restricciones que exceden el alcance de este TFM y sus adaptaciones actuales no incorporan paralelismo con directivas OMP4Py.

Todos los benchmarks siguen una interfaz común:

```
python <BENCHMARK>_Python.py -c <CLASE> -t <HILOS> -m <MODOS>
```

donde `-c` selecciona la clase NAS (S, W, A), `-t` fija el número de hilos y `-m` selecciona el modo de ejecución.

La Tabla 3.1 resume el estado funcional de los ocho benchmarks disponibles en el repositorio.

3.2. Metodología de adaptación

El port se realizó de forma incremental. Se partió de las versiones Python seriales de referencia y, cuando estaban disponibles, se consultaron también las versiones C/OpenMP originales. El proceso se estructuró en diez pasos:

1. Partir de la versión Python serial NAS.
2. Comparar con la versión C/OpenMP cuando existe.

Benchmark	Estado	Observación principal
EP	Completo	Paralelismo y tipado completos; escala en modo 3
CG	Completo	Reducción paralela; sincronización dominante
IS	Completo	Semántica de cubos revisada para OMP4Py
MG	Completo	Múltiples kernels en malla; granularidad desigual
FT	Completo	FFT compleja; escala en modo 3
BT	Diferido	Sin directivas OMP4Py; solo compilación Cython
SP	Diferido	Sin directivas OMP4Py; solo compilación Cython
LU	Diferido	Sin directivas OMP4Py; solo compilación Cython

Tabla 3.1: Estado de los benchmarks NAS en el repositorio.

3. Conservar clase, tamaño, iteraciones, salida y verificación NAS.
4. Eliminar dependencias de Numba [9] y adaptar imports.
5. Añadir la interfaz común `-c`, `-t`, `-m`.
6. Conseguir verificación correcta en modo 0 antes de cualquier otra modificación.
7. Separar los kernels numéricos del código de orquestación (temporizadores, E/S, verificación).
8. Compilar solo los kernels y pasar tamaños y constantes como argumentos.
9. Cambiar el indexado NumPy encadenado (`u[k][j][i]`) por indexado directo (`u[k, j, i]`).
10. Añadir anotaciones de tipo Cython y memoryviews en el modo 3.

Una optimización solo se acepta si la verificación sigue siendo SUCCESSFUL. Este criterio condicionó todo el proceso. Las versiones más rápidas que fallaban la verificación se descartaban.

3.3. Decisiones de implementación

Separación de kernels y orquestación. Las funciones de alto nivel de los NAS mezclan inicialización, temporizadores, E/S, selección de clase, verificación y kernels numéricos. Compilar esas funciones completas con OMP4Py resulta frágil porque la transformación AST no puede aplicarse de forma segura a código que combina efectos secundarios con regiones paralelas. Por ello, la orquestación se mantuvo en Python puro y solo se compilaban las funciones con carga numérica significativa.

Compilación dinámica tras inicialización. En varios benchmarks, las constantes y tamaños de array dependen de la clase seleccionada y se calculan durante la ejecución. Si un kernel compilado captura esos valores antes de que la inicialización se complete, puede usar dimensiones incorrectas. Para evitarlo, la decoración OMP4Py se aplica después de completar la inicialización de constantes, en una función `compile_kernels()` que se invoca explícitamente una vez conocida la clase.

Indexado directo en NumPy. Las expresiones como `u[k][j][i][m]` generan objetos intermedios en NumPy y dificultan la generación de código eficiente por parte de Cython. La sustitución por `u[k, j, i, m]` es semánticamente equivalente y permite a Cython generar accesos directos a los buffers C cuando se combina con `memoryviews`.

Variables privadas explícitas. En EP apareció un fallo de verificación en modo compilado tipado: varias variables temporales se compartían entre hilos y producían resultados incorrectos. La corrección consistió en declararlas como privadas dentro de la región paralela. Este caso ilustra que una traducción estructuralmente parecida al código C/OpenMP no garantiza una semántica equivalente en Python. La visibilidad de las variables locales depende del ámbito léxico del intérprete, no de las reglas de ámbito de OpenMP.

3.4. Particularidades por benchmark

EP. EP genera pares de variables aleatorias con distribución normal mediante el método de Box-Muller. El kernel paralelo central recorre 2^{nk} bloques de dos muestras cada uno. Dentro de cada bloque calcula una semilla local, evalúa raíces cuadradas y aritmética de punto flotante en escalares, y acumula dos contadores globales (`sx`, `sy`) con una reducción.

La implementación tiene dos versiones compiladas: `ep_compute` (modo 2) y `ep_compute_types` (modo 3). Ambas usan la misma directiva `parallel reduction(+:sx, sy)` con planificación estática, pero en `ep_compute_types` todas las variables del bucle interno están declaradas con tipos C de Cython (`cython.int`, `cython.double`) y se declaran explícitamente privadas para evitar condiciones de carrera. El módulo principal selecciona la versión tipada en modo 3 mediante `use_compiled_types()`. Sin ese tipado, cada operación aritmética pasa por el mecanismo de despacho de objetos Python, lo que multiplica el tiempo por un factor cercano a 30 en las clases S y W.

La sección crítica al final de la región paralela actualiza el histograma de pares. Esta operación es secuencial y representa un porcentaje despreciable del tiempo total en modo 3.

CG. CG resuelve un sistema lineal disperso con el método del gradiente conjugado y calcula el autovalor más pequeño de la matriz. El bucle de gradiente conjugado tiene 25 iteraciones.

Cada una realiza un producto matriz-vector disperso seguido de varias operaciones de reducción.

La implementación distingue entre modo no compilado y modo compilado. En los modos 0 y 1, la función `conj_grad` usa directivas `parallel for` independientes para cada bucle. En los modos 2 y 3, la función `conj_grad_compiled` envuelve todo el bucle de iteraciones en una única directiva `parallel`, de modo que el grupo de hilos se crea una sola vez por llamada y los bucles internos se distribuyen con `for`. Esta distinción reduce el overhead de creación y destrucción de hilos, que es especialmente relevante en clase S donde los tiempos de kernel son del orden de décimas de segundo.

Los kernels auxiliares (`normalize_z_compiled`, `shift_column_indices_compiled`, `fill_vector_compiled`) están anotados con memoryviews de Cython (`cython.long[:]`, `cython.double[:]`) para el acceso a arrays. La versión tipada evita que las operaciones escalares de los bucles internos pasen por objetos Python. Esta diferencia es decisiva en rendimiento absoluto: en clase W, el modo 3 reduce el tiempo con un hilo de 105.6 s a 0.78 s frente al modo 2.

La limitación principal de CG es el número de barreras de sincronización. Dentro de cada iteración del gradiente conjugado hay cuatro reducciones escalares. En clases con tiempos de kernel cortos, el overhead de sincronización supera el trabajo paralelo y provoca degradación activa con más hilos.

IS. IS ordena enteros de 32 bits mediante un bucket sort en dos fases. La primera construye el histograma de claves y la segunda realiza la ordenación local en cada cubo. Para inicializar el vector de claves, el módulo principal usa `create_seq` (función Python sin compilar, modos 0/1/2) o `create_seq_types` (compilada con Cython y tipos explícitos, modo 3). Además, las rutas críticas de ordenación usan variables privadas y tipos Cython explícitos en modo 3. Esta distinción es visible en clase W: el modo 3 reduce el tiempo con un hilo de 5.54 s a 0.03 s frente al modo 2.

La función `rank` usa una única directiva `parallel` con un bloque de variables privadas explícitas por hilo. Cada hilo gestiona su identificador, los offsets de cubos, las posiciones de inserción y los contadores locales. Una directiva `barrier` explícita separa la fase de histograma de la fase de ordenación. Todos los hilos deben completar el conteo global de claves antes de calcular los offsets acumulados, que se calculan en una sección secuencial.

La limitación estructural de IS es ese paso de prefijos acumulados, que es necesariamente secuencial y separa las dos fases paralelas. El resultado es que, con tiempos de kernel ya cortos en clase W (0.17 s con un hilo), el overhead de la barrera y la gestión de hilos supera el trabajo paralelo a medida que aumenta el número de hilos.

MG. MG aplica un ciclo en V del método multigrid para resolver una ecuación de Poisson tridimensional discreta. Los kernels del ciclo son `psinv` (suavizado), `resid` (cálculo de residuo), `rprj3` (restricción), `interp` (interpolación), `comm3` (comunicación de fronteras) y `zero3` (inicialización).

Durante el port se eliminaron dependencias de estado global en kernels como `zran3` y `norm2u3`, y se añadieron rutas tipadas para que Cython pudiese compilar los accesos a malla y los acumuladores escalares sin atravesar el intérprete. La diferencia entre modo 2 y modo 3 es muy grande en tiempo absoluto: en clase W, el modo 3 baja de 198.2 s a 0.45 s con un hilo.

Los arrays de malla se representan como vectores 1D con indexado manual (`r[(i3*n2 + i2)*n1 + i1]`) porque las memoryviews de Cython en sus versiones más accesibles solo funcionan eficientemente con arrays de forma fija o con vistas 1D. El bucle más externo de cada kernel (`for i3 in range`) se distribuye entre hilos con `with omp("for")`.

La limitación de MG es la jerarquía de niveles. El ciclo en V desciende hasta una malla de $2 \times 2 \times 2$. En los niveles gruesos, el bucle externo tiene solo unas pocas iteraciones (1 o 2), lo que hace imposible distribuir trabajo entre más de 2 hilos. Esos niveles concentran una fracción del tiempo total que limita el speedup global. Por eso MG mejora mucho con el tipado, pero no con el aumento de hilos.

FT. FT calcula la transformada de Fourier tridimensional discreta de un array complejo mediante FFTs 1D sucesivas sobre cada eje (tres pasadas). La función principal abre una única directiva `parallel` que agrupa todos los hilos para el cómputo completo. Dentro de esa región, cada pasada llama a `cffts1/2/3`, que distribuye las filas del array entre hilos con `for` y delega el cómputo a `cfftz` y `fftz2`. El patrón de región `parallel` exterior persistente evita crear y destruir el grupo de hilos en cada pasada de la FFT.

La implementación tiene una jerarquía completa de funciones tipadas: `fft_types`, `cffts1/2/3_types`, `cfftz_types`, `fftz2_types`. En sus versiones no tipadas, las operaciones sobre pares complejos usan el tipo `complex` de Python, que almacena parte real e imaginaria como objetos separados y llama a funciones Python para la suma y el producto. En las versiones tipadas, esos valores son `cython.doublecomplex` (`double complex` de C99), y las operaciones aritméticas son instrucciones de punto flotante del procesador. La relación entre modo 2 y modo 3 se sitúa entre $15\times$ y $19\times$ según la clase y la versión del intérprete, porque cada iteración del bucle interior de `fftz2` realiza dos multiplicaciones y dos sumas sobre complejos.

FT sí muestra speedup en modo 3. En clase W, el mejor punto aparece con 16 hilos y en clase A el speedup óptimo es $14.69\times$. La limitación aparece al aumentar demasiado el número de hilos. La FFT tridimensional tiene dependencias entre pasadas: la pasada sobre el eje x debe

completarse antes de la pasada sobre el eje y , que a su vez precede a la pasada sobre el eje z . Dentro de cada pasada las filas son independientes y el `parallel for` funciona, pero el balance de carga y la presión de memoria reducen la eficiencia en recuentos altos.

Metodología experimental

4.1. Objetivo de la evaluación

La evaluación experimental mide tres aspectos del comportamiento de OMP4Py sobre los cinco benchmarks adaptados: corrección, rendimiento absoluto y escalabilidad. La corrección se comprueba mediante la verificación oficial de cada benchmark. El rendimiento se mide con el tiempo interno reportado por NAS y con los Mop/s de salida. La escalabilidad se estudia comparando el mejor tiempo observado con 1, 2, 4, 8, 16 y 32 hilos dentro del mismo modo, clase e intérprete.

La comparación entre modos de ejecución articula el estudio. Los modos 0 y 1 sirven como referencia funcional y permiten cuantificar el coste de interpretar directivas OMP4Py sin compilación. Los modos 2 y 3 son las configuraciones orientadas a rendimiento, y la diferencia entre ambos mide el impacto del tipado Cython sobre los kernels numéricos.

4.2. Entorno de ejecución

Los experimentos se ejecutan en Finisterrae III, el supercomputador del Centro de Supercomputación de Galicia (CESGA), mediante el gestor de trabajos Slurm. Se utilizan nodos de tipo c1k, con procesadores Intel Xeon de 48 núcleos y memoria DDR4. Las ejecuciones se limitan a 32 hilos para facilitar la planificación de Slurm y cubrir los rangos de baja, media y alta concurrencia dentro de un nodo.

El intérprete principal es Python 3.15.0b1t, compilado con soporte *free-threaded* (`--disable-gil`) en el directorio de usuario. Para la comparación entre versiones se utilizaron también Python 3.13.13t y Python 3.14.4t, ambos compilados como intérpretes *free-threaded*. En las tres versiones de la campaña comparativa se comprobó que el GIL estaba desactivado.

El entorno de ejecución se automatiza mediante `scripts/ft3/setup_venvs.sh`, que crea un entorno virtual por versión de Python e instala las dependencias necesarias. Las

campañas finales usaron NumPy 2.4.6 y Cython 3.2.5 en los tres intérpretes. El código fuente se mantuvo idéntico entre versiones para que la variable experimental fuese el intérprete y no la implementación del benchmark.

Las campañas finales no fijan por defecto variables de afinidad de OpenMP como `OMP_PROC_BIND`, `OMP_PLACES` o `GOMP_CPU_AFFINITY`. Estas variables se eliminan explícitamente en los scripts de ejecución salvo que se active un modo experimental específico. La decisión evita que la política de afinidad del entorno distorsione la comparación entre versiones y recuentos de hilos.

4.3. Automatización de la campaña

La ejecución en Finisterrae III se automatiza con los scripts de `scripts/ft3`. El script `make_matrix.py` genera la matriz de combinaciones de cada campaña, `submit_campaign.sh` crea el directorio de resultados y lanza el array de Slurm, `array_run.sbatch` ejecuta los trabajos, `run_one.py` ejecuta una combinación concreta y `parse_results.py` reconstruye las tablas `summary.csv`, `summary_best.csv` y los registros JSON. La Figura 4.1 resume este flujo.

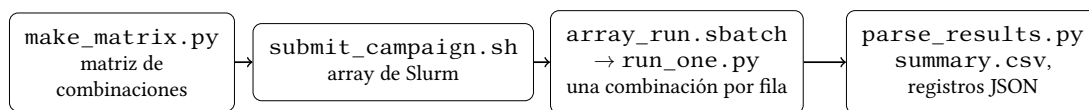


Figura 4.1: Flujo de la infraestructura de campaña en Finisterrae III: generación de la matriz de combinaciones, envío como array de Slurm, ejecución de cada combinación y consolidación de resultados.

Cada fila de la matriz invoca el benchmark correspondiente mediante la interfaz común descrita en el Capítulo 3:

```
python <BENCHMARK>_Python.py -c <CLASE> -t <HILOS> -m <MODO>
```

Para cada ejecución se registra la salida estándar, la salida de error, el código de retorno, el tiempo de proceso externo, el tiempo interno NAS, los Mop/s y el estado de verificación. Las combinaciones se distribuyen entre múltiples trabajos Slurm mediante arrays (`--array`), donde cada elemento ejecuta varias filas de la matriz de forma secuencial. Esta organización evita una ejecución monolítica, limita el número de trabajos enviados al planificador y produce ficheros con nombres semánticos, no dependientes del identificador interno de Slurm.

Una ejecución se considera válida si el proceso termina con código de retorno 0 y la verificación reporta `SUCCESSFUL`. Las ejecuciones con error, timeout o verificación incorrecta se conservan en los resultados como indicadores de limitaciones del modo o la implementación.

4.4. Diseño de la matriz experimental

La evaluación se divide en dos bloques: una campaña principal con Python 3.15.0b1t y una campaña comparativa entre versiones *free-threaded*.

Campaña principal: clase S, todos los modos. La clase S es la más pequeña de las NAS y permite comparar los cuatro modos en un tiempo razonable. Su papel principal es validar la corrección en todas las combinaciones y cuantificar el coste relativo de cada modo con un único hilo.

Parámetro	Valores
Python	3.15.0b1t
Benchmarks	CG, EP, FT, IS, MG
Clase	S
Modos	0, 1, 2, 3
Hilos	1, 2, 4, 8, 16, 32
Repeticiones	3

Campaña principal: clase W, modos compilados. La clase W representa una carga de tamaño intermedio. Se restringe a los modos 2 y 3 porque los modos 0 y 1 producen tiempos excesivamente altos en cargas de este tamaño y no aportan información adicional sobre paralelismo real.

Parámetro	Valores
Python	3.15.0b1t
Benchmarks	CG, EP, FT, IS, MG
Clase	W
Modos	2, 3
Hilos	1, 2, 4, 8, 16, 32
Repeticiones	3

Campaña principal: clase A, modo tipado. La clase A es la carga principal para analizar escalabilidad. En problemas de mayor tamaño, el cociente entre trabajo útil y overhead de paralelización es más favorable, por lo que esta campaña resulta más informativa. Se ejecuta únicamente el modo 3 porque las clases S y W ya muestran que es el único modo práctico para cargas grandes.

Parámetro	Valores
Python	3.15.0b1t
Benchmarks	CG, EP, FT, IS, MG
Clase	A
Modos	3
Hilos	1, 2, 4, 8, 16, 32
Repeticiones	3

Campaña comparativa entre versiones. La campaña comparativa mantiene el mismo código fuente y ejecuta Python 3.13.13t, Python 3.14.4t y Python 3.15.0b1t. Se limita a las clases S y W y a los modos 2 y 3 para concentrar el coste de cómputo en las configuraciones compiladas. La clase S usa cinco repeticiones por combinación y la clase W usa tres.

Parámetro	Valores
Python	3.13.13t, 3.14.4t, 3.15.0b1t
Benchmarks	CG, EP, FT, IS, MG
Clases	S, W
Modos	2, 3
Hilos	1, 2, 4, 8, 16, 32
Repeticiones	S: 5; W: 3

Campaña de profiling. Se lanzó una campaña específica de profiling con `perf stat` para relacionar los tiempos NAS con ocupación de CPU, IPC y eventos de memoria. Esta campaña no repite toda la matriz experimental, sino que se concentra en los casos necesarios para explicar por qué EP y FT escalan y por qué CG, IS y MG no obtienen aceleración sostenida. EP y FT se midieron en clase W, modo 3, con Python 3.14t y Python 3.15t, usando 1, 8, 16 y 32 hilos. CG, IS y MG se midieron en clase W, modo 3, con Python 3.13t, Python 3.14t y Python 3.15t, usando 1, 8 y 32 hilos.

Parámetro	Valores
Herramienta	<code>perf stat</code>
Benchmarks EP/FT	Python 3.14t y 3.15t; hilos 1, 8, 16, 32
Benchmarks CG/IS/MG	Python 3.13t, 3.14t y 3.15t; hilos 1, 8, 32
Clase y modo	W, modo 3
Métricas	CPUs utilizadas, ciclos, instrucciones, IPC y eventos de caché

Cada fila de profiling ejecuta primero una repetición de calentamiento y después una repetición medida con `perf stat`. Esta separación reduce el impacto de la compilación

Cython en las métricas de rendimiento. Aun así, las métricas de `perf stat` corresponden al proceso completo, no solo a la ventana temporizada por NAS. En benchmarks muy cortos, como IS clase W, esta limitación obliga a interpretar los contadores como apoyo diagnóstico y no como medida exacta del kernel.

4.5. Justificación de la restricción de modos en clases grandes

Los modos 0 y 1 no incluyen compilación Cython. En clase S, el modo 0 ya multiplica los tiempos del modo 3 por factores que van de $16\times$ en FT a $381\times$ en MG. Extender esos modos a clases W o A consumiría recursos de cluster desproporcionados sin añadir información relevante sobre la escalabilidad paralela, que es el objetivo de las clases mayores. Por ello, los modos 0 y 1 se evalúan de forma completa solo en clase S.

La misma lógica justifica no ejecutar modo 2 en clase A dentro de la campaña principal. El modo 2 es útil para cuantificar el efecto del tipado en clases S y W, pero sus tiempos absolutos en W ya muestran que no es una configuración práctica para cargas grandes. La comparación entre versiones se restringe a S y W por el mismo motivo: permite estudiar el efecto del intérprete sin multiplicar el coste de la campaña con clase A.

4.6. Métricas y tratamiento de los resultados

La métrica principal es el tiempo NAS interno, es decir, el tiempo que el propio benchmark mide entre el inicio y el fin del cómputo, excluyendo inicialización, E/S y verificación. Los Mop/s reportados por el benchmark se utilizan como métrica complementaria.

El análisis se basa en el mejor tiempo de las repeticiones válidas para cada combinación. Esta elección aproxima el tiempo de ejecución menos perturbado por ruido del sistema y evita que una interrupción puntual del nodo domine la comparación.

La variabilidad entre repeticiones es baja y respalda esta elección. Sobre las 207 combinaciones de la campaña principal con al menos dos ejecuciones válidas, el coeficiente de variación mediano del tiempo NAS es del 0.65 % y el 95 % de las combinaciones queda por debajo del 5 %. Los pocos casos con variación superior al 10 % corresponden a kernels de menos de 0.1 s (IS y MG en clases pequeñas), donde la resolución de dos decimales de la salida NAS domina el cociente. Por tanto, usar el mejor tiempo en lugar de la media no introduce un sesgo apreciable en las combinaciones con señal temporal relevante.

La fórmula de speedup utilizada es

$$\text{speedup}(t) = \frac{\text{mejor_tiempo}(1 \text{ hilo})}{\text{mejor_tiempo}(t \text{ hilos})} \quad (4.1)$$

El speedup se calcula siempre dentro del mismo modo y clase, de forma que el denominador y el numerador comparten el mismo modo de ejecución. Un valor inferior a 1 indica que la ejecución con más hilos fue más lenta que la ejecución secuencial equivalente en ese modo. El límite superior alcanzable está acotado por la fracción secuencial de cada benchmark, de acuerdo con la ley de Amdahl [16]: las reducciones, barreras y secciones críticas que no se reparten entre hilos fijan el techo de escalabilidad de cada kernel.

Los resultados se clasifican en tres categorías. La primera comprende ejecuciones correctas y medibles. La segunda agrupa ejecuciones correctas en las que el overhead domina al trabajo útil. La tercera recoge ejecuciones no viables por timeout, error o verificación incorrecta. Esta clasificación separa las limitaciones del algoritmo o del tamaño de clase de los fallos de implementación.

Capítulo 5

Resultados

Este capítulo presenta los resultados de la campaña principal ejecutada en Finisterrae III con Python 3.15.0b1t *free-threaded*. La campaña mide tres aspectos: la corrección funcional de los cinco benchmarks portados, el impacto de los modos de ejecución de OMP4Py y la escalabilidad al aumentar el número de hilos.

La campaña cubre CG, EP, FT, IS y MG. La clase S se ejecutó en los cuatro modos de ejecución para validar todos los caminos de código. La clase W se ejecutó en los modos compilados 2 y 3, porque los modos interpretados no son representativos para cargas mayores. La clase A se ejecutó únicamente en modo 3, ya que las clases menores muestran que el tipado Cython es condición necesaria para obtener tiempos prácticos. En todos los casos se usaron 1, 2, 4, 8, 16 y 32 hilos, con tres repeticiones por combinación.

La métrica principal es el tiempo NAS interno reportado por cada benchmark. Para cada combinación se toma el mejor tiempo entre las repeticiones válidas. El tiempo externo del proceso se conserva en los ficheros de resultados, pero no se usa como métrica principal porque incluye arranque del intérprete, importación, compilación y escritura de logs.

Campaña	Ejecuciones	Correctas	Éxito	Alcance
Campaña principal Python 3.15t	630	630	100 %	S: modos 0–3; W: modos 2–3; A: modo 3
Comparación de versiones	1440	1440	100 %	Python 3.13t, 3.14t y 3.15t; clases S y W; modos 2–3

Tabla 5.1: Cobertura de las campañas finales ejecutadas en Finisterrae III.

La Tabla 5.1 resume la cobertura final. Las 630 ejecuciones de la campaña principal terminaron con código de retorno cero y verificación NAS SUCCESSFUL. Por tanto, el análisis de rendimiento se realiza sobre implementaciones funcionalmente correctas.

5.1. Efecto del modo de ejecución en clase S

La clase S permite comparar los cuatro modos de OMP4Py con bajo coste. El modo 0 conserva la ejecución Python de referencia, el modo 1 aplica la transformación OMP4Py sin

compilar los kernels, el modo 2 genera código Cython sin tipos explícitos y el modo 3 añade tipado Cython y *memoryviews*.

Benchmark	M0 (s)	M1 (s)	M2 (s)	M3 (s)	M0/M3	M2/M3
CG	18.23	18.23	16.35	0.13	140.23×	125.77×
EP	51.97	50.45	42.32	1.44	36.09×	29.39×
FT	38.76	38.70	36.77	2.41	16.08×	15.26×
IS	0.33	0.33	0.30	0.00	—	—
MG	3.81	3.81	3.24	0.01	381.00×	324.00×

Tabla 5.2: Mejor tiempo NAS en clase S con un hilo para los cuatro modos de ejecución en Python 3.15t.

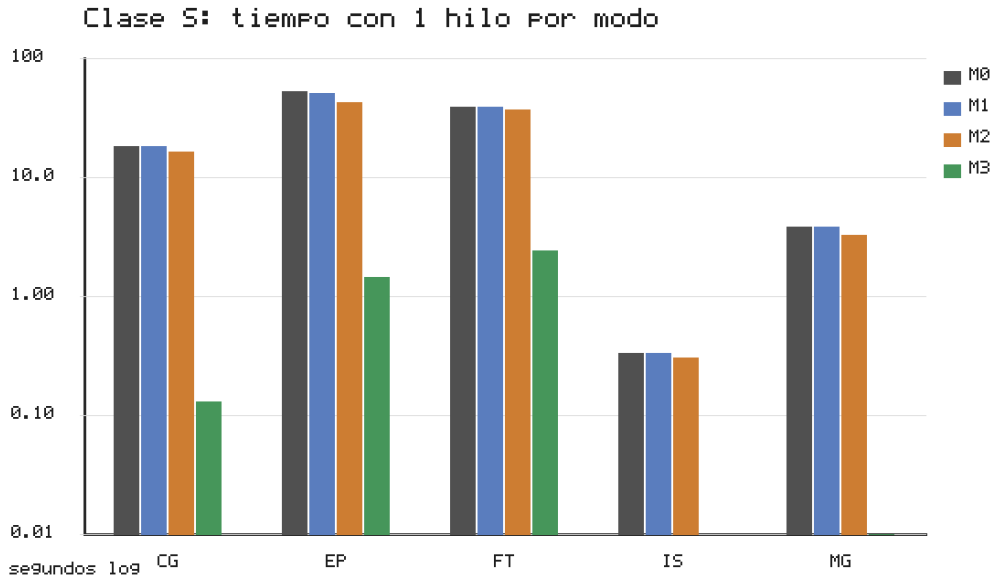


Figura 5.1: Tiempo NAS con un hilo en clase S para los cuatro modos de ejecución. El eje vertical usa escala logarítmica.

La Tabla 5.2 y la Figura 5.1 muestran que el modo 3 obtiene el menor tiempo con un hilo en todos los benchmarks. Frente al modo 0, los factores son 140.23× en CG, 36.09× en EP, 16.08× en FT y 381.00× en MG. IS aparece como 0.00 s en modo 3 porque el tiempo NAS se imprime con dos decimales y la clase S queda por debajo de esa resolución.

La diferencia entre modo 2 y modo 3 confirma que el cuello principal no es solo la compilación, sino el tipado. En CG, el modo 2 tarda 16.35 s y el modo 3 0.13 s. En MG, el modo 2 tarda 3.24 s y el modo 3 0.01 s. EP y FT muestran el mismo patrón con factores de 29.39× y 15.26×. Sin anotaciones de tipo, Cython conserva demasiada semántica de objetos Python dentro de los bucles críticos. Con el modo 3, esos bucles pasan a ejecutarse como código nativo efectivo.

Los modos 0 y 1 quedan prácticamente superpuestos con un hilo. La transformación OMP4Py, por sí sola, no aporta rendimiento cuando no se combina con compilación y tipado. En clase S, su utilidad principal es validar que el código portado mantiene caminos funcionales para todos los modos.

5.2. Escalabilidad en clase S

La clase S no es la carga principal para extraer conclusiones de escalabilidad, pero permite detectar qué kernels tienen suficiente trabajo incluso en tamaños pequeños y qué modos introducen overhead dominante.

Benchmark	M0	M1	M2	M3
CG	0.96×	0.96×	2.34×	0.15×
EP	23.84×	29.16×	24.05×	9.00×
FT	4.00×	4.05×	4.79×	9.64×
IS	1.00×	1.00×	1.25×	—
MG	2.10×	2.23×	8.53×	0.05×

Tabla 5.3: Speedup a 32 hilos respecto a 1 hilo en clase S para cada modo de Python 3.15t.

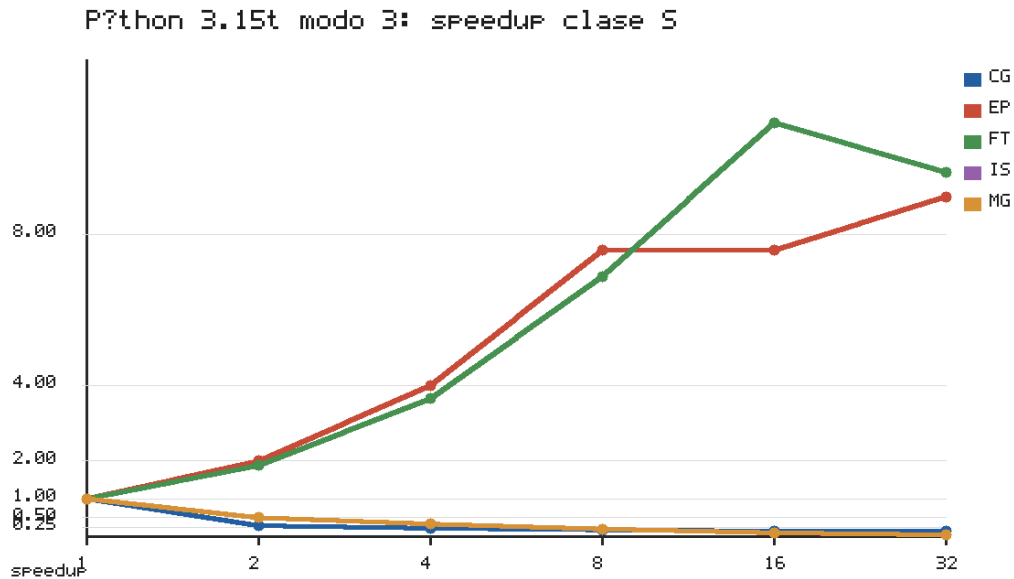


Figura 5.2: Speedup en clase S, modo 3, Python 3.15t. La línea discontinua marca el umbral de speedup 1.

La Tabla 5.3 muestra que EP y FT ya obtienen speedups apreciables a 32 hilos en clase S. En modo 3, EP alcanza 9.00× y FT 9.64×. En EP, los modos 0, 1 y 2 también muestran speedups

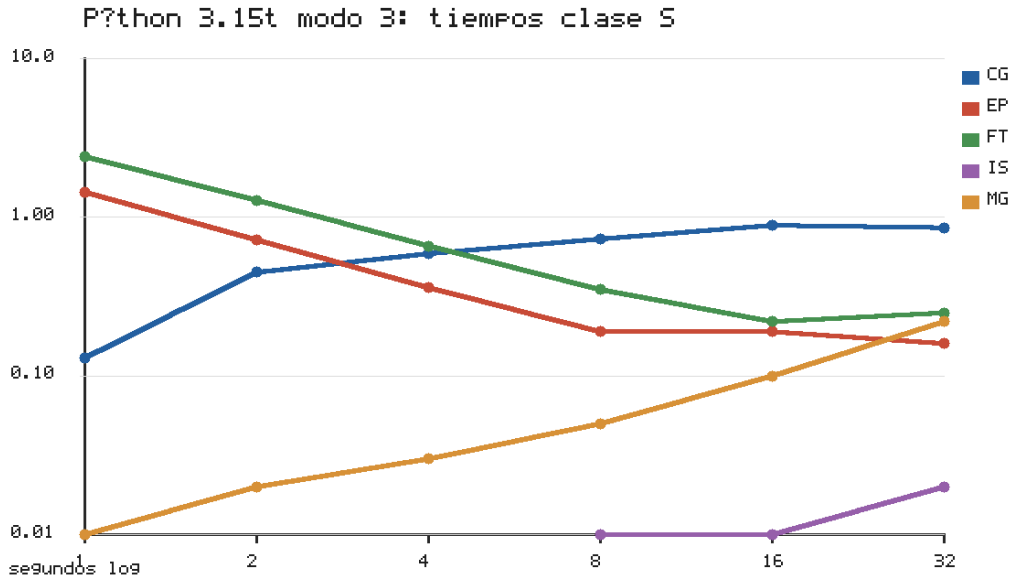


Figura 5.3: Tiempo NAS por número de hilos en clase S, modo 3, Python 3.15t. Cada línea corresponde a un benchmark.

altos, señal de que el kernel dispone de trabajo paralelo suficiente incluso en la clase pequeña.

El caso de EP en los modos interpretados merece una lectura propia. En modo 1, EP alcanza $29.16\times$ a 32 hilos, una aceleración casi lineal sobre código Python interpretado. Este resultado es la evidencia más directa de la campaña de que el intérprete *free-threaded* cumple su objetivo. Sin el GIL, los hilos Python ejecutan trabajo en paralelo real sin necesidad de compilación alguna. Que el speedup relativo del modo 3 sea menor ($9.00\times$) no contradice esa lectura. El tipado acelera el cómputo pero no la sincronización, por lo que la fracción de espera pesa más cuanto más rápido es el kernel. La comparación en tiempo absoluto mantiene, no obstante, la conclusión práctica. El modo 3 con un hilo (1.44 s) ya es más rápido que el modo 1 con 32 hilos (aproximadamente 1.7 s), de modo que la escalabilidad casi ideal del modo interpretado tiene valor diagnóstico, no práctico.

El comportamiento no es uniforme. CG y MG empeoran en modo 3, con $0.15\times$ y $0.05\times$ respectivamente. IS no permite calcular un speedup fiable en modo 3 porque el tiempo base aparece redondeado a 0.00 s. La clase S confirma, por tanto, una división que se mantiene en clases mayores. EP y FT son los candidatos naturales a escalar, mientras que CG, IS y MG están más condicionados por sincronización, granularidad o accesos irregulares.

La lectura metodológica es que la clase S sirve para validar todos los caminos y para observar tendencias iniciales, pero las conclusiones prácticas deben apoyarse en W y A. En tamaños tan pequeños, pequeñas diferencias de tiempo o de redondeo pueden afectar mucho

a los cocientes.

5.3. Clase W: modos compilados

La clase W aumenta el tamaño del problema y restringe la comparación a los modos 2 y 3. Esta decisión evita dedicar recursos a modos interpretados que ya son varios órdenes de magnitud más lentos en clase S.

Benchmark	M2 (s)	M3 (s)	M2/M3
CG	105.6	0.78	135.42×
EP	84.53	2.87	29.45×
FT	80.07	5.04	15.89×
IS	5.54	0.03	184.67×
MG	198.2	0.45	440.51×

Tabla 5.4: Mejor tiempo NAS en clase W con un hilo para los modos compilados de Python 3.15t.

La Tabla 5.4 mantiene la misma conclusión de clase S: el modo 3 es la única configuración con tiempos prácticos. Con un hilo, el modo 2 tarda 105.6 s en CG frente a 0.78 s en modo 3. En MG la diferencia es de 198.2 s frente a 0.45 s. EP y FT mantienen factores de mejora cercanos a 29.45× y 15.89×, e IS mejora de 5.54 s a 0.03 s.

Benchmark	M2	M3
CG	7.77×	0.14×
EP	26.09×	9.90×
FT	4.37×	8.00×
IS	0.81×	0.43×
MG	23.68×	0.57×

Tabla 5.5: Speedup a 32 hilos respecto a 1 hilo en clase W para los modos compilados de Python 3.15t.

La escalabilidad en clase W separa los benchmarks en dos grupos. En modo 3, EP alcanza 9.90× a 32 hilos y FT 8.00×. FT obtiene su mejor punto con 16 hilos, donde baja de 5.04 s a 0.42 s, equivalente a 12.00×. EP alcanza su mejor tiempo con 32 hilos, bajando de 2.87 s a 0.29 s.

CG, IS y MG no escalan en modo 3. CG cae a 0.14×, IS a 0.43× y MG a 0.57× a 32 hilos. El modo 2 sí muestra speedups altos en varios de estos casos, por ejemplo 7.77× en CG y 23.68× en MG, pero sus tiempos absolutos siguen siendo muy superiores a los del modo 3 con un hilo. Por ello, el speedup aislado no basta para escoger configuración: un modo puede escalar mejor en términos relativos y seguir siendo peor en tiempo absoluto.

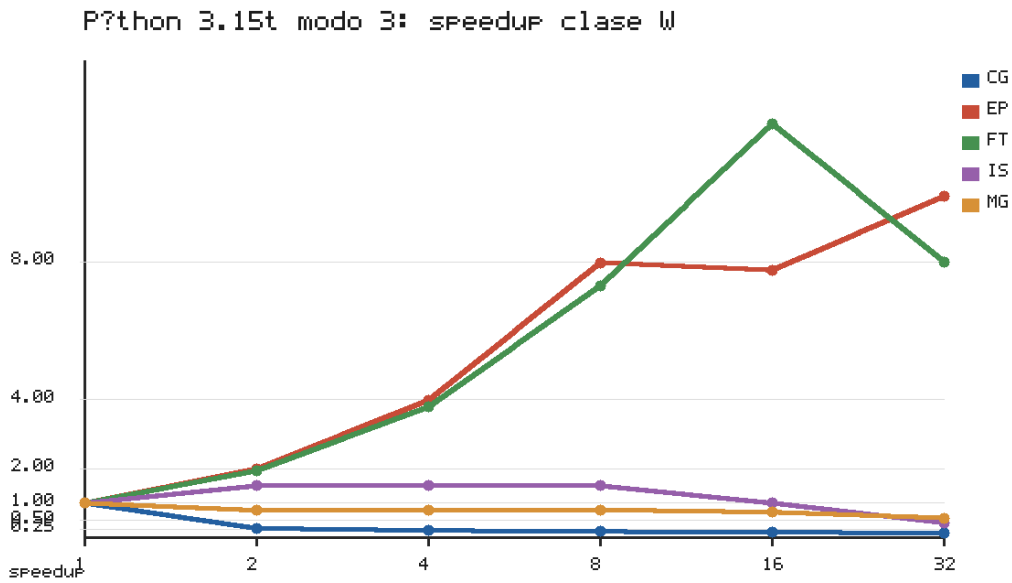


Figura 5.4: Speedup en clase W, modo 3, Python 3.15t.

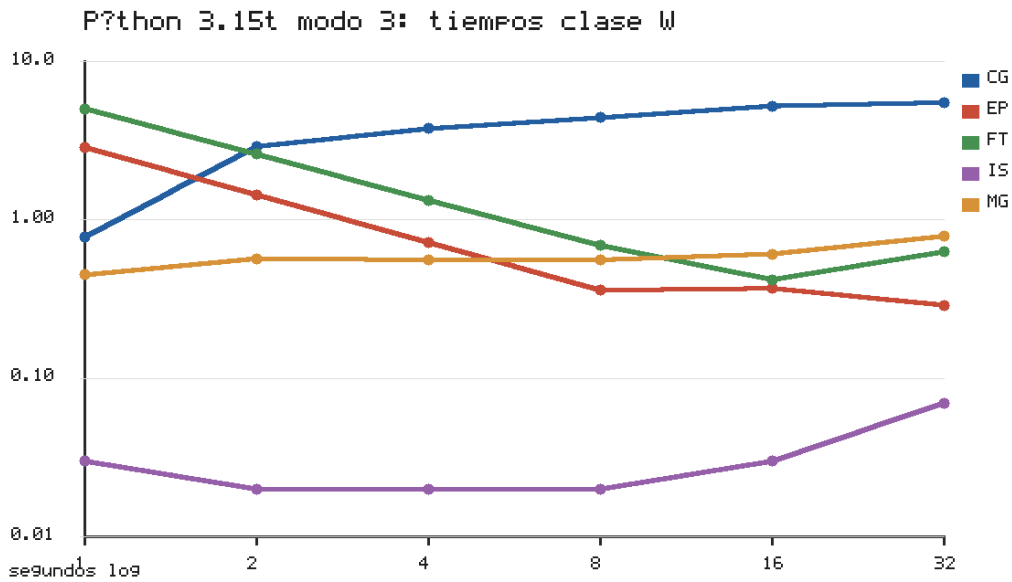


Figura 5.5: Tiempo NAS por número de hilos en clase W, modo 3, Python 3.15t. Cada línea corresponde a un benchmark.

5.4. Clase A: evaluación principal de escalabilidad

La clase A es la carga de mayor tamaño de la campaña principal. Se ejecutó solo en modo 3 porque es la única configuración que produce tiempos prácticos en las clases anteriores.

Benchmark	1	2	4	8	16	32
CG	2.76	10.29	14.35	15.98	18.91	21.17
EP	22.99	11.48	5.75	2.88	2.90	2.19
FT	58.63	30.05	15.26	7.70	3.99	4.18
IS	0.36	0.21	0.15	0.18	0.24	0.44
MG	3.57	4.34	4.14	3.96	3.79	4.56

Tabla 5.6: Mejor tiempo NAS en clase A, modo 3, Python 3.15t.

Benchmark	1	2	4	8	16	32
CG	1.00×	0.27×	0.19×	0.17×	0.15×	0.13×
EP	1.00×	2.00×	4.00×	7.98×	7.93×	10.50×
FT	1.00×	1.95×	3.84×	7.61×	14.69×	14.03×
IS	1.00×	1.71×	2.40×	2.00×	1.50×	0.82×
MG	1.00×	0.82×	0.86×	0.90×	0.94×	0.78×

Tabla 5.7: Speedup por número de hilos en clase A, modo 3, Python 3.15t.

Los resultados de clase A son los más representativos para valorar el port en cargas grandes. EP baja de 22.99 s con un hilo a 2.19 s con 32 hilos, lo que supone $10.50\times$. FT baja de 58.63 s a 4.18 s con 32 hilos ($14.03\times$), y su mejor punto está en 16 hilos con 3.99 s ($14.69\times$). Estos dos benchmarks muestran que Python 3.15t, OMP4Py y el código Cython tipado pueden producir escalabilidad real cuando el kernel ofrece trabajo paralelo suficiente y una estructura adecuada.

IS obtiene una mejora moderada con pocos hilos: pasa de 0.36 s a 0.15 s con 4 hilos, equivalente a $2.40\times$. Sin embargo, la curva empeora después y a 32 hilos cae a $0.82\times$. El tamaño de clase A permite amortizar parte del coste inicial, pero no sostiene la eficiencia al aumentar la concurrencia.

CG y MG no escalan en clase A. CG empeora desde 2.76 s con un hilo hasta 21.17 s con 32 hilos. MG sube de 3.57 s a 4.56 s. En CG, las reducciones y sincronizaciones frecuentes del gradiente conjugado limitan el trabajo independiente entre barreras. En MG, los niveles de malla tienen granularidad desigual y combinan fases de cómputo con accesos intensivos a memoria.

5.5. Síntesis global del modo 3

Como el modo 3 es el único que produce tiempos competitivos en todas las clases, la síntesis global se centra en esa configuración.

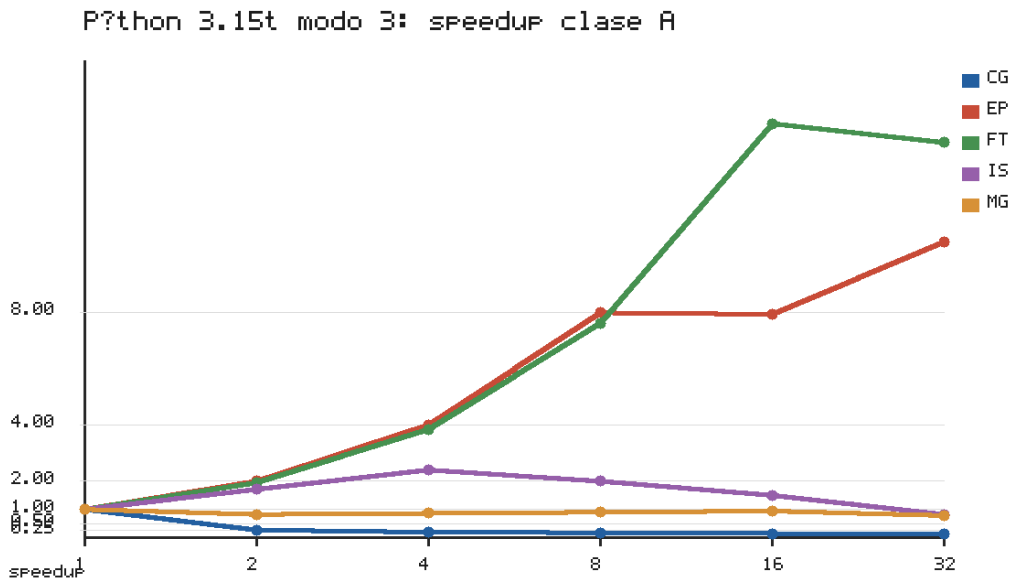


Figura 5.6: Speedup en clase A, modo 3, Python 3.15t.

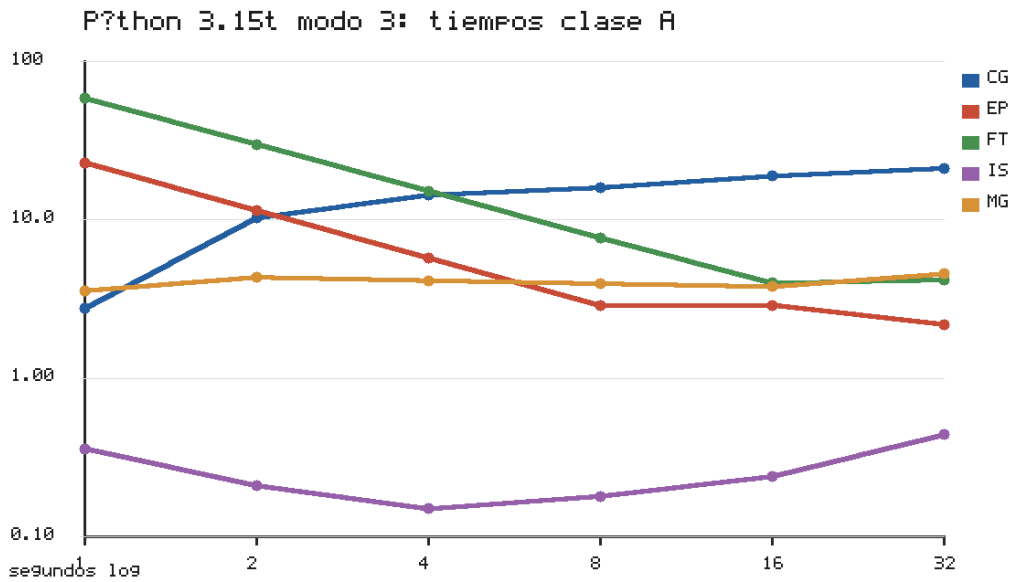


Figura 5.7: Tiempo NAS por número de hilos en clase A, modo 3, Python 3.15t. Cada línea corresponde a un benchmark.

Clase	Bench.	Mejor hilo	Mejor tiempo (s)	T1 (s)	Speedup ópt.	T32 (s)	Speedup32
S	CG	1	0.13	0.13	1.00×	0.86	0.15×
S	EP	32	0.16	1.44	9.00×	0.16	9.00×
S	FT	16	0.22	2.41	10.95×	0.25	9.64×
S	IS	1	0.00	0.00	—	0.02	0.00×
S	MG	1	0.01	0.01	1.00×	0.22	0.05×
W	CG	1	0.78	0.78	1.00×	5.51	0.14×
W	EP	32	0.29	2.87	9.90×	0.29	9.90×
W	FT	16	0.42	5.04	12.00×	0.63	8.00×
W	IS	2	0.02	0.03	1.50×	0.07	0.43×
W	MG	1	0.45	0.45	1.00×	0.79	0.57×
A	CG	1	2.76	2.76	1.00×	21.17	0.13×
A	EP	32	2.19	22.99	10.50×	2.19	10.50×
A	FT	16	3.99	58.63	14.69×	4.18	14.03×
A	IS	4	0.15	0.36	2.40×	0.44	0.82×
A	MG	1	3.57	3.57	1.00×	4.56	0.78×

Tabla 5.8: Mejor número de hilos y speedup asociado en modo 3, Python 3.15t.

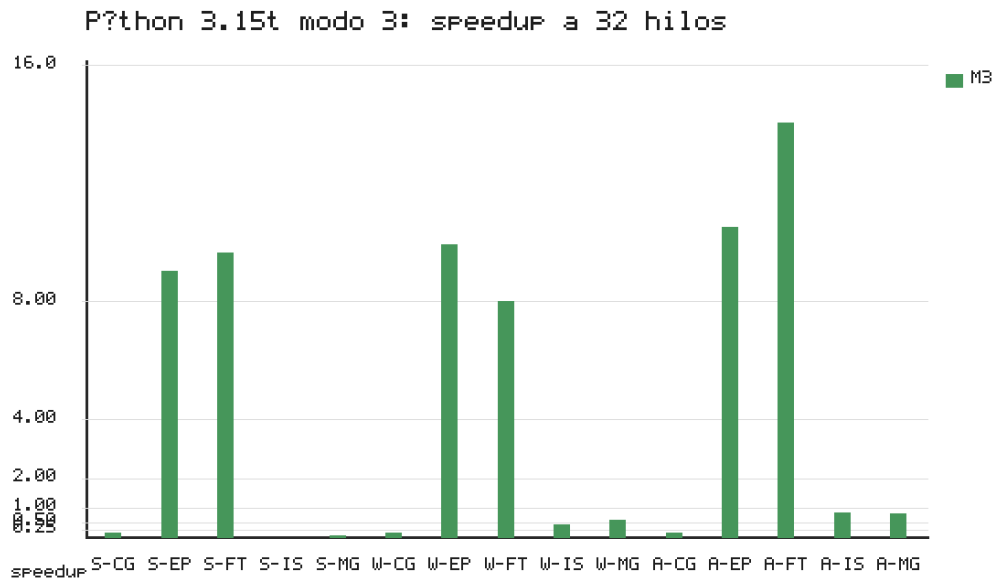


Figura 5.8: Speedup a 32 hilos en modo 3 para las clases S, W y A.

La Tabla 5.8 resume el mejor número de hilos en cada combinación. EP alcanza su mejor punto en 32 hilos en las tres clases, con $9.00\times$ en S, $9.90\times$ en W y $10.50\times$ en A. FT alcanza su mejor punto en 16 hilos, con $10.95\times$ en S, $12.00\times$ en W y $14.69\times$ en A. IS solo muestra una mejora clara en A hasta 4 hilos. CG y MG mantienen el óptimo en un hilo en las tres clases.

Clase	Bench.	T1 (s)	T32 (s)	Speedup32	Mop/s32
S	CG	0.13	0.86	$0.15\times$	77.9
S	EP	1.44	0.16	$9.00\times$	213
S	FT	2.41	0.25	$9.64\times$	702
S	IS	0.00	0.02	—	27.7
S	MG	0.01	0.22	$0.05\times$	33.9
W	CG	0.78	5.51	$0.14\times$	76.3
W	EP	2.87	0.29	$9.90\times$	229
W	FT	5.04	0.63	$8.00\times$	592
W	IS	0.03	0.07	$0.43\times$	158
W	MG	0.45	0.79	$0.57\times$	618
A	CG	2.76	21.17	$0.13\times$	70.7
A	EP	22.99	2.19	$10.50\times$	245
A	FT	58.63	4.18	$14.03\times$	1706
A	IS	0.36	0.44	$0.82\times$	191
A	MG	3.57	4.56	$0.78\times$	854

Tabla 5.9: Síntesis de tiempos, speedup y Mop/s a 32 hilos en modo 3, Python 3.15t.

La Tabla 5.9 añade tiempos y Mop/s a 32 hilos. Los Mop/s refuerzan la lectura de EP y FT: FT clase A alcanza 1706 Mop/s y EP clase A 245 Mop/s con speedups superiores a $10\times$. En MG, en cambio, la cifra de 854 Mop/s a 32 hilos no implica escalabilidad, porque el tiempo empeora frente a un hilo. Por ello, el análisis combina siempre tiempo absoluto, speedup y Mop/s.

5.6. Curvas por benchmark

Las Figuras 5.9, 5.10, 5.11, 5.12 y 5.13 reorganizan la misma información por benchmark. En estas figuras cada línea corresponde a una clase de problema. Esta vista facilita comparar si el aumento de tamaño cambia la forma de la curva de escalabilidad.

La lectura por benchmark confirma la separación observada en las tablas. EP mantiene una

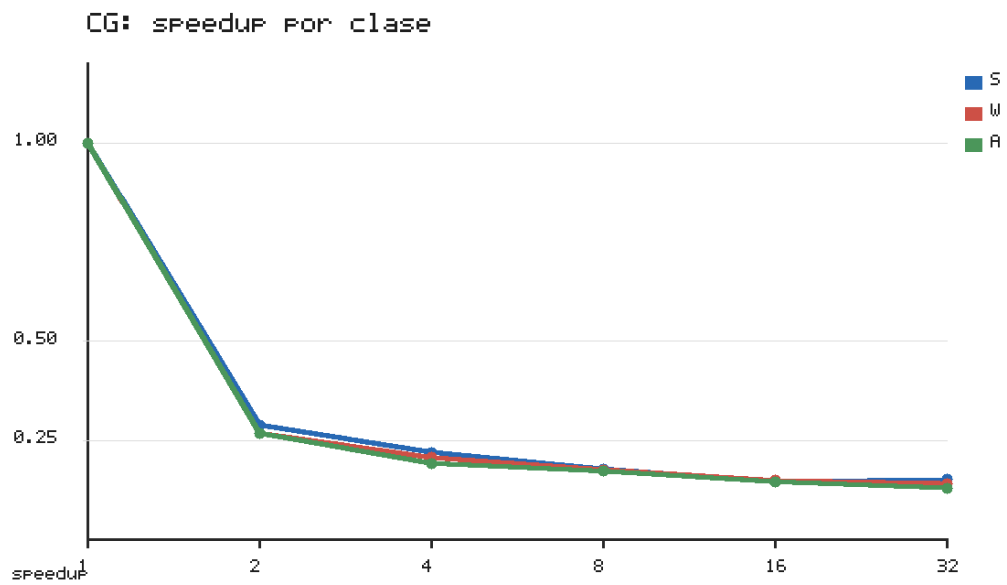


Figura 5.9: CG: speedup por clase en modo 3, Python 3.15t.

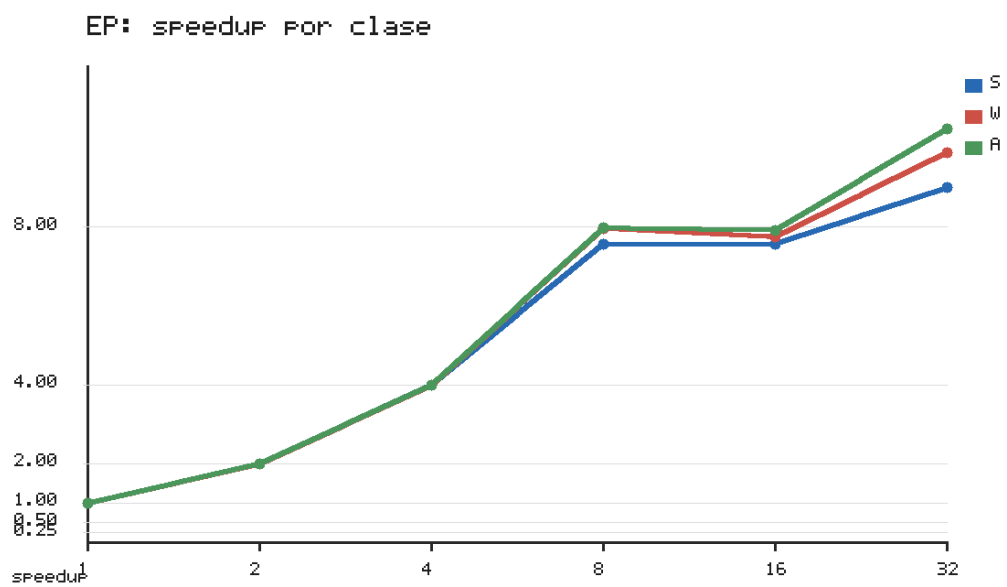


Figura 5.10: EP: speedup por clase en modo 3, Python 3.15t.

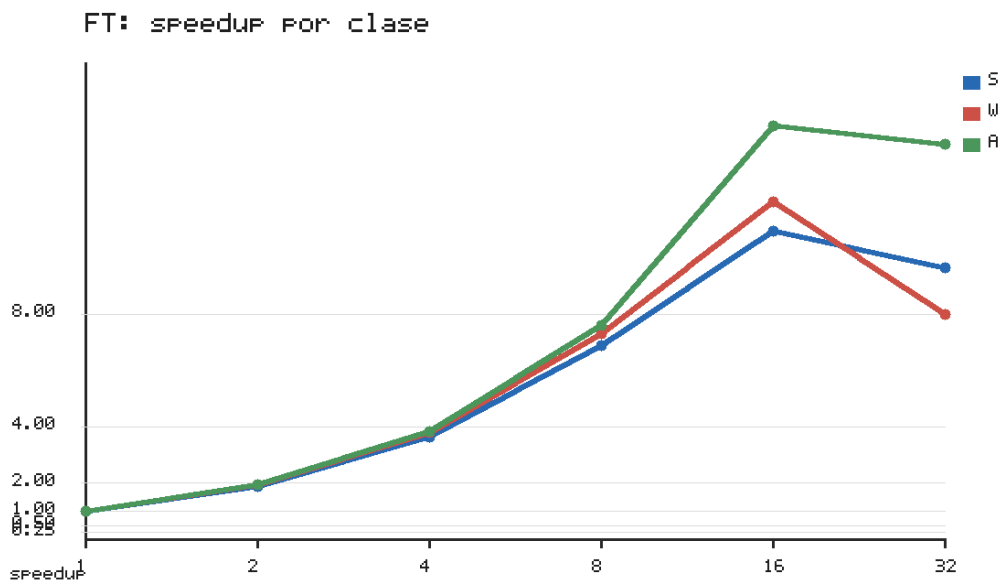


Figura 5.11: FT: speedup por clase en modo 3, Python 3.15t.

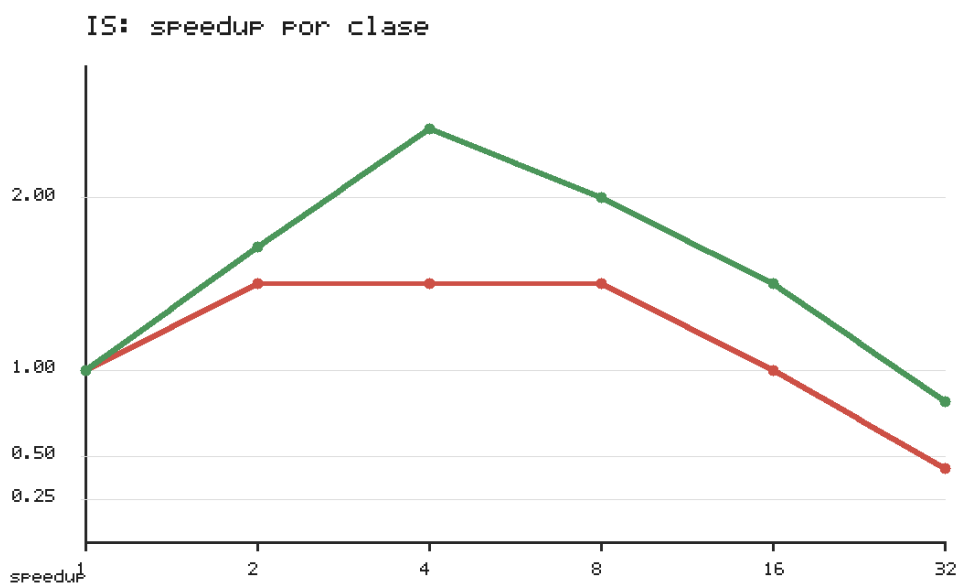


Figura 5.12: IS: speedup por clase en modo 3, Python 3.15t.

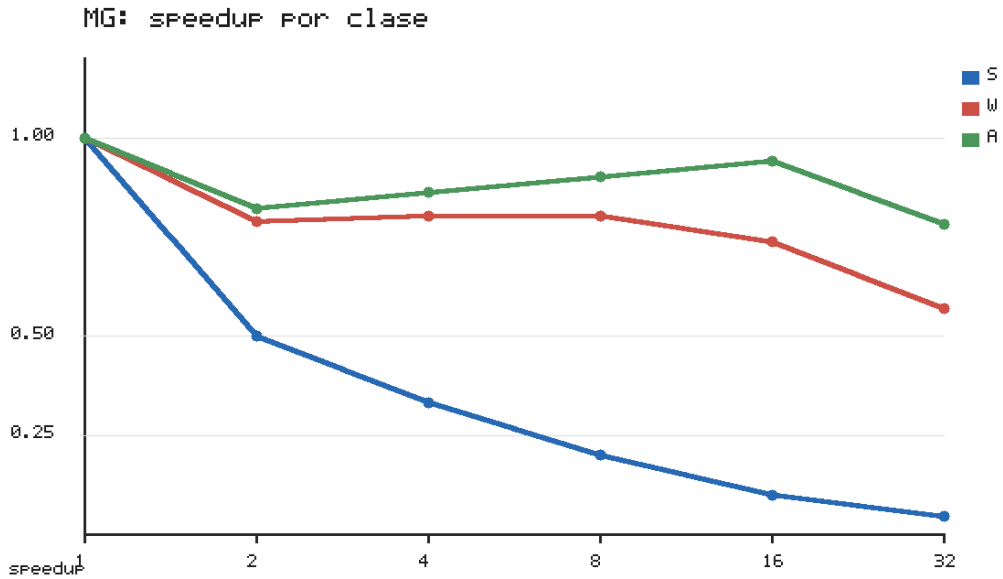


Figura 5.13: MG: speedup por clase en modo 3, Python 3.15t.

curva creciente en S, W y A. FT también mejora con el tamaño, aunque su máximo aparece antes de 32 hilos. IS solo se beneficia de forma clara en clase A y con pocos hilos. CG y MG no mejoran al aumentar la clase, lo que refuerza que su límite no es solo el tamaño del problema, sino la estructura de sincronización y acceso a memoria.

5.7. Interpretación por benchmark

CG. CG se beneficia enormemente del tipado, pero no del aumento de hilos en modo 3. En clase W, el modo 3 reduce el tiempo con un hilo de 105.6 s a 0.78 s frente al modo 2. En clase A, sin embargo, 32 hilos elevan el tiempo a 21.17 s. La causa más probable es la frecuencia de reducciones globales, productos escalares, normas y barreras propias del gradiente conjugado.

EP. EP es el caso más favorable. Su estructura embarzosamente paralela se traduce en speedups altos en las tres clases: $9.00\times$ en S, $9.90\times$ en W y $10.50\times$ en A a 32 hilos. El resultado muestra que el port de EP no se limita a mejorar por tipado, sino que también explota paralelismo de memoria compartida de forma efectiva.

FT. FT también escala, aunque su punto óptimo aparece antes de 32 hilos. En clase A, el mejor resultado está en 16 hilos con $14.69\times$, mientras que a 32 hilos queda en $14.03\times$. La FFT combina cómputo numérico con accesos a memoria y sincronizaciones entre fases. Esto

explica que añadir más hilos siga ayudando hasta cierto punto, pero que el máximo no sea necesariamente el mayor recuento evaluado.

IS. IS obtiene tiempos absolutos muy bajos y una mejora moderada en clase A hasta 4 hilos, pero no mantiene la escalabilidad a 32 hilos. La ordenación entera incluye generación de claves, histogramas, acumulaciones y redistribución, con patrones de acceso que no ofrecen la misma granularidad que EP o FT.

MG. MG es uno de los benchmarks con mayor mejora por tipado, pero no escala en modo 3. En clase W, el modo 2 tarda 198.2 s con un hilo y el modo 3 0.45 s. Aun así, 32 hilos elevan el tiempo a 0.79 s en W y 4.56 s en A. El patrón es coherente con la estructura multigrad: niveles de distinto tamaño, accesos intensivos a memoria y sincronizaciones entre restricción, interpolación, residuo y norma.

5.8. Limitaciones de la medida

La primera limitación es la resolución temporal de la salida NAS. Algunos tiempos de clase S, especialmente en IS y MG, aparecen como 0.00 s o 0.01 s. Esos valores son útiles para confirmar tiempos absolutos bajos, pero no para calcular ratios finos.

La segunda limitación es el uso del mejor tiempo de tres repeticiones. Esta decisión reduce el efecto de ruido externo del cluster, pero no sustituye a una campaña estadística más amplia. Para el objetivo del TFM, la señal principal es suficientemente clara: el modo 3 mejora drásticamente el tiempo absoluto, EP y FT escalan, y CG, IS y MG presentan límites de paralelización.

La tercera limitación es que la clase A solo se ejecutó en modo 3. No se dispone de medidas de modo 2 para A en la campaña final. La restricción fue deliberada porque el modo 2 ya había demostrado tiempos demasiado altos en clase W. Por tanto, las conclusiones de clase A se refieren a la escalabilidad del mejor modo disponible.

5.9. Conclusiones del capítulo

La campaña principal deja cuatro conclusiones. Primero, las cinco adaptaciones son funcionalmente correctas en todos los caminos evaluados. Segundo, el modo 3 reduce los tiempos frente al modo 2 en factores que van de $15\times$ en FT a $440\times$ en MG. Tercero, Python 3.15t obtiene escalabilidad significativa en EP y FT, con speedups superiores a $10\times$ en clase A. Cuarto, esa escalabilidad no es general. CG sigue limitado por sus reducciones y barreras, IS por su fase secuencial de prefijos y MG por la granularidad desigual de los niveles de malla.

El Capítulo 6 utiliza `perf stat` para relacionar los speedups observados con ocupación real de CPU, IPC y eventos de memoria. El Capítulo 7 compara estos resultados con Python 3.13t y Python 3.14t.

Profiling y diagnóstico

6.1. Objetivo del profiling

Los capítulos anteriores establecen dos hechos: el modo 3 reduce de forma marcada los tiempos frente al modo 2, y solo EP y FT obtienen aceleración sostenida. El objetivo de este capítulo es relacionar esos resultados con métricas de ejecución de bajo nivel: ocupación de CPU, IPC y eventos de caché. El diagnóstico combina dos herramientas: `perf stat`, que mide el uso de recursos a nivel de proceso, y el nuevo sistema de muestreo de Python 3.15 (Sección 6.7), que atribuye el tiempo de sincronización y el balance de carga por función e hilo.

El profiling se realizó con `perf stat`. Esta herramienta mide el proceso completo y, por tanto, no aísla únicamente la ventana temporizada por el benchmark NAS. Aun así, proporciona una base homogénea para comparar Python 3.14t y Python 3.15t en el mismo entorno. Las métricas principales son el número medio de CPUs utilizadas, ciclos, instrucciones, IPC y fallos de caché.

La campaña de profiling se ejecutó en clase W y modo 3. Para EP y FT se midieron Python 3.14t y Python 3.15t con 1, 8, 16 y 32 hilos. Para CG, IS y MG se midieron Python 3.13t, 3.14t y 3.15t con 1, 8 y 32 hilos. Cada medición incluyó una ejecución de calentamiento previa para reducir el efecto de la compilación Cython sobre los contadores.

Las métricas de `perf stat` deben interpretarse con cautela en benchmarks cortos. IS clase W tarda centésimas de segundo. En ese caso, el arranque del proceso y las bibliotecas importadas pueden pesar tanto como el kernel. También se observan valores de CPUs utilizadas superiores a uno en algunas ejecuciones con un hilo, precisamente porque `perf` mide el proceso completo y no solo la región paralela.

6.2. Control de entorno y afinidad

Durante la fase de diagnóstico se detectó que las campañas preliminares fijaban afinidad de OpenMP mediante variables como `OMP_PROC_BIND` y `OMP_PLACES`. Esa configuración producía una serialización aparente en algunas ejecuciones de Python 3.15t, especialmente en EP y FT. Las campañas finales eliminan esas variables por defecto en los scripts de ejecución y solo permiten activarlas explícitamente para experimentos controlados.

Este punto cambia la interpretación del trabajo. La falta de escalabilidad observada inicialmente en Python 3.15t no se reproduce en las campañas finales: con la afinidad corregida, Python 3.15t escala en EP y FT de forma comparable a Python 3.13t y Python 3.14t. Por tanto, el análisis final no atribuye a Python 3.15t una incapacidad general para ejecutar estos kernels en paralelo.

También se realizaron pruebas de aislamiento. `sys._is_gil_enabled()` se mantuvo en `False` antes y después de importar OMP4Py, Cython y los módulos compilados. Además, un microbenchmark con hilos Python puros y un kernel OMP4Py mínimo usaron varias CPUs en Python 3.14t y Python 3.15t. Estas pruebas apoyan la interpretación de que los problemas restantes pertenecen a los kernels concretos o a su interacción con el runtime, no a una reactivación implícita del GIL.

6.3. Cuellos de botella corregidos durante el port

Antes de la campaña final se corrigieron varios cuellos de botella de código. En EP, la función `vranlc`, encargada de generar números pseudoaleatorios, se llamaba desde el kernel compilado como función Python externa. Esa llamada impedía que el modo 2 compilase todo el cuerpo relevante. La solución fue incluir en `EP_Python.py` versiones localmente compilables de `vranlc`, una sin tipos y otra tipada.

En IS se revisó la inicialización de la secuencia de claves para que cada modo ejecutase la ruta correcta y no quedase una parte secuencial accidental en modo híbrido. En MG se redujo la dependencia de estado global en kernels como `zran3` y `norm2u3`, pasando los datos necesarios de forma explícita para facilitar la compilación efectiva por Cython.

Estas correcciones son relevantes porque separan los límites del port de los límites del runtime. Los resultados finales miden versiones donde los kernels críticos están dentro del propio fichero del benchmark y son compilables por OMP4Py/Cython.

6.4. EP y FT: kernels que sí escalan

La Tabla 6.1 confirma que EP y FT ejecutan trabajo en paralelo en Python 3.14t y Python 3.15t. En EP, Python 3.14t baja de 2.87 s a 0.29 s con 32 hilos, con $9.90\times$ de speedup. Python 3.15t

Bench.	Python	Hilos	NAS (s)	Speedup	CPUs usadas	IPC	Fallos cache (%)
EP	3.14t	1	2.87	1.00×	1.90	0.86	13.9
EP	3.14t	8	0.36	7.97×	3.09	1.02	22.0
EP	3.14t	16	0.37	7.76×	5.08	0.86	14.1
EP	3.14t	32	0.29	9.90×	4.68	0.90	33.3
EP	3.15t	1	2.87	1.00×	0.97	1.09	14.8
EP	3.15t	8	0.37	7.76×	2.47	1.02	44.8
EP	3.15t	16	0.48	5.98×	2.55	1.09	19.3
EP	3.15t	32	0.48	5.98×	2.55	1.09	18.9
FT	3.14t	1	5.00	1.00×	1.30	2.11	13.7
FT	3.14t	8	0.67	7.46×	5.45	1.86	8.5
FT	3.14t	16	0.42	11.90×	6.81	1.77	8.1
FT	3.14t	32	0.64	7.81×	7.25	1.55	40.5
FT	3.15t	1	4.90	1.00×	0.97	2.71	14.0
FT	3.15t	8	0.70	7.00×	3.57	2.49	30.0
FT	3.15t	16	0.41	11.95×	4.38	2.38	8.2
FT	3.15t	32	0.64	7.66×	5.72	1.88	44.2

Tabla 6.1: Perfil con perf stat de EP y FT en clase W, modo 3.

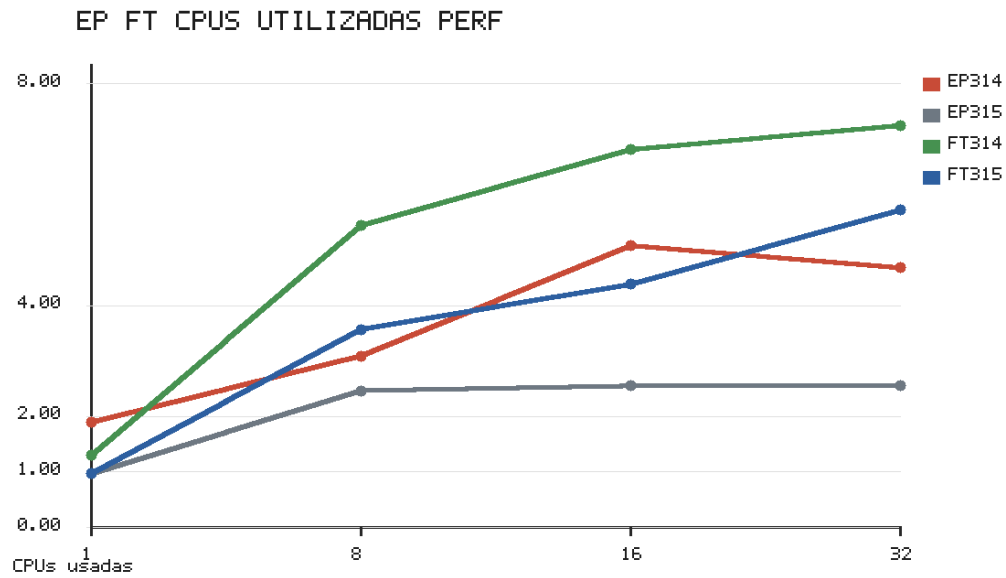


Figura 6.1: CPUs utilizadas por EP y FT en clase W, modo 3, según perf stat.

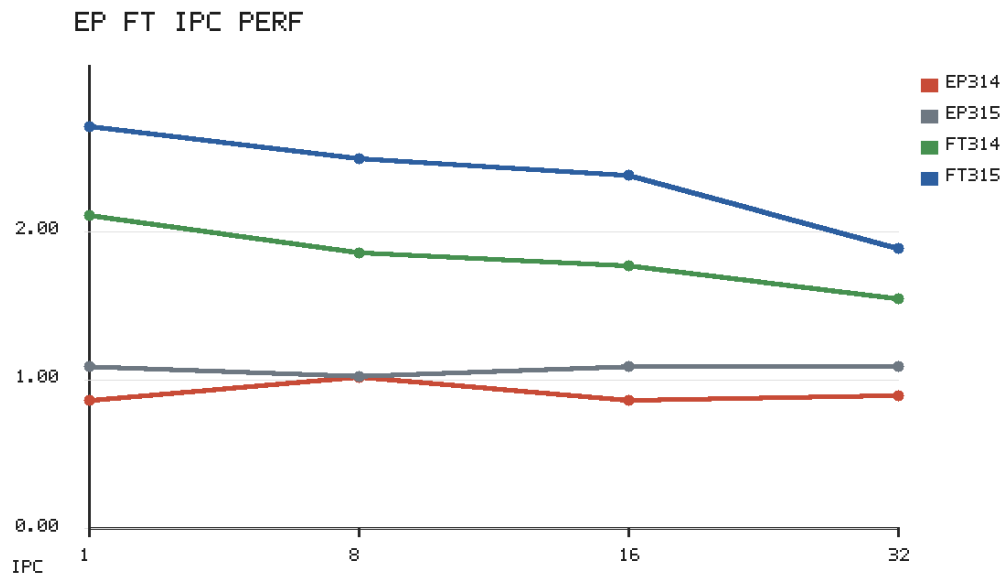


Figura 6.2: IPC de EP y FT en clase W, modo 3, según perf stat.

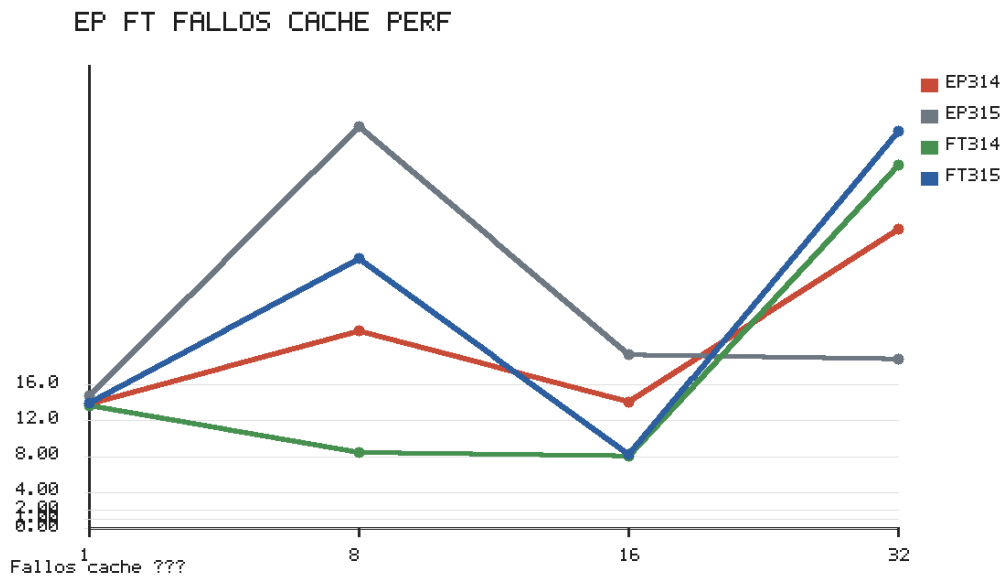


Figura 6.3: Porcentaje de fallos de caché de EP y FT en clase W, modo 3, según perf stat.

baja de 2.87 s a 0.37 s con 8 hilos en esta campaña de profiling y mantiene speedups por encima de $5\times$ en 16 y 32 hilos. La campaña principal, sin `perf`, obtiene para EP clase W un mejor tiempo de 0.29 s a 32 hilos, por lo que el profiling debe leerse como diagnóstico y no como sustituto de la medida principal.

La ocupación de CPU respalda esa lectura. Python 3.14t usa varias CPUs en EP: 3.09 con 8 hilos, 5.08 con 16 y 4.68 con 32. Python 3.15t también supera una CPU en las ejecuciones paralelas, aunque con menor ocupación media en EP (2.47–2.55 CPUs). Esto explica que escale, pero con una eficiencia inferior en la medición con `perf`.

FT presenta un patrón aún más claro. Python 3.14t baja de 5.00 s a 0.42 s con 16 hilos, y Python 3.15t baja de 4.90 s a 0.41 s con el mismo recuento. Los speedups son $11.90\times$ y $11.95\times$. Con 32 hilos ambos empeoran ligeramente, lo que coincide con la campaña principal y con la campaña de versiones: FT escala, pero su óptimo medido está alrededor de 16 hilos.

El IPC ayuda a interpretar el límite de FT. Al aumentar los hilos, el IPC baja y los fallos de caché suben especialmente a 32 hilos, un patrón coherente con una FFT que combina cómputo con accesos a memoria y sincronización entre fases. El límite de FT no es falta de paralelismo efectivo, sino eficiencia decreciente al aumentar el equipo de hilos.

6.5. CG, IS y MG: paralelismo ineficiente o granularidad insuficiente

CG usa varias CPUs con hilos, pero el tiempo empeora. En Python 3.15t, pasa de 0.78 s a 4.59 s con 8 hilos y a 6.42 s con 32 hilos en la campaña de profiling. Al mismo tiempo, `perf` mide 2.41 CPUs con 8 hilos y 5.98 con 32. El problema no es que el proceso permanezca estrictamente serial, sino que la ejecución paralela no retira trabajo útil con eficiencia. El IPC cae de 3.07 con un hilo a 1.04 con 8 y 0.39 con 32.

Python 3.13t y Python 3.14t reproducen el mismo patrón en CG (Figuras 6.10–6.12). Ambas versiones usan varias CPUs al aumentar los hilos, pero el IPC cae y el tiempo empeora. La estructura del gradiente conjugado lo explica: productos escalares, normas, reducciones y barreras frecuentes reducen el trabajo independiente por región paralela. Con muchos hilos, la sincronización domina al cómputo.

IS debe interpretarse con más cautela porque los tiempos son muy pequeños. En clase W, el mejor tiempo está alrededor de 0.02–0.03 s con pocos hilos. A 32 hilos sube a 0.08 s en la campaña de profiling y a 0.06 s en la campaña de versiones. La señal de rendimiento es clara aunque los contadores sean ruidosos: el trabajo por hilo no compensa la coordinación, los histogramas y las fases de acumulación.

MG también usa varias CPUs al aumentar los hilos, pero no obtiene speedup. En Python 3.15t, `perf` mide 3.04 CPUs con 8 hilos y 3.95 con 32, mientras el tiempo sube de 0.46 s a 0.60 s y

Bench.	Python	Hilos	NAS (s)	Speedup	CPUs usadas	IPC	Fallos cache (%)
CG	3.13t	1	0.76	1.00×	1.21	2.71	5.7
CG	3.13t	8	4.56	0.17×	2.62	1.01	2.0
CG	3.13t	32	5.67	0.13×	6.30	0.40	33.7
CG	3.14t	1	0.76	1.00×	1.20	2.66	5.8
CG	3.14t	8	4.49	0.17×	2.58	1.02	3.6
CG	3.14t	32	5.81	0.13×	6.05	0.41	52.2
CG	3.15t	1	0.78	1.00×	0.99	3.07	6.7
CG	3.15t	8	4.59	0.17×	2.41	1.04	3.0
CG	3.15t	32	6.42	0.12×	5.98	0.39	44.5
IS	3.13t	1	0.03	1.00×	3.21	0.96	22.5
IS	3.13t	8	0.02	1.50×	1.69	1.36	30.8
IS	3.13t	32	0.08	0.38×	3.71	0.99	23.1
IS	3.14t	1	0.03	1.00×	2.88	1.02	19.8
IS	3.14t	8	0.02	1.50×	3.91	0.94	24.5
IS	3.14t	32	0.08	0.38×	3.90	0.96	26.8
IS	3.15t	1	0.03	1.00×	0.89	1.93	25.2
IS	3.15t	8	0.03	1.00×	1.08	1.90	23.1
IS	3.15t	32	0.08	0.38×	1.23	1.83	25.8
MG	3.13t	1	0.45	1.00×	2.54	1.37	23.9
MG	3.13t	8	0.60	0.75×	3.66	0.82	63.6
MG	3.13t	32	0.91	0.49×	4.64	0.56	43.2
MG	3.14t	1	0.45	1.00×	2.64	1.34	35.0
MG	3.14t	8	0.56	0.80×	3.44	0.80	64.5
MG	3.14t	32	0.81	0.56×	5.24	0.64	41.4
MG	3.15t	1	0.46	1.00×	0.93	2.86	25.0
MG	3.15t	8	0.60	0.77×	3.04	0.85	39.8
MG	3.15t	32	0.93	0.49×	3.95	0.65	40.7

Tabla 6.2: Perfil con `perf stat` de CG, IS y MG en clase W, modo 3.

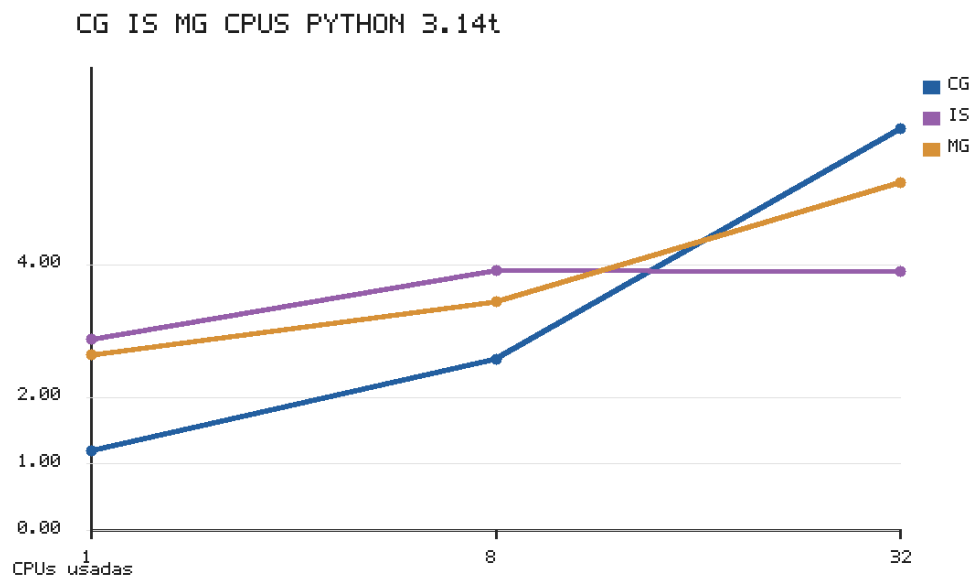


Figura 6.4: CPUs utilizadas por CG, IS y MG con Python 3.14t en clase W, modo 3.

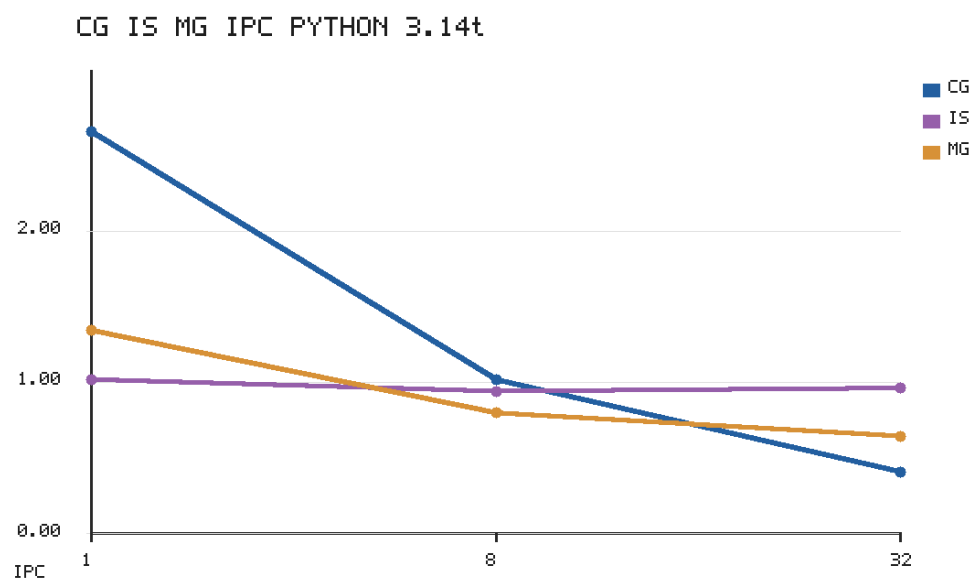


Figura 6.5: IPC de CG, IS y MG con Python 3.14t en clase W, modo 3.

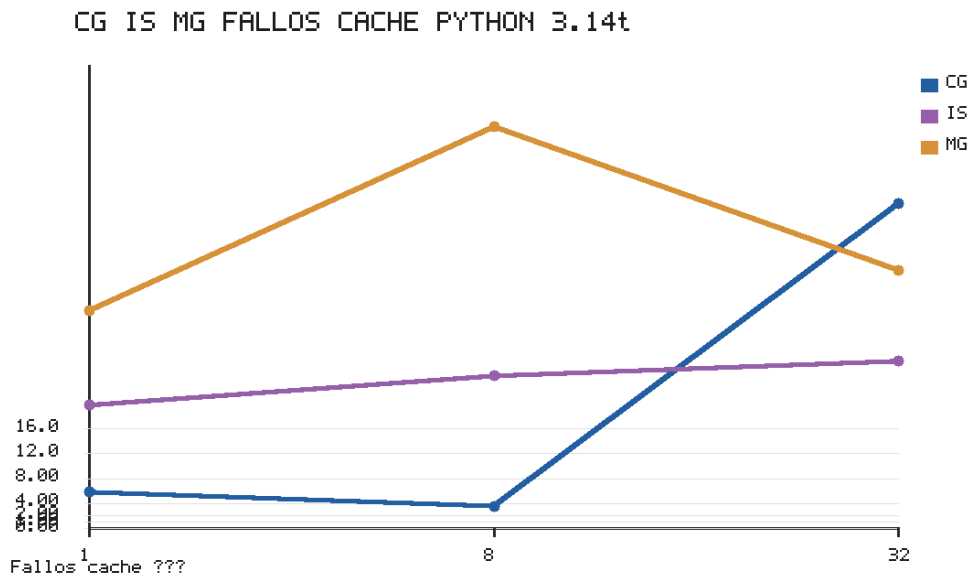


Figura 6.6: Porcentaje de fallos de caché de CG, IS y MG con Python 3.14t en clase W, modo 3.

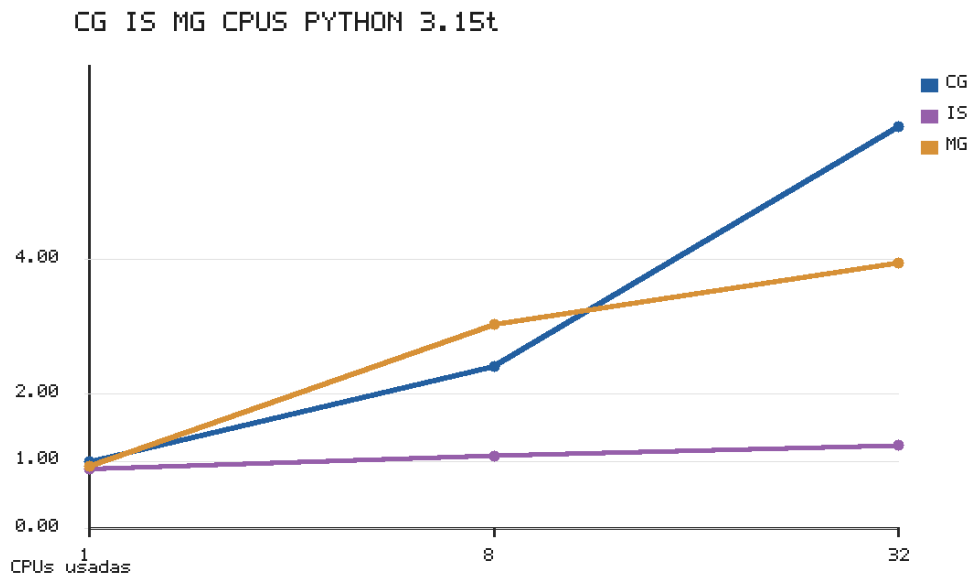


Figura 6.7: CPUs utilizadas por CG, IS y MG con Python 3.15t en clase W, modo 3.

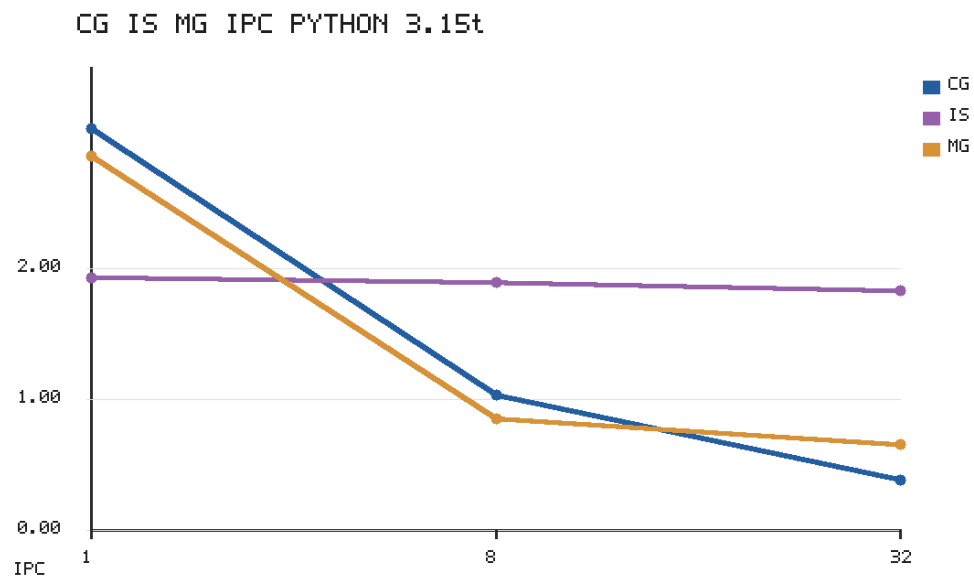


Figura 6.8: IPC de CG, IS y MG con Python 3.15t en clase W, modo 3.

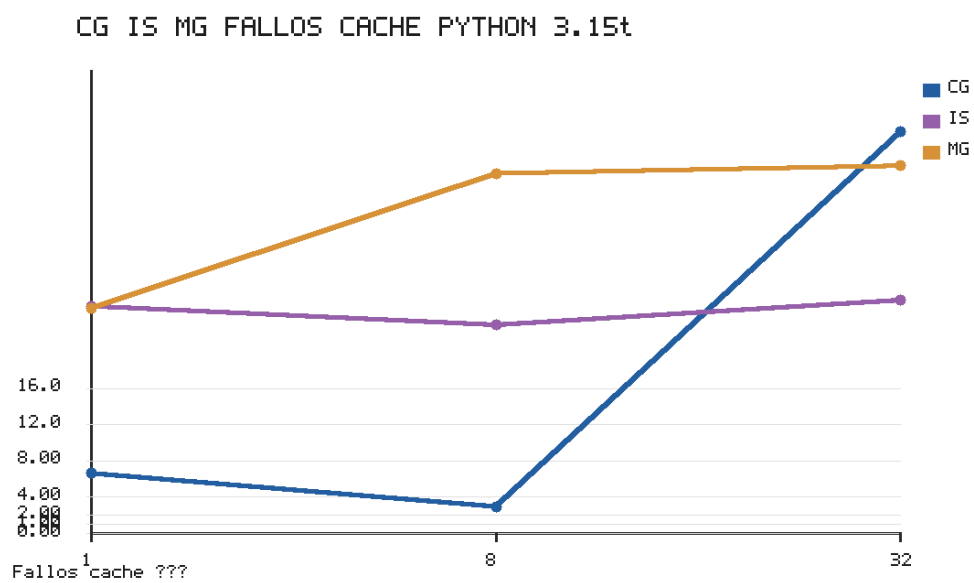


Figura 6.9: Porcentaje de fallos de caché de CG, IS y MG con Python 3.15t en clase W, modo 3.

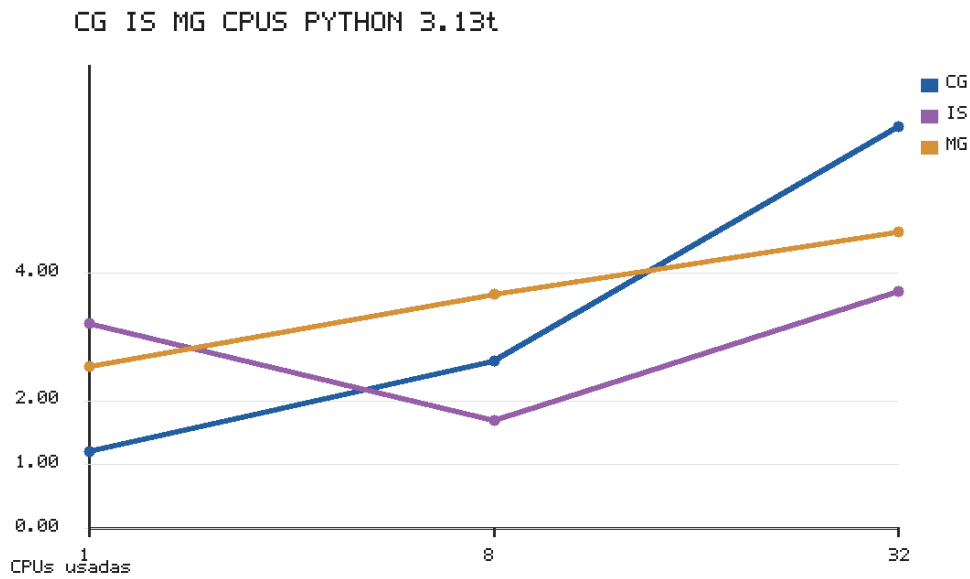


Figura 6.10: CPUs utilizadas por CG, IS y MG con Python 3.13t en clase W, modo 3.

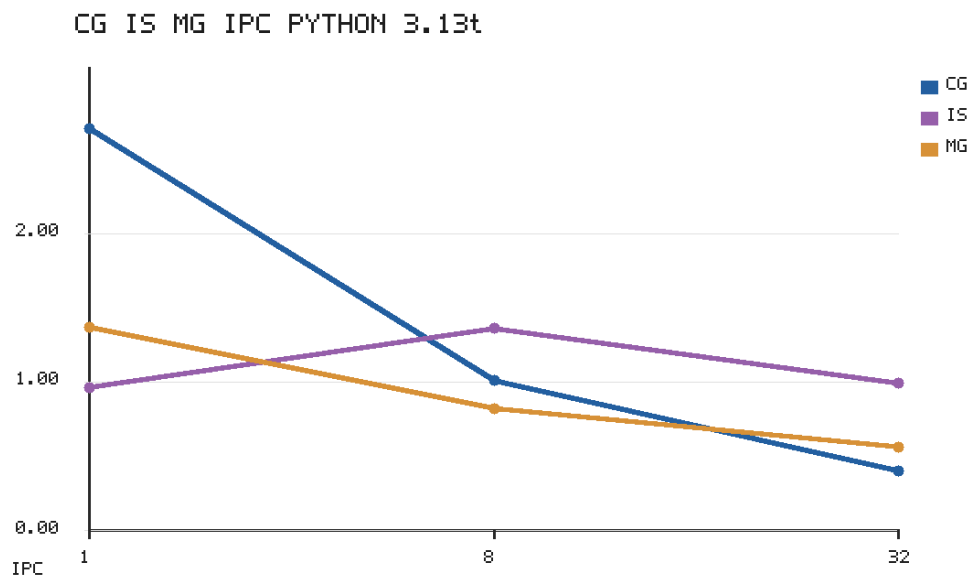


Figura 6.11: IPC de CG, IS y MG con Python 3.13t en clase W, modo 3.

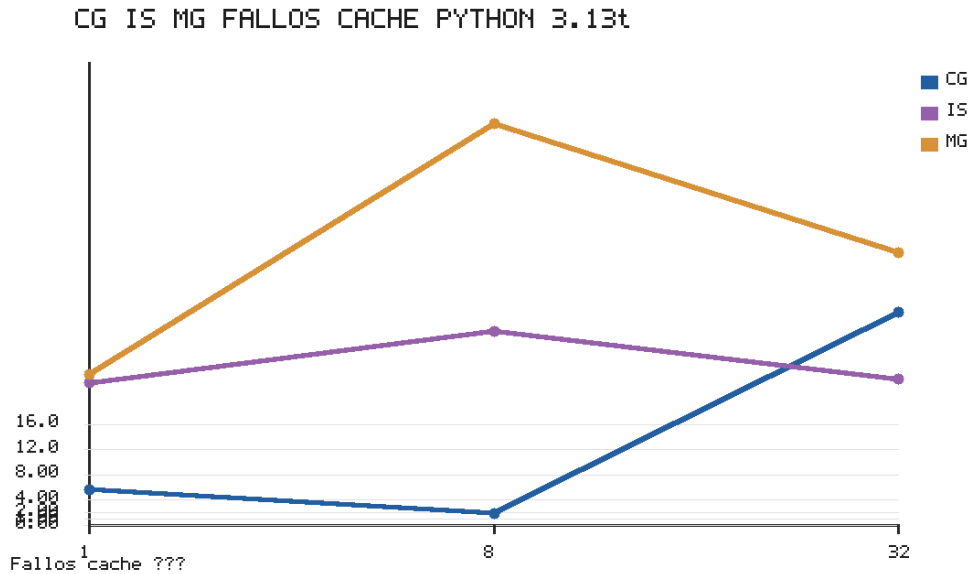


Figura 6.12: Porcentaje de fallos de caché de CG, IS y MG con Python 3.13t en clase W, modo 3.

0.93 s. Python 3.13t y 3.14t siguen la misma tendencia. La caída de IPC y el aumento de fallos de caché indican que el límite combina presión de memoria, sincronización y granularidad desigual entre los niveles de la malla.

6.6. Interpretación conjunta

El profiling separa los benchmarks en tres grupos.

El primer grupo contiene EP y FT. En ambos, el modo 3 genera código capaz de usar varios núcleos y de reducir el tiempo de forma significativa. EP tiene una estructura casi independiente y mantiene el mejor resultado hasta 32 hilos. FT escala hasta 16 hilos y después pierde eficiencia por memoria y sincronización.

El segundo grupo contiene CG y MG. Estos benchmarks sí pueden activar varias CPUs, pero el IPC cae al aumentar los hilos y el tiempo empeora. El paralelismo existe, pero las reducciones, las barreras y las fases de granularidad distinta fragmentan cada región paralela y dejan poco trabajo útil entre sincronizaciones.

El tercer grupo es IS. Su tiempo absoluto es tan bajo en W que el overhead de coordinar muchos hilos domina rápidamente. En clase A se observa una mejora moderada hasta 4 hilos, pero a 32 hilos también pierde eficiencia.

La conclusión del diagnóstico es que los malos speedups no tienen una única causa. En las campañas preliminares existía un problema de afinidad que falseaba la lectura de Python 3.15t.

En las campañas finales, ese problema desaparece en EP y FT. Los límites restantes dependen de la estructura de cada benchmark y de la eficiencia del runtime en regiones con sincronización o granularidad difícil.

6.7. Profiling con el sistema de muestreo de Python 3.15

El análisis con `perf stat` caracteriza el uso de recursos a nivel de proceso (CPUs, IPC, caché), pero no descompone el tiempo entre trabajo útil y sincronización dentro del runtime. Python 3.15 incorpora un nuevo sistema de profiling en la biblioteca estándar, apoyado en la interfaz de depuración externa de CPython [17]. Su componente de muestreo, `profiling.sampling`, toma muestras estadísticas de las pilas de llamadas de todos los hilos del proceso [18]. En modo *wall*, un hilo bloqueado en una barrera o una reducción aparece en la muestra dentro de su función de espera, lo que permite atribuir tiempo a la sincronización por nombre de función. Esta sección emplea esa herramienta para cubrir el objetivo de identificar costes de sincronización y balance de carga, que `perf stat` solo sugiere de forma indirecta.

Entorno y metodología. El profiler de muestreo localiza la sección de runtime del intérprete en memoria. En el intérprete *free-threaded* compilado en Finisterrae III (un beta temprano de Python 3.15) esa localización falla sobre la `libpython` compartida, por lo que esta campaña complementaria se ejecutó en un nodo local de 16 núcleos con el mismo intérprete Python 3.15.0b1t *free-threaded*, el mismo código y la misma clase de problema. Las campañas de rendimiento de los capítulos anteriores siguen siendo la referencia sobre Finisterrae III. Este estudio añade la visión por función e hilo que aquéllas no proporcionan. Se midió en modo 1 (híbrido, hilos Python reales sobre el runtime interpretado) porque en ese modo las primitivas de sincronización de OMP4Py son funciones Python visibles al profiler. En modo 3 los kernels y sus barreras se compilan a código nativo y el muestreador de pilas Python no las observa. Cada ejecución se muestreó a 1000–2000 Hz con el muestreo de todos los hilos activado.

Coste de sincronización. La Tabla 6.3 mide la fracción de las muestras de ejecución paralela que caen en primitivas de espera del runtime. El hilo que espera aparece en `threading.Condition.wait`, alcanzada a través de `parallelism.py:parallel_run` y la barrera interna del runtime. El coste de sincronización crece con el número de hilos en los cuatro benchmarks. Con 4 hilos, EP dedica solo el 1.0 % del tiempo paralelo a esperar, frente al 9–12 % de CG, FT y MG. Con 8 hilos los cuatro se sitúan entre el 16 % y el 20 %. En FT, donde la FFT impone barreras entre pasadas, la fracción sigue subiendo hasta el 25.8 % con 16 hilos. Este patrón es coherente con la ley de Amdahl [16] y con los resultados de rendimiento: el tipado del modo 3 acelera

el cómputo pero no la sincronización, de modo que la fracción de espera observada aquí se convierte en el límite efectivo de escalabilidad cuando el kernel es rápido.

Benchmark	1 hilo	4 hilos	8 hilos
EP	0.0 %	1.0 %	16.3 %
FT	0.0 %	9.0 %	17.4 %
CG	0.1 %	11.6 %	19.7 %
MG	0.0 %	9.7 %	17.0 %

Tabla 6.3: Coste de sincronización medido con el profiler de muestreo de Python 3.15 (`profiling.sampling`, modo *wall*, todos los hilos). Porcentaje de las muestras de ejecución paralela que caen en primitivas de espera del runtime OMP4Py (barrera y reducción, `threading.Condition.wait`), frente al número de hilos. Clase S, modo 1.

Gestión de hilos y balance de carga. El profiler revela además una diferencia estructural que `perf stat` no expone (Tabla 6.4). EP ejecuta toda su región paralela sobre un único equipo persistente de hilos: con 8 hilos se observan 7 hilos trabajadores distintos. CG, FT y MG, en cambio, crean y destruyen un equipo por cada región paralela, de modo que a lo largo de una ejecución aparecen cientos o miles de hilos efímeros: 246 en FT, 422 en MG y 4859 en CG. Esa creación y destrucción repetida de equipos es en sí misma una fuente de sobrecoste y explica parte de la degradación de CG y MG al aumentar la concurrencia. Sobre el equipo persistente de EP, el reparto de muestras de cómputo entre hilos trabajadores es razonablemente equilibrado (cociente entre el mínimo y el máximo entre 0.76 y 0.97 según el número de hilos). En los kernels con rotación de equipos no existe un equipo estable sobre el que medir balance, y el propio recuento de hilos efímeros pasa a ser el indicador relevante.

Benchmark	Hilos trabajadores distintos	Sincr./paralelo
EP	7 (equipo persistente)	16.3 %
FT	246	17.4 %
MG	422	17.0 %
CG	4859	19.7 %

Tabla 6.4: Gestión de hilos con 8 hilos según el profiler de muestreo de Python 3.15 (clase S, modo 1). *Hilos trabajadores distintos* es el número de identificadores de hilo con muestras de cómputo observados durante la ejecución: EP emplea un único equipo persistente, mientras que CG, FT y MG crean y destruyen un equipo por región paralela, generando cientos o miles de hilos efímeros.

Lectura conjunta. Las dos tablas refuerzan, desde el nivel del intérprete, la conclusión del diagnóstico con `perf stat`. EP escala porque combina trabajo independiente, un equipo persistente y poca sincronización a baja concurrencia. FT escala con meseta porque su sincronización entre pasadas crece de forma sostenida. CG y MG no escalan porque acumulan

a la vez sincronización creciente y rotación de equipos de hilos. El sistema de muestreo de Python 3.15 permite así pasar de los síntomas agregados de `perf stat` a una atribución por función e hilo del tiempo de sincronización, que es precisamente la instrumentación interna del runtime que se planteaba como trabajo necesario.

6.8. Implicaciones para OMP4Py

Los resultados no invalidan el port. EP y FT son evidencia experimental de que las regiones paralelas adaptadas con OMP4Py pueden producir speedups altos cuando el kernel es adecuado. A la vez, el modo 3 demuestra que el tipado Cython elimina gran parte del coste de objetos Python en los bucles críticos.

La implicación para OMP4Py es que el rendimiento debe evaluarse en dos niveles. El primero es la calidad del código Cython generado: si quedan llamadas Python en el bucle crítico, el tiempo absoluto se dispara. El segundo es la ejecución paralela real: aunque el código esté tipado, las reducciones, barreras, distribución de trabajo y presión de memoria pueden hacer que más hilos no reduzcan el tiempo.

Como trabajo posterior, la línea más útil es instrumentar OMP4Py por región paralela. Sería necesario medir cuántos hilos entran realmente en cada región, cuánto tiempo pasan en trabajo útil y cuánto en barreras, reducciones, secciones críticas o espera activa. Esa instrumentación permitiría pasar de los síntomas observados con `perf stat` a una explicación causal dentro del runtime.

Comparación entre versiones de Python *free-threaded*

El capítulo anterior muestra que, en la campaña principal con Python 3.15t, el modo 3 de OMP4Py produce tiempos competitivos y escala en EP y FT, mientras que CG, IS y MG presentan límites de paralelización. Este capítulo estudia si ese comportamiento depende de la versión concreta del intérprete *free-threaded*.

La comparación cubre Python 3.13.13t, Python 3.14.4t y Python 3.15.0b1t, todas compiladas con el GIL desactivado. Las campañas finales se ejecutaron con el mismo código fuente, la misma infraestructura de Slurm, las mismas clases y el mismo rango de hilos. Además, se eliminaron las variables de afinidad de OpenMP que habían producido resultados preliminares no representativos. Por tanto, la variable experimental de este capítulo es la versión del intérprete y su ABI *free-threaded*.

7.1. Diseño de la campaña

La campaña de comparación entre versiones se diseñó como un subconjunto de la campaña principal, pero manteniendo todas las potencias de dos hasta 32 hilos. Se ejecutaron CG, EP, FT, IS y MG, las clases S y W, los modos compilados 2 y 3, y los recuentos de hilos 1, 2, 4, 8, 16 y 32. La clase S se ejecutó con cinco repeticiones por combinación y la clase W con tres repeticiones. La campaña completa contiene 1440 ejecuciones y todas finalizaron con verificación SUCCESSFUL.

No se incluyó la clase A porque el coste habría multiplicado el uso del cluster. La clase A ya se evaluó con Python 3.15t en la campaña principal. La comparación entre versiones se centra en observar si las tendencias de S y W cambian al modificar el intérprete.

Al igual que en el resto de la memoria, se usa el mejor tiempo NAS entre las repeticiones válidas. Los tiempos externos del proceso se conservan en los registros, pero no se emplean

para comparar rendimiento porque incluyen costes de arranque, importación y compilación.

7.2. Rendimiento secuencial en modo 3

El primer paso es comparar el rendimiento con un hilo. Esta medida elimina el efecto de planificación entre hilos y permite observar si una versión introduce una ventaja secuencial clara en los kernels compilados.¹

Clase	Bench.	3.13t (s)	3.14t (s)	3.15t (s)
S	CG	0.12	0.12	0.13
S	EP	1.43	1.43	1.44
S	FT	2.30	2.43	2.42
S	IS	0.00	0.00	0.00
S	MG	0.01	0.01	0.01
W	CG	0.76	0.76	0.78
W	EP	2.86	2.87	2.87
W	FT	4.54	4.80	4.86
W	IS	0.03	0.03	0.03
W	MG	0.44	0.62	0.45

Tabla 7.1: Mejor tiempo NAS con 1 hilo en modo 3 para cada versión de Python.

La Tabla 7.1 muestra que las diferencias con un hilo son pequeñas en la mayoría de combinaciones. EP es prácticamente idéntico: en clase W, las tres versiones se sitúan entre 2.86 s y 2.87 s. CG clase W oscila entre 0.76 s y 0.78 s. FT favorece ligeramente a Python 3.13t, con 4.54 s en clase W frente a 4.80 s en 3.14t y 4.86 s en 3.15t.

MG clase W es el caso con más variación relativa: Python 3.13t tarda 0.44 s, Python 3.15t 0.45 s y Python 3.14t 0.62 s. Aun así, las diferencias de un hilo no explican por sí solas la escalabilidad posterior. En EP y FT, donde los speedups son altos, los tiempos secuenciales son muy parecidos entre versiones.

Los tiempos de IS y MG en clase S deben interpretarse con cautela. IS aparece como 0.00 s con un hilo en las tres versiones y MG como 0.01 s. Estos valores confirman que los kernels son muy pequeños en clase S, pero no proporcionan precisión suficiente para analizar diferencias finas.

La Tabla 7.2 recoge los tiempos de modo 3 para todos los recuentos de hilos. Esta tabla es la base para analizar las curvas de speedup y los óptimos de cada combinación.

¹Los tiempos absolutos de esta campaña pueden diferir ligeramente de los del Capítulo 5 para la misma combinación (por ejemplo, FT clase W con un hilo aparece como 4.86 s aquí y 5.04 s en la campaña principal): son campañas independientes y la diferencia refleja la variabilidad entre ejecuciones del clúster. Todas las comparaciones de speedup se realizan siempre dentro de la misma campaña.

Clase	Bench.	Python	1	2	4	8	16	32
S	CG	3.13t	0.12	0.48	0.65	0.72	0.86	0.91
S	CG	3.14t	0.12	0.47	0.62	0.71	0.86	0.92
S	CG	3.15t	0.13	0.46	0.61	0.70	0.86	0.92
S	EP	3.13t	1.43	0.72	0.36	0.19	0.19	0.15
S	EP	3.14t	1.43	0.72	0.36	0.18	0.19	0.15
S	EP	3.15t	1.44	0.72	0.36	0.18	0.19	0.16
S	FT	3.13t	2.30	1.19	0.62	0.33	0.20	0.22
S	FT	3.14t	2.43	1.25	0.65	0.35	0.22	0.23
S	FT	3.15t	2.42	1.26	0.66	0.35	0.22	0.24
S	IS	3.13t	0.00	0.00	0.00	0.01	0.01	0.02
S	IS	3.14t	0.00	0.00	0.00	0.01	0.01	0.02
S	IS	3.15t	0.00	0.00	0.00	0.01	0.01	0.02
S	MG	3.13t	0.01	0.02	0.03	0.05	0.10	0.21
S	MG	3.14t	0.01	0.02	0.03	0.05	0.10	0.21
S	MG	3.15t	0.01	0.02	0.03	0.06	0.10	0.22
W	CG	3.13t	0.76	2.88	4.16	4.46	5.49	5.63
W	CG	3.14t	0.76	2.78	3.91	4.46	5.35	5.63
W	CG	3.15t	0.78	2.77	4.05	4.60	5.43	5.70
W	EP	3.13t	2.86	1.44	0.72	0.37	0.37	0.29
W	EP	3.14t	2.87	1.44	0.72	0.36	0.37	0.29
W	EP	3.15t	2.87	1.44	0.72	0.36	0.37	0.29
W	FT	3.13t	4.54	2.37	1.20	0.63	0.38	0.40
W	FT	3.14t	4.80	2.49	1.28	0.67	0.40	0.42
W	FT	3.15t	4.86	2.53	1.30	0.67	0.41	0.42
W	IS	3.13t	0.03	0.02	0.02	0.02	0.03	0.06
W	IS	3.14t	0.03	0.02	0.02	0.02	0.03	0.06
W	IS	3.15t	0.03	0.02	0.02	0.02	0.04	0.06
W	MG	3.13t	0.44	0.55	0.55	0.54	0.58	0.74
W	MG	3.14t	0.62	0.56	0.52	0.51	0.55	0.71
W	MG	3.15t	0.45	0.57	0.54	0.54	0.59	0.78

Tabla 7.2: Mejor tiempo NAS por número de hilos en modo 3 para cada versión de Python.

7.3. Efecto del tipado Cython entre versiones

Antes de estudiar la escalabilidad conviene comprobar si el efecto del modo 3 se mantiene en todas las versiones. La Tabla 7.3 compara modo 2 y modo 3 con un hilo para cada versión, clase y benchmark.

El resultado es estable: el modo 3 domina al modo 2 en las tres versiones. En clase W, CG mejora entre $120.16\times$ y $135.33\times$, MG entre $303.19\times$ y $432.18\times$, EP alrededor de $25.55\times$ – $29.53\times$ y FT entre $16.64\times$ y $19.45\times$. IS también presenta factores por encima de $160\times$ en W.

Esta comparación separa dos fenómenos. El primero es el coste de ejecutar código compilado sin tipos explícitos, alto en Python 3.13t, 3.14t y 3.15t. El segundo es la escalabilidad entre hilos, que depende además de la estructura del benchmark y del runtime. El modo 3 es condición necesaria para obtener buenos tiempos absolutos, pero no garantiza que todos los kernels escalen.

7.4. Escalabilidad en modo 3

La Tabla 7.4 compara el speedup a 32 hilos en modo 3. La Tabla 7.5 extiende la comparación a todos los recuentos de hilos medidos.

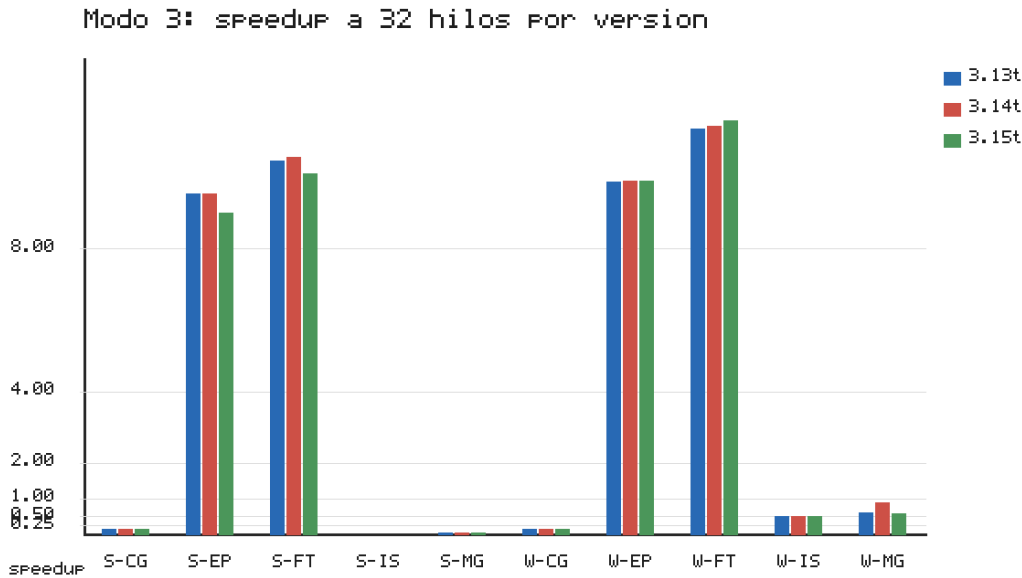


Figura 7.1: Speedup a 32 hilos en modo 3 para Python 3.13t, 3.14t y 3.15t.

El patrón principal es que EP y FT escalan en las tres versiones. En EP clase W, los speedups

Clase	Bench.	Python	M2 (s)	M3 (s)	M2/M3
S	CG	3.13t	14.27	0.12	118.92×
S	CG	3.14t	15.31	0.12	127.58×
S	CG	3.15t	16.16	0.13	124.31×
S	EP	3.13t	36.60	1.43	25.59×
S	EP	3.14t	42.45	1.43	29.69×
S	EP	3.15t	42.24	1.44	29.33×
S	FT	3.13t	35.48	2.30	15.43×
S	FT	3.14t	41.65	2.43	17.14×
S	FT	3.15t	36.27	2.42	14.99×
S	IS	3.13t	0.29	0.00	—
S	IS	3.14t	0.30	0.00	—
S	IS	3.15t	0.30	0.00	—
S	MG	3.13t	2.83	0.01	283.00×
S	MG	3.14t	3.12	0.01	312.00×
S	MG	3.15t	3.24	0.01	324.00×
W	CG	3.13t	91.32	0.76	120.16×
W	CG	3.14t	98.84	0.76	130.05×
W	CG	3.15t	105.6	0.78	135.33×
W	EP	3.13t	73.08	2.86	25.55×
W	EP	3.14t	83.94	2.87	29.25×
W	EP	3.15t	84.76	2.87	29.53×
W	FT	3.13t	82.84	4.54	18.25×
W	FT	3.14t	93.36	4.80	19.45×
W	FT	3.15t	80.89	4.86	16.64×
W	IS	3.13t	4.83	0.03	161.00×
W	IS	3.14t	4.98	0.03	166.00×
W	IS	3.15t	5.40	0.03	180.00×
W	MG	3.13t	170.8	0.44	388.11×
W	MG	3.14t	188.0	0.62	303.19×
W	MG	3.15t	194.5	0.45	432.18×

Tabla 7.3: Efecto del tipado Cython con 1 hilo en la campaña de comparación de versiones.

Clase	Bench.	3.13t	3.14t	3.15t
S	CG	0.13×	0.13×	0.14×
S	EP	9.53×	9.53×	9.00×
S	FT	10.45×	10.57×	10.08×
S	IS	—	—	—
S	MG	0.05×	0.05×	0.05×
W	CG	0.13×	0.13×	0.14×
W	EP	9.86×	9.90×	9.90×
W	FT	11.35×	11.43×	11.57×
W	IS	0.50×	0.50×	0.50×
W	MG	0.59×	0.87×	0.58×

Tabla 7.4: Speedup a 32 hilos respecto a 1 hilo en modo 3 para cada versión de Python.

a 32 hilos son 9.86×, 9.90× y 9.90× para Python 3.13t, 3.14t y 3.15t. En FT clase W, son 11.35×, 11.43× y 11.57×. En clase S se observa la misma tendencia, con speedups alrededor de 9×–10×.

CG, IS y MG no muestran escalabilidad fuerte. CG cae a aproximadamente 0.13×–0.14× a 32 hilos en S y W. IS clase W queda en 0.50× en las tres versiones, y MG clase W se mueve entre 0.58× y 0.87×. La similitud entre versiones indica que, para estos benchmarks, el límite principal está en la estructura del kernel o en la forma en que el runtime ejecuta sus regiones paralelas, no en una regresión específica de una versión de Python.

Esta lectura corrige la interpretación de las campañas preliminares. Antes de eliminar la afinidad explícita de OpenMP, Python 3.15t parecía no escalar en EP y FT. Las campañas finales muestran que ese comportamiento no era una propiedad intrínseca del intérprete ni del port, sino un efecto del entorno de ejecución.

7.5. Curvas por benchmark

Las Figuras 7.2, 7.3, 7.4 y 7.5 muestran las curvas de speedup de EP y FT con 1, 2, 4, 8, 16 y 32 hilos. Las Figuras 7.6–7.11 completan la misma vista para CG, IS y MG.

En EP, las tres versiones siguen curvas casi idénticas. La aceleración es casi lineal hasta 8 hilos y se estabiliza después. En clase W, el mejor punto medido aparece en 32 hilos para las tres versiones, con tiempos de 0.29 s. El límite no parece estar en la versión del intérprete, sino en el coste fijo de la región paralela y en la eficiencia del runtime al aumentar el equipo de hilos.

FT también escala en las tres versiones, pero su punto óptimo aparece antes. En clase W, las tres versiones alcanzan el mejor tiempo con 16 hilos: 0.38 s en Python 3.13t, 0.40 s en 3.14t y 0.41 s en 3.15t. Con 32 hilos el tiempo sube ligeramente. Esto es coherente con la estructura de la FFT, que combina cómputo, accesos a memoria y sincronizaciones entre fases.

Las curvas de CG, IS y MG siguen el patrón contrario. En CG, añadir hilos degrada el

Clase	Bench.	Python	1	2	4	8	16	32
S	CG	3.13t	1.00×	0.25×	0.18×	0.17×	0.14×	0.13×
S	CG	3.14t	1.00×	0.26×	0.19×	0.17×	0.14×	0.13×
S	CG	3.15t	1.00×	0.28×	0.21×	0.19×	0.15×	0.14×
S	EP	3.13t	1.00×	1.99×	3.97×	7.53×	7.53×	9.53×
S	EP	3.14t	1.00×	1.99×	3.97×	7.94×	7.53×	9.53×
S	EP	3.15t	1.00×	2.00×	4.00×	8.00×	7.58×	9.00×
S	FT	3.13t	1.00×	1.93×	3.71×	6.97×	11.50×	10.45×
S	FT	3.14t	1.00×	1.94×	3.74×	6.94×	11.05×	10.57×
S	FT	3.15t	1.00×	1.92×	3.67×	6.91×	11.00×	10.08×
S	IS	3.13t	—	—	—	—	—	—
S	IS	3.14t	—	—	—	—	—	—
S	IS	3.15t	—	—	—	—	—	—
S	MG	3.13t	1.00×	0.50×	0.33×	0.20×	0.10×	0.05×
S	MG	3.14t	1.00×	0.50×	0.33×	0.20×	0.10×	0.05×
S	MG	3.15t	1.00×	0.50×	0.33×	0.17×	0.10×	0.05×
W	CG	3.13t	1.00×	0.26×	0.18×	0.17×	0.14×	0.13×
W	CG	3.14t	1.00×	0.27×	0.19×	0.17×	0.14×	0.13×
W	CG	3.15t	1.00×	0.28×	0.19×	0.17×	0.14×	0.14×
W	EP	3.13t	1.00×	1.99×	3.97×	7.73×	7.73×	9.86×
W	EP	3.14t	1.00×	1.99×	3.99×	7.97×	7.76×	9.90×
W	EP	3.15t	1.00×	1.99×	3.99×	7.97×	7.76×	9.90×
W	FT	3.13t	1.00×	1.92×	3.78×	7.21×	11.95×	11.35×
W	FT	3.14t	1.00×	1.93×	3.75×	7.16×	12.00×	11.43×
W	FT	3.15t	1.00×	1.92×	3.74×	7.25×	11.85×	11.57×
W	IS	3.13t	1.00×	1.50×	1.50×	1.50×	1.00×	0.50×
W	IS	3.14t	1.00×	1.50×	1.50×	1.50×	1.00×	0.50×
W	IS	3.15t	1.00×	1.50×	1.50×	1.50×	0.75×	0.50×
W	MG	3.13t	1.00×	0.80×	0.80×	0.81×	0.76×	0.59×
W	MG	3.14t	1.00×	1.11×	1.19×	1.22×	1.13×	0.87×
W	MG	3.15t	1.00×	0.79×	0.83×	0.83×	0.76×	0.58×

Tabla 7.5: Speedup por número de hilos en modo 3 para cada versión de Python.

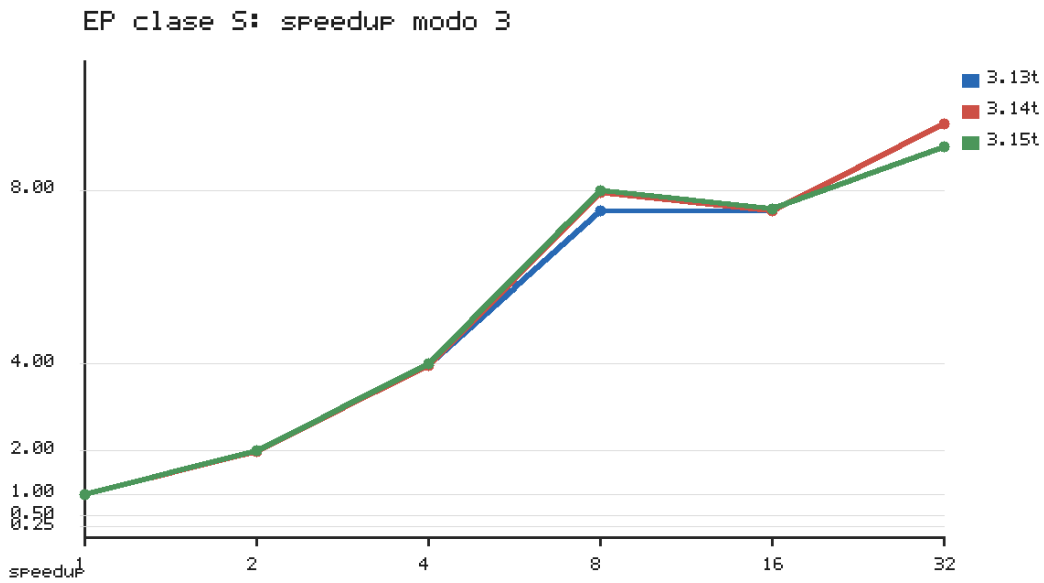


Figura 7.2: EP clase S, modo 3: speedup por versión de Python.

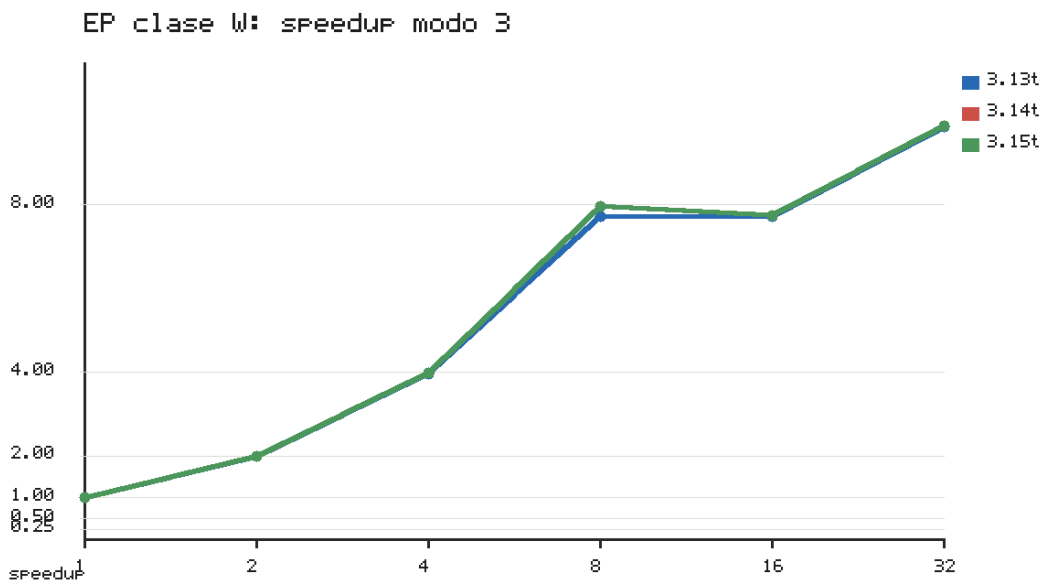


Figura 7.3: EP clase W, modo 3: speedup por versión de Python.

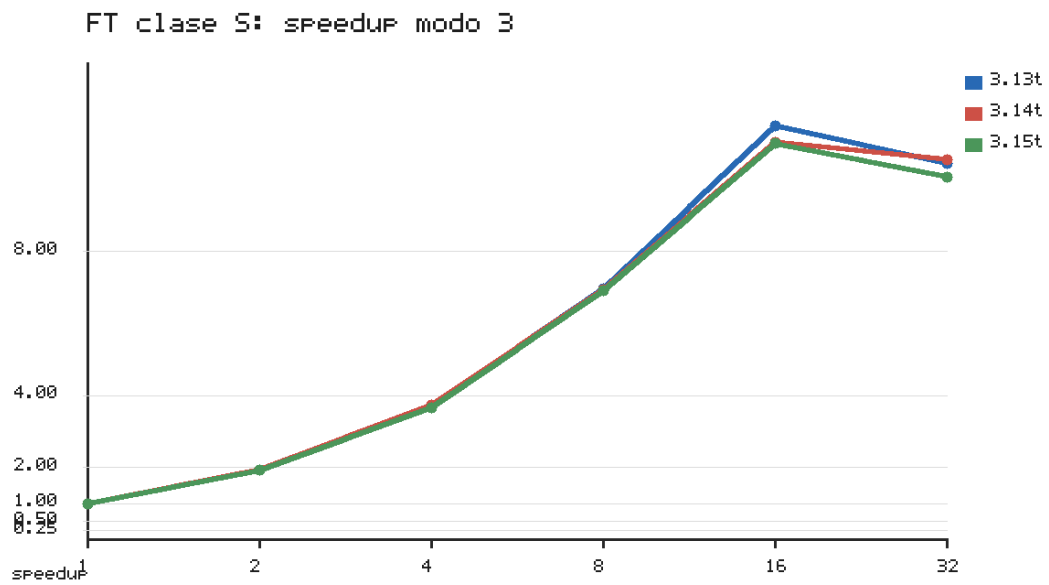


Figura 7.4: FT clase S, modo 3: speedup por versión de Python.

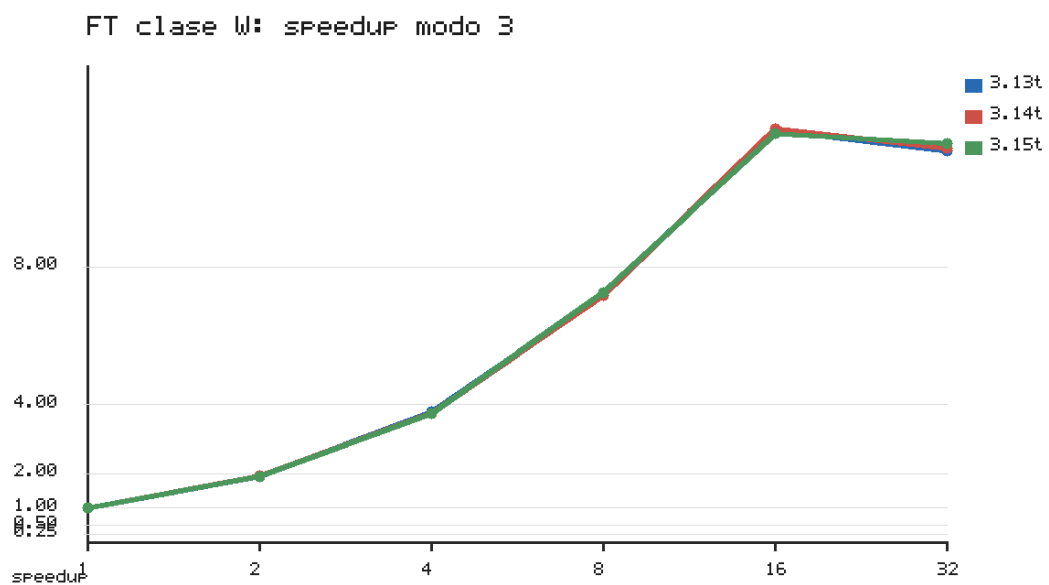


Figura 7.5: FT clase W, modo 3: speedup por versión de Python.

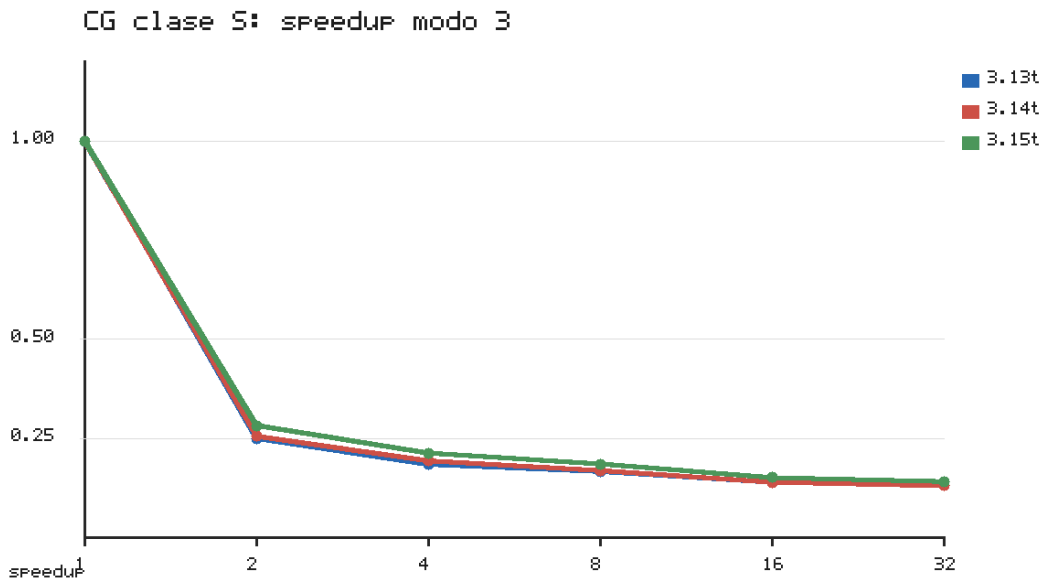


Figura 7.6: CG clase S, modo 3: speedup por versión de Python.

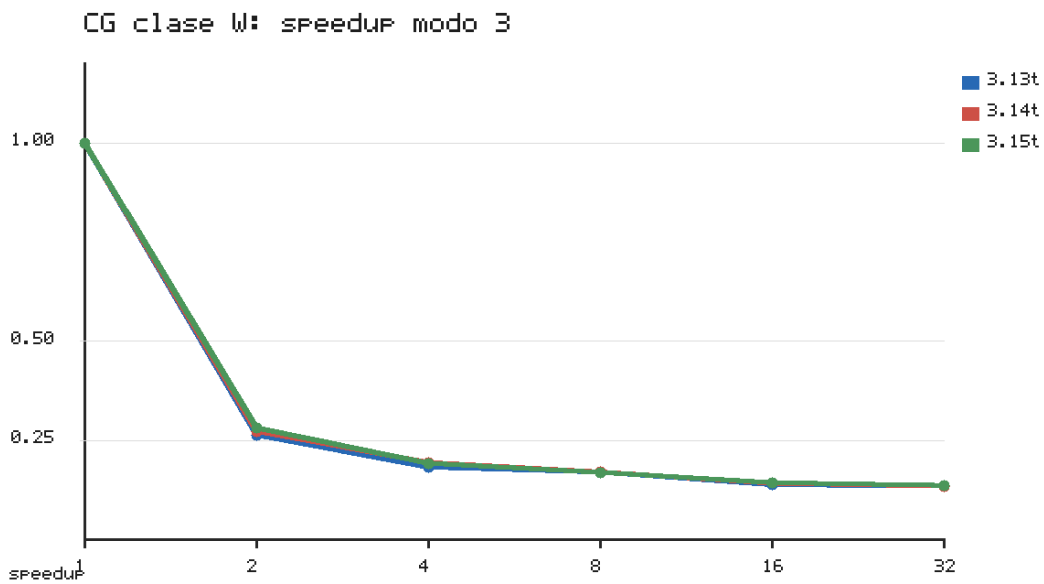


Figura 7.7: CG clase W, modo 3: speedup por versión de Python.

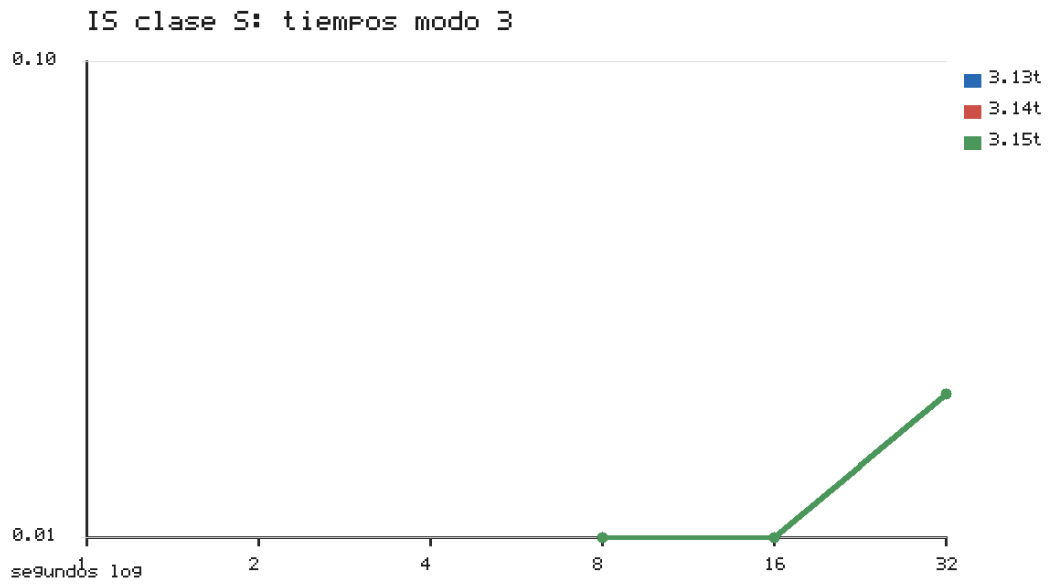


Figura 7.8: IS clase S, modo 3: tiempo NAS por versión de Python. El speedup no es fiable porque el tiempo con un hilo aparece redondeado a 0.00 s.

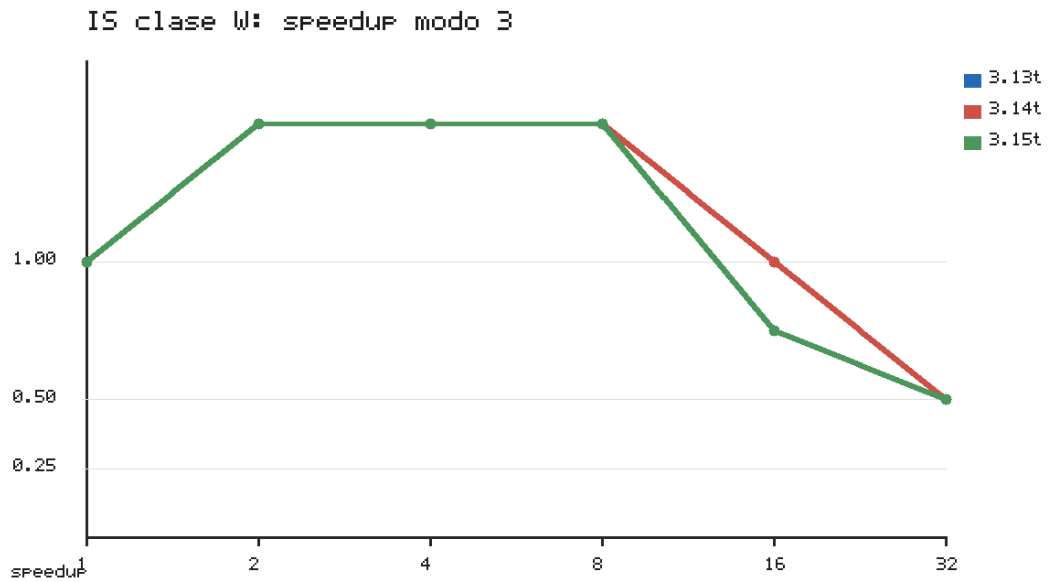


Figura 7.9: IS clase W, modo 3: speedup por versión de Python.

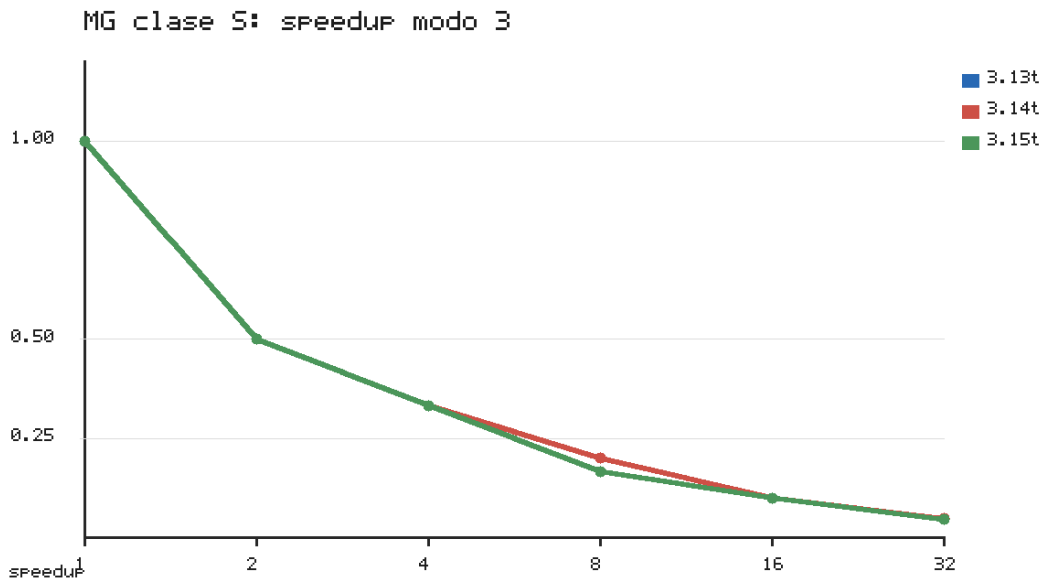


Figura 7.10: MG clase S, modo 3: speedup por versión de Python.

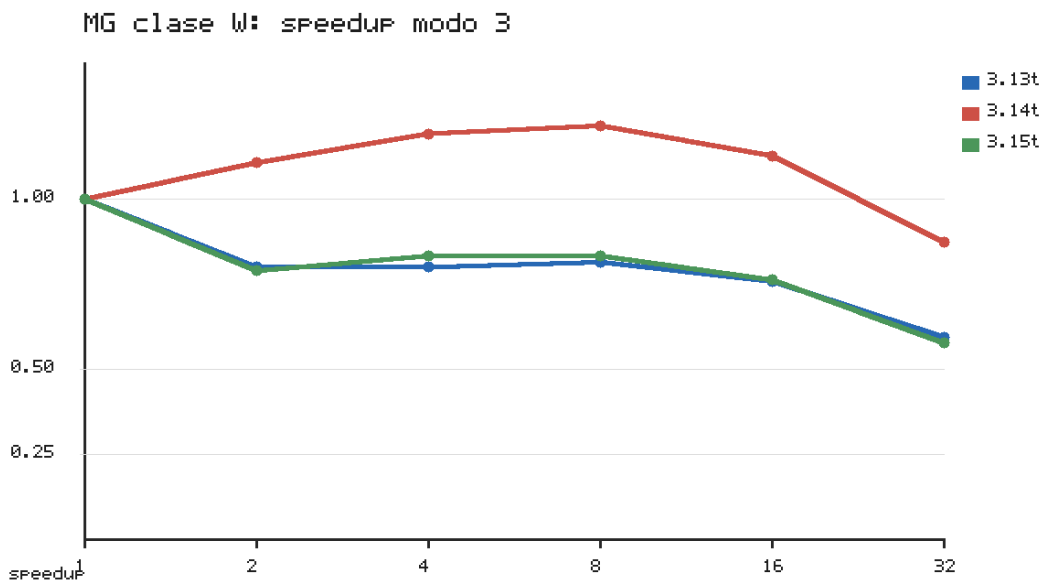


Figura 7.11: MG clase W, modo 3: speedup por versión de Python.

rendimiento en las tres versiones y en las dos clases. En IS, la clase W mantiene una mejora solo con pocos hilos y cae a 32 hilos. En MG, la clase S queda dominada por overhead y la clase W no consigue speedup sostenido. La comparación visual refuerza que la versión de Python no cambia la forma básica de estas tres curvas.

Clase	Bench.	Python	Mejor hilo	Mejor tiempo (s)	Speedup ópt.	T32 (s)	Speedup32
S	CG	3.13t	1	0.12	1.00×	0.91	0.13×
S	CG	3.14t	1	0.12	1.00×	0.92	0.13×
S	CG	3.15t	1	0.13	1.00×	0.92	0.14×
S	EP	3.13t	32	0.15	9.53×	0.15	9.53×
S	EP	3.14t	32	0.15	9.53×	0.15	9.53×
S	EP	3.15t	32	0.16	9.00×	0.16	9.00×
S	FT	3.13t	16	0.20	11.50×	0.22	10.45×
S	FT	3.14t	16	0.22	11.05×	0.23	10.57×
S	FT	3.15t	16	0.22	11.00×	0.24	10.08×
S	IS	3.13t	1	0.00	—	0.02	0.00×
S	IS	3.14t	1	0.00	—	0.02	0.00×
S	IS	3.15t	1	0.00	—	0.02	0.00×
S	MG	3.13t	1	0.01	1.00×	0.21	0.05×
S	MG	3.14t	1	0.01	1.00×	0.21	0.05×
S	MG	3.15t	1	0.01	1.00×	0.22	0.05×
W	CG	3.13t	1	0.76	1.00×	5.63	0.13×
W	CG	3.14t	1	0.76	1.00×	5.63	0.13×
W	CG	3.15t	1	0.78	1.00×	5.70	0.14×
W	EP	3.13t	32	0.29	9.86×	0.29	9.86×
W	EP	3.14t	32	0.29	9.90×	0.29	9.90×
W	EP	3.15t	32	0.29	9.90×	0.29	9.90×
W	FT	3.13t	16	0.38	11.95×	0.40	11.35×
W	FT	3.14t	16	0.40	12.00×	0.42	11.43×
W	FT	3.15t	16	0.41	11.85×	0.42	11.57×
W	IS	3.13t	2	0.02	1.50×	0.06	0.50×
W	IS	3.14t	2	0.02	1.50×	0.06	0.50×
W	IS	3.15t	2	0.02	1.50×	0.06	0.50×
W	MG	3.13t	1	0.44	1.00×	0.74	0.59×
W	MG	3.14t	8	0.51	1.22×	0.71	0.87×
W	MG	3.15t	1	0.45	1.00×	0.78	0.58×

Tabla 7.6: Mejor número de hilos en modo 3 para cada versión, clase y benchmark.

La Tabla 7.6 resume el mejor número de hilos. EP tiende a favorecer 32 hilos, FT 16 hilos, IS pocos hilos y CG/MG un hilo. La versión de Python desplaza ligeramente algunos tiempos, pero no altera el patrón de cada benchmark.

7.6. Tiempos absolutos con 32 hilos

El speedup describe la evolución dentro de una misma versión, pero no indica qué versión es más rápida en tiempo absoluto. La Tabla 7.7 muestra los tiempos con 32 hilos en modo 3.

Clase	Bench.	3.13t (s)	3.14t (s)	3.15t (s)
S	CG	0.91	0.92	0.92
S	EP	0.15	0.15	0.16
S	FT	0.22	0.23	0.24
S	IS	0.02	0.02	0.02
S	MG	0.21	0.21	0.22
W	CG	5.63	5.63	5.70
W	EP	0.29	0.29	0.29
W	FT	0.40	0.42	0.42
W	IS	0.06	0.06	0.06
W	MG	0.74	0.71	0.78

Tabla 7.7: Mejor tiempo NAS con 32 hilos en modo 3 para cada versión de Python.

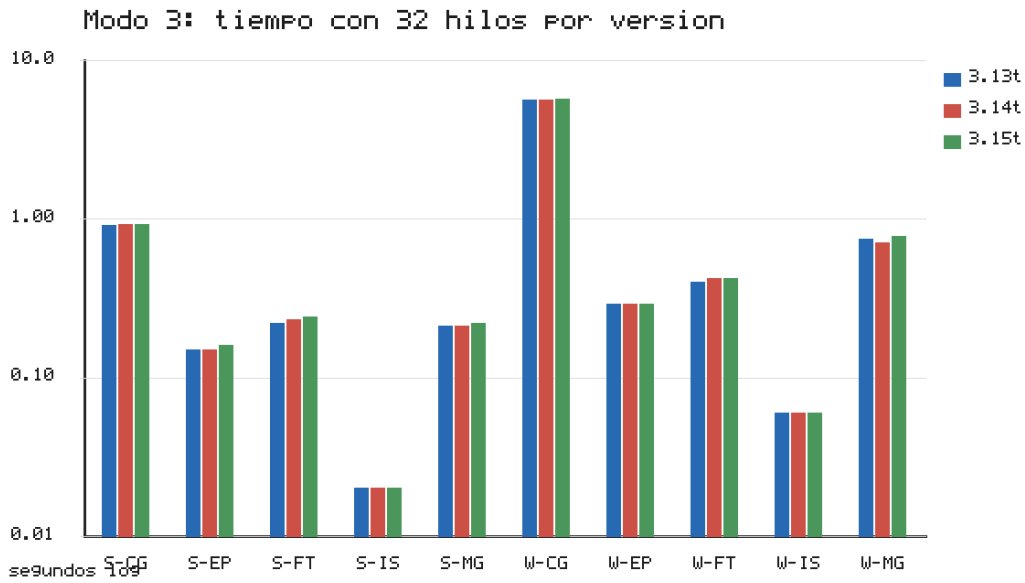


Figura 7.12: Tiempo NAS con 32 hilos en modo 3 para las tres versiones de Python. El eje vertical usa escala logarítmica.

Las diferencias absolutas a 32 hilos son pequeñas en EP y FT. En EP clase W, las tres versiones reportan 0.29 s. En FT clase W, Python 3.13t obtiene 0.40 s y Python 3.14t/3.15t 0.42 s. Estos márgenes son mucho menores que los factores observados entre modos 2 y 3, y deben interpretarse como diferencias de segundo orden dentro del mismo patrón de escalabilidad.

CG, IS y MG tampoco muestran un ganador claro con relevancia práctica. En CG clase W, Python 3.13t y 3.14t empatan a 5.63 s y Python 3.15t queda en 5.70 s. En IS clase W, las tres versiones reportan 0.06 s. En MG clase W, Python 3.14t es ligeramente más rápida con 0.71 s, frente a 0.74 s y 0.78 s.

Clase	Bench.	Ganadora	Tiempo (s)	Segunda (s)	Margen
S	CG	3.13t	0.91	0.92	1.10 %
S	EP	3.13t	0.15	0.15	0.00 %
S	FT	3.13t	0.22	0.23	4.55 %
S	IS	3.13t	0.02	0.02	0.00 %
S	MG	3.13t	0.21	0.21	0.00 %
W	CG	3.13t	5.63	5.63	0.00 %
W	EP	3.13t	0.29	0.29	0.00 %
W	FT	3.13t	0.40	0.42	5.00 %
W	IS	3.13t	0.06	0.06	0.00 %
W	MG	3.14t	0.71	0.74	4.23 %

Tabla 7.8: Versión más rápida con 32 hilos en modo 3 y margen sobre la segunda mejor.

La Tabla 7.8 resume la versión ganadora con 32 hilos. Python 3.13t aparece como ganadora en la mayoría de filas, pero muchos márgenes son 0.00 % por redondeo de la salida NAS. La conclusión no es que Python 3.13t sea universalmente superior, sino que, una vez corregida la afinidad de hilos, las tres versiones ofrecen resultados muy próximos en los benchmarks que escalan.

7.7. Modo 2 frente a modo 3 con varias versiones

Aunque el capítulo se centra en modo 3, la campaña también ejecutó el modo 2. La Tabla 7.9 compara, para cada benchmark y clase, el mejor resultado de modo 2 y el mejor resultado de modo 3 considerando las tres versiones de Python y todos los recuentos de hilos.

El modo 3 gana en todas las combinaciones. En CG clase W, el mejor modo 2 queda en 7.91 s y el mejor modo 3 en 0.76 s. En FT clase W, el mejor modo 2 tarda 14.22 s y el mejor modo 3 0.38 s. En IS clase W, el mejor modo 2 tarda 2.00 s frente a 0.02 s en modo 3. Incluso cuando el modo 2 obtiene speedups relativos altos, su tiempo absoluto no compite con el modo tipado.

Esta tabla refuerza la decisión metodológica de usar modo 3 como configuración principal para las clases grandes. La comparación entre versiones debe hacerse sobre el modo que realmente elimina el coste de objetos Python en los kernels críticos.

Clase	Bench.	Mejor M2	Hilos M2	M2 (s)	Mejor M3	Hilos M3	M3 (s)	M2/M3
S	CG	3.13t	16	2.60	3.13t	1	0.12	21.67×
S	EP	3.13t	32	1.48	3.13t	32	0.15	9.87×
S	FT	3.13t	16	6.36	3.13t	16	0.20	31.80×
S	IS	3.13t	8	0.09	3.13t	1	0.00	—
S	MG	3.13t	16	0.30	3.13t	1	0.01	30.00×
W	CG	3.13t	16	7.91	3.13t	1	0.76	10.41×
W	EP	3.13t	32	2.79	3.13t	32	0.29	9.62×
W	FT	3.15t	16	14.22	3.13t	16	0.38	37.42×
W	IS	3.13t	4	2.00	3.13t	2	0.02	100.00×
W	MG	3.13t	32	7.01	3.13t	1	0.44	15.93×

Tabla 7.9: Mejor resultado de modo 2 frente a modo 3 considerando todas las versiones de Python y todos los recuentos de hilos.

7.8. Relación con la campaña principal

La campaña principal y la campaña de versiones son coherentes: Python 3.15t escala en EP y FT, pero no en CG, IS y MG. En la campaña principal, EP clase A alcanza $10.50\times$ y FT clase A $14.03\times$ a 32 hilos. En la campaña de versiones, EP y FT clase W escalan de forma similar en Python 3.13t, 3.14t y 3.15t.

La diferencia más importante respecto a las ejecuciones preliminares es la configuración de afinidad, analizada en detalle en la Sección 6.2. Con los scripts finales, que no fijan afinidad de OpenMP por defecto, desaparece la serialización aparente de Python 3.15t en EP y FT que se observaba antes.

7.9. Implicaciones

La primera implicación es metodológica. En evaluaciones de Python *free-threaded* no basta con indicar que el GIL está desactivado: también hay que documentar la versión exacta del intérprete, Cython, NumPy, OMP4Py y las variables de entorno que afectan al runtime OpenMP. La afinidad de hilos puede cambiar la conclusión experimental.

La segunda implicación afecta a OMP4Py. El mismo port escala en EP y FT con las tres versiones evaluadas, lo que refuerza que las regiones paralelas están expresadas correctamente para esos kernels. En CG, IS y MG, en cambio, cambiar de versión no basta. Sus límites están en la estructura de cada kernel y en el coste de coordinación del runtime, como muestra el Capítulo 6.

La tercera implicación es práctica. Para estos benchmarks, la elección entre Python 3.13t, 3.14t y 3.15t no cambia la conclusión principal cuando el entorno está correctamente configurado. Python 3.13t tiene pequeñas ventajas en algunos tiempos absolutos, Python 3.14t mejora MG clase W a 32 hilos, y Python 3.15t queda muy cerca en EP y FT. Las diferencias

son mucho menores que el efecto del modo de ejecución y del tipo de benchmark.

7.10. Limitaciones

La comparación entre versiones no incluye la clase A. Por tanto, no se puede afirmar con esta campaña que Python 3.13t y Python 3.14t mantengan en clase A exactamente los mismos speedups que Python 3.15t. La evidencia de S y W sugiere que el patrón debería ser similar en EP y FT, pero confirmarlo requeriría una campaña adicional.

La segunda limitación es que el barrido usa potencias de dos. Esto permite localizar tendencias y óptimos gruesos, pero no determina el máximo exacto de cada curva. En FT, por ejemplo, el óptimo medido aparece en 16 hilos, pero una campaña más fina podría encontrar un punto intermedio ligeramente mejor.

La tercera limitación es la resolución de la salida NAS en benchmarks cortos. IS clase S y MG clase S tienen tiempos tan pequeños que varios ratios quedan afectados por redondeo.

7.11. Conclusiones del capítulo

La comparación entre versiones muestra que el comportamiento final no depende de una única versión de Python. Con la afinidad corregida, Python 3.13t, 3.14t y 3.15t escalan de forma muy parecida en EP y FT. CG, IS y MG siguen sin escalar bien en ninguna versión. Por tanto, las diferencias relevantes del trabajo no están en una regresión específica de Python 3.15t, sino en la combinación entre modo de ejecución, estructura del benchmark y configuración del runtime.

Conclusiones

ESTE trabajo ha adaptado cinco NAS Parallel Benchmarks al modelo de programación OMP4Py y los ha evaluado experimentalmente en Finisterrae III con intérpretes Python *free-threaded*. La evaluación cubre CG, EP, FT, IS y MG; las clases de problema S, W y A; los cuatro modos de ejecución cuando el coste lo permite; y configuraciones de 1, 2, 4, 8, 16 y 32 hilos. La campaña principal con Python 3.15.0b1t incluye 630 ejecuciones. La campaña comparativa entre Python 3.13.13t, 3.14.4t y 3.15.0b1t añade 1440 ejecuciones. Todas las ejecuciones finales terminaron con verificación SUCCESSFUL.

8.1. Hallazgos principales

El port es funcional y cubre los caminos relevantes de OMP4Py. La clase S se ejecutó con los modos 0, 1, 2 y 3, validando los caminos interpretados y compilados. La clase W se evaluó con los modos compilados 2 y 3, y la clase A con el modo 3. Esta estructura permite comprobar corrección sin dedicar recursos a configuraciones que ya son varios órdenes de magnitud más lentas en clases menores.

El tipado Cython es el factor de rendimiento dominante. El modo 3 reduce de forma muy marcada los tiempos frente al modo 2. En clase S, la mejora con un hilo es de $125.77\times$ en CG, $29.39\times$ en EP, $15.26\times$ en FT y $324.00\times$ en MG. En clase W, los factores son $135.42\times$ en CG, $29.45\times$ en EP, $15.89\times$ en FT, $184.67\times$ en IS y $440.51\times$ en MG. Por tanto, para estos benchmarks el tipado no es una optimización menor. Es la condición que convierte el código OMP4Py en una ejecución competitiva.

Python 3.15t escala en EP y FT. La campaña principal obtiene speedups altos con Python 3.15.0b1t en los benchmarks con estructura más favorable. En clase A, EP alcanza $10.50\times$ a 32 hilos y FT alcanza $14.03\times$ a 32 hilos, con un óptimo de $14.69\times$ en 16 hilos. En clase W, EP llega a

$9.90\times$ y FT a $8.00\times$ a 32 hilos. Estos resultados muestran que el port puede explotar paralelismo real cuando el kernel tiene suficiente trabajo independiente. Además, EP en modo 1 alcanza $29.16\times$ a 32 hilos en clase S, una aceleración casi lineal sobre código interpretado que demuestra que la eliminación del GIL produce paralelismo real al nivel del propio Python, aunque sin valor práctico en tiempo absoluto frente al modo tipado.

CG, IS y MG siguen siendo los límites principales. CG, IS y MG no obtienen escalabilidad sostenida en modo 3. CG empeora al aumentar hilos por el coste de reducciones, normas y barreras frecuentes. IS solo mejora de forma moderada con pocos hilos y pierde eficiencia a 32 hilos. MG combina accesos intensivos a memoria, niveles de malla de distinta granularidad y sincronizaciones entre fases. En estos benchmarks, el tipado mejora el tiempo absoluto, pero no basta para convertir más hilos en speedup.

La comparación entre versiones no muestra una regresión específica de Python 3.15t.

Con la configuración final, Python 3.13t, 3.14t y 3.15t escalan de forma muy parecida en EP y FT. En EP clase W, los speedups a 32 hilos son $9.86\times$, $9.90\times$ y $9.90\times$ respectivamente. En FT clase W, son $11.35\times$, $11.43\times$ y $11.57\times$. Las diferencias entre versiones son mucho menores que las diferencias entre modos de ejecución o entre benchmarks.

La afinidad de hilos fue un factor experimental crítico. Las ejecuciones preliminares con variables de afinidad de OpenMP fijadas producían una serialización aparente en Python 3.15t. Al eliminar OMP_PROC_BIND, OMP_PLACES y GOMP_CPU_AFFINITY por defecto en las campañas finales, EP y FT pasaron a escalar correctamente. Esta corrección cambia la interpretación del trabajo: el problema inicial no era una incapacidad general de Python 3.15t ni un fallo intrínseco del port, sino una configuración de ejecución que condicionaba la planificación de hilos.

El profiling separa escalabilidad real de paralelismo ineficiente. Las mediciones con `perf stat` muestran que EP y FT usan varias CPUs y reducen el tiempo. En FT, el IPC y los fallos de caché explican que el óptimo aparezca en 16 hilos en lugar de 32. En CG y MG, el proceso también puede usar varias CPUs, pero el IPC cae y el tiempo empeora. Por tanto, los malos speedups no tienen una única causa: en unos casos el kernel escala, y en otros el paralelismo existe pero la sincronización o la memoria dominan. El sistema de muestreo de Python 3.15 cuantifica ese coste desde el nivel del intérprete: la fracción de tiempo paralelo en espera de barreras y reducciones crece con el número de hilos (hasta el 20–26 % con 8–16 hilos en CG, FT y MG) y revela que CG, FT y MG crean y destruyen un equipo de hilos por región paralela, frente al equipo persistente de EP.

8.2. Limitaciones

La primera limitación es que la comparación entre versiones no incluye la clase A. La decisión fue deliberada para contener el uso del cluster. La campaña principal demuestra escalabilidad de EP y FT en clase A con Python 3.15t, pero no mide esa misma clase con Python 3.13t y Python 3.14t.

La segunda limitación es que `perf stat` mide el proceso completo, no solo la ventana temporizada por NAS. En benchmarks muy cortos, especialmente IS, los contadores deben interpretarse como apoyo diagnóstico y no como medición exacta del kernel.

La tercera limitación afecta a las clases pequeñas. Algunos tiempos de clase S, especialmente en IS y MG, aparecen como 0.00 s o 0.01 s por la resolución de la salida NAS. Esos valores sirven para confirmar corrección y tiempos bajos, pero no para calcular speedups precisos.

8.3. Trabajo futuro

Instrumentación interna de OMP4Py. El muestreo con el profiler de Python 3.15 ya proporciona una primera atribución del tiempo de sincronización y del número de equipos de hilos por función, pero lo hace desde fuera del runtime y sobre el modo interpretado. El siguiente paso es integrar esa instrumentación dentro de OMP4Py, por región paralela y para los modos compilados, de modo que se mida directamente cuántos hilos entran en cada región y cuánto tiempo pasan en trabajo útil frente a barreras, reducciones, secciones críticas o espera activa, y reproducirla sobre Finisterrae III cuando el intérprete *free-threaded* permita adjuntar el muestreador.

Campaña de clase A selectiva para versiones. Una campaña adicional centrada en EP y FT, modo 3, clase A y Python 3.13t/3.14t permitiría comprobar si los speedups observados en S y W se mantienen con la carga mayor. No sería necesario repetir toda la matriz experimental.

Optimización de sincronizaciones y reducciones. CG, IS y MG apuntan a límites asociados a reducciones, accesos irregulares, histogramas, barreras y granularidad desigual. El desarrollo futuro de OMP4Py debería centrarse en reducir el coste de esas operaciones y en mejorar la distribución de trabajo en regiones con tamaño variable.

Actuaciones con mayor probabilidad de mejora. Las mejoras más útiles serían las que reduzcan el coste de coordinación dentro del runtime. En CG, conviene optimizar reducciones y barreras, o agrupar más trabajo por región paralela para amortizar la sincronización. En IS, sería útil paralelizar de forma más eficiente los prefijos acumulados y reducir el coste de los histogramas locales. En MG, la línea más prometedora es adaptar la distribución de trabajo al

tamaño de cada nivel de malla, evitando crear equipos grandes de hilos cuando el nivel grueso solo ofrece unas pocas iteraciones útiles. En todos los casos, mantener los bucles críticos completamente compilables por Cython es condición necesaria: si queda una llamada Python dentro del kernel, el tiempo absoluto vuelve a estar dominado por el intérprete.

Actuaciones poco prometedoras. La campaña también descarta varias explicaciones simples. Cambiar únicamente de Python 3.13t a 3.14t o 3.15t no resuelve CG, IS ni MG, porque las tres versiones muestran curvas muy parecidas cuando la afinidad está corregida. Aumentar el número de hilos tampoco ayuda por sí solo: en CG y MG empeora el tiempo, y en IS solo aporta una mejora limitada con pocos hilos. El modo 2 no es una alternativa práctica aunque en algunos casos muestre speedups relativos altos, porque sus tiempos absolutos siguen siendo muy superiores a los del modo 3. Por último, fijar afinidad de OpenMP sin control puede distorsionar la medida. Cualquier experimento con pinning debe documentarse y compararse frente a una ejecución sin afinidad explícita.

Extensión a BT, SP y LU. La incorporación de paralelismo OMP4Py en BT, SP y LU completaría la cobertura de los NAS Parallel Benchmarks y permitiría evaluar patrones algorítmicos con dependencias espaciales más complejas.

8.4. Valoración global

OMP4Py permite expresar paralelismo de estilo OpenMP en Python y producir ejecuciones correctas sobre benchmarks numéricos no triviales. El resultado más sólido es la combinación de corrección funcional y mejora por tipado: el modo 3 reduce los tiempos hasta en un factor de $440\times$ y hace viables cargas que en modo 2 serían demasiado costosas.

El rendimiento paralelo es positivo, pero selectivo. EP y FT muestran speedups altos con Python 3.15t y se comportan de forma similar en Python 3.13t y Python 3.14t. CG, IS y MG no escalan bien, incluso cuando el código está tipado y verificado. Por tanto, la conclusión final no es que OMP4Py escale o no escale de forma uniforme, sino que su utilidad práctica depende de la combinación entre tipado Cython, estructura del kernel, configuración de afinidad y eficiencia del runtime.

El valor principal del trabajo es haber portado benchmarks completos, medido sus caminos de ejecución y separado varios efectos que podrían confundirse: corrección funcional, mejora por compilación, mejora por tipado, escalabilidad real, problemas de configuración de entorno y límites propios de cada benchmark. Esa separación deja una base experimental clara para continuar el desarrollo de OMP4Py y para evaluar futuras versiones de Python *free-threaded*.

Apéndice

Interfaz de ejecución de los benchmarks

Todos los benchmarks adaptados comparten la misma interfaz de línea de comandos, lo que permite reutilizar la infraestructura de campaña sin cambios por benchmark. La invocación general es:

```
python <BENCHMARK>_Python.py -c <CLASE> -t <HILOS> -m <MODOS>
```

donde `-c` selecciona la clase NAS (S, W, A), `-t` fija el número de hilos del equipo OpenMP y `-m` selecciona el modo de ejecución de OMP4Py (0 puro, 1 híbrido, 2 compilado, 3 compilado tipado). La salida estándar incluye el tiempo NAS interno, los Mop/s y el estado de verificación (SUCCESSFUL o UNSUCCESSFUL), que son los valores que recopila la infraestructura de medición descrita en el Capítulo 4.

Ejemplo de kernel adaptado: EP en modo tipado

El siguiente extracto corresponde al kernel principal de EP en su versión tipada (modo 3), `ep_compute_types`. Ilustra las decisiones de implementación descritas en la Sección 3.3: la región `parallel` con reducción sobre `sx` y `sy`, la lista explícita de variables privadas, las `memoryviews` tipadas (`cython.double[:]`) y la sección crítica final que acumula el histograma. Se ha abreviado el cuerpo aritmético del bucle interno por claridad.

```
@omp(compile=use_compiled_types())
def ep_compute_types(nn: int, nk: int, nk_plus: int, nq: int, an: float)
    sx: float = 0.0
    sy: float = 0.0
    ...
    q_values = numpy.repeat(0.0, nq)
    q: cython.double[:] = q_values

    with omp("parallel reduction(+:sx,sy) "
            "private(i,k,seed_iter,ik,kk,l,a1,x1_seed,t2_int,t4_int,"
            "t1,t3,a2,x2_seed,z,t1_local,t2_local,x1,x2,"
            "t1_value,t2_value,t3_value,t4_value)"):
        qq_values = numpy.repeat(0.0, nq)
        x_values = numpy.empty(nk_plus, dtype=numpy.float64)
        qq: cython.double[:] = qq_values
        x_local: cython.double[:] = x_values

        with omp("for schedule(static)"):
            for k in range(1, nn + 1):
```

```

...                               # generacion de muestras (Box-Muller)
for i in range(nk):
    x1 = 2.0 * x_local[2 * i] - 1.0
    x2 = 2.0 * x_local[2 * i + 1] - 1.0
    t1_value = x1 * x1 + x2 * x2
    if t1_value <= 1.0:
        t2_value = math.sqrt(-2.0 * math.log(t1_value)
                               / t1_value)
        t3_value = x1 * t2_value
        t4_value = x2 * t2_value
    l = int(max(abs(t3_value), abs(t4_value)))
    qq[l] = qq[l] + 1.0
    sx = sx + t3_value
    sy = sy + t4_value

with omp("critical"):
    for i in range(nq):
        q[i] = q[i] + qq[i]

```

La declaración explícita de variables privadas en la cláusula `private(...)` es la corrección descrita en la Sección 3.3: sin ella, varias temporales se compartían entre hilos y la verificación NAS fallaba.

Infraestructura de ejecución en Finisterrae III

La campaña se automatiza con los scripts del directorio `scripts/ft3`. El script `array_run.sbatch` es la plantilla de Slurm que ejecuta cada elemento del array de trabajos; cada elemento procesa varias filas de la matriz de combinaciones de forma secuencial mediante `run_one.py`.

```
#!/usr/bin/env bash
#SBATCH --job-name=omp4py-npb
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=32
#SBATCH --mem-per-cpu=4G
#SBATCH --time=06:00:00

source "$FT3_ROOT/scripts/ft3/env.sh"
ft3_export_python_runtime "${RUNNER_PYTHON_TAG:-3.15t}"
RUNNER_PYTHON="$(ft3_venv_python "${RUNNER_PYTHON_TAG:-3.15t}")"

start_index=$(( (SLURM_ARRAY_TASK_ID - 1) * rows_per_task + 1 ))
end_index=$(( SLURM_ARRAY_TASK_ID * rows_per_task ))
for index in $(seq "$start_index" "$end_index"); do
    "$RUNNER_PYTHON" "$FT3_ROOT/scripts/ft3/run_one.py" \
        --manifest "$MANIFEST" --index "$index" \
        --campaign-dir "$CAMPAIGN_DIR"
done
```

La selección del intérprete *free-threaded* y la desactivación explícita del GIL se realizan en

`env.sh` mediante la variable `PYTHON_GIL=0` y la ruta de bibliotecas correspondiente a cada versión:

```
ft3_export_python_runtime() {  
    local prefix; prefix="$(ft3_python_prefix "$1")"  
    export LD_LIBRARY_PATH="$prefix/lib${LD_LIBRARY_PATH:+:$LD_LIBRARY_PATH}"  
    export PYTHON_GIL=0  
}
```

Estructura de los resultados de campaña

Cada campaña genera un directorio bajo `results/` con los siguientes artefactos, que constituyen la base de las tablas y figuras de los Capítulos 5, 6 y 7:

- `manifest.csv`: matriz completa de combinaciones de la campaña.
- `raw/`: registros crudos con nombre semántico (versión de Python, benchmark, clase, modo, número de hilos y repetición).
- `records/`: un fichero JSON por ejecución, con tiempo NAS interno, tiempo externo de proceso, Mop/s, código de retorno y verificación.
- `summary.csv`: tabla plana con todas las ejecuciones.
- `summary_best.csv`: agregados de mejor y media por combinación.
- `env.txt`: metadatos de la campaña (versiones de Python, NumPy, Cython y variables de entorno).

El registro de la versión exacta de cada componente y de las variables de entorno responde a la implicación metodológica del Capítulo 7: en evaluaciones de Python *free-threaded* la configuración del runtime y la afinidad de hilos pueden alterar la conclusión experimental, por lo que deben quedar documentadas junto con los resultados.

Glosario de acrónimos

ABI *Application Binary Interface*. Interfaz binaria de aplicación; en este trabajo, la del intérprete CPython compilado en modo *free-threaded*.

API *Application Programming Interface*. Interfaz de programación de aplicaciones.

AST *Abstract Syntax Tree*. Árbol de sintaxis abstracta generado por el analizador sintáctico del intérprete Python a partir del código fuente.

CESGA Centro de Supercomputación de Galicia.

CG *Conjugate Gradient*. Benchmark NAS que evalúa operaciones sobre matrices dispersas mediante el método del gradiente conjugado.

CPU *Central Processing Unit*. Unidad central de procesamiento.

DSL *Domain-Specific Language*. Lenguaje específico de dominio.

EP *Embarrassingly Parallel*. Benchmark NAS que evalúa la generación de pares gaussianos mediante el método de Box-Muller.

FFT *Fast Fourier Transform*. Transformada rápida de Fourier, algoritmo eficiente para calcular la transformada discreta de Fourier.

FT *Fourier Transform*. Benchmark NAS que evalúa la transformada discreta de Fourier tridimensional sobre un array complejo.

GIL *Global Interpreter Lock*. Mecanismo de bloqueo global del intérprete CPython que impide la ejecución simultánea de hilos Python sobre el bytecode del intérprete.

HPC *High Performance Computing*. Computación de altas prestaciones.

IPC *Instructions Per Cycle*. Instrucciones retiradas por ciclo de reloj; métrica de eficiencia de ejecución medida con `perf stat`.

IS *Integer Sort*. Benchmark NAS que evalúa la ordenación de enteros mediante un algoritmo de tipo bucket sort.

MG *Multi-Grid*. Benchmark NAS que evalúa operadores de restricción, prolongación e interpolación sobre mallas en múltiples niveles.

Mop/s Millones de operaciones por segundo. Métrica de rendimiento empleada por los NAS Parallel Benchmarks.

NAS *NASA Advanced Supercomputing*. División de la NASA que desarrolló los NAS Parallel Benchmarks.

NPB *NAS Parallel Benchmarks*. Conjunto de programas de referencia para la evaluación de sistemas de computación paralela.

OpenMP *Open Multi-Processing*. Estándar de programación paralela basado en directivas de compilador para sistemas de memoria compartida.

PEP *Python Enhancement Proposal*. Documento de propuesta de mejora del lenguaje Python.

Glosario de términos

Aceleración (*speedup*) Cociente entre el tiempo de ejecución con un hilo y el tiempo con t hilos dentro del mismo modo y clase. Un valor superior a 1 indica ganancia con el paralelismo; un valor inferior a 1 indica degradación.

Barrera (*barrier*) Punto de sincronización que obliga a todos los hilos de una región paralela a esperar hasta que el último de ellos llegue al mismo punto antes de continuar.

Benchmark Programa de referencia diseñado para medir el rendimiento de un sistema de forma reproducible y comparable entre plataformas.

Cython Compilador que acepta un superconjunto del lenguaje Python con anotaciones de tipo opcionales y genera código C que se compila como módulo de extensión para CPython. Las anotaciones de tipo permiten sustituir objetos Python por tipos C nativos en los bucles críticos.

Free-threaded Python Variante experimental de CPython, disponible desde la versión 3.13 mediante la opción `--disable-gil`, que elimina el GIL y permite la ejecución simultánea de múltiples hilos sobre el bytecode del intérprete.

Kernel numérico Función que concentra la mayor parte del cómputo útil de un benchmark o aplicación paralela, separada del código de orquestación (inicialización, E/S, verificación).

Memoryview Objeto Python que expone el buffer interno de un array NumPy sin copiar los datos. En Cython, las memoryviews tipadas permiten acceder directamente a los elementos del array como punteros C, eliminando la indirección de los objetos Python en los bucles internos.

Modo de ejecución En OMP4Py, configuración que determina qué partes del código se compilan: modo 0 (runtime puro de OMP4Py en Python), modo 1 (híbrido, runtime nativo

cuando está disponible; kernels sin compilar), modo 2 (kernels compilados con Cython) y modo 3 (kernels compilados con tipado Cython explícito).

OMP4Py Implementación en Python puro del modelo de programación OpenMP. Traduce directivas OpenMP embebidas como cadenas de texto en código Python paralelo mediante transformación del AST, con soporte opcional de compilación Cython.

Overhead de sincronización Tiempo invertido en operaciones de coordinación entre hilos (barreras, secciones críticas, reducciones) que no contribuye directamente al cómputo útil.

Reducción Operación que combina los valores parciales calculados por cada hilo en un único resultado escalar (suma, máximo, mínimo, etc.) al final de una región paralela.

Región paralela Bloque de código delimitado por una directiva `parallel` de OpenMP que se ejecuta simultáneamente en múltiples hilos.

Slurm Sistema de gestión y planificación de trabajos utilizado en los supercomputadores del CESGA para asignar recursos de cómputo y organizar la ejecución de trabajos en cola.

Tipado Cython Proceso de anotar variables en código Cython con tipos C explícitos (`cython.int`, `cython.double`, etc.) para que el compilador genere operaciones nativas en lugar de pasar por el mecanismo de despacho de objetos Python.

Verificación NAS Comprobación numérica integrada en cada NAS Parallel Benchmark que calcula un checksum de los resultados y lo compara con un valor de referencia para la clase ejecutada. Una verificación **SUCCESSFUL** garantiza la corrección del cómputo.

Bibliografía

- [1] M. Gorelick and I. Ozsvald, *High Performance Python: Practical Performant Programming for Humans*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2020.
- [2] S. Gross, “Pep 703 – making the global interpreter lock optional in cpython,” <https://peps.python.org/pep-0703/>, 2023.
- [3] C. Piñeiro and J. C. Pichel, “Omp4py: A pure python implementation of openmp,” *Future Generation Computer Systems*, vol. 175, 2026.
- [4] —, “Unlocking python multithreading capabilities using openmp-based programming with omp4py,” in *IEEE/ACM International Symposium on Code Generation and Optimization*, 2026.
- [5] OpenMP Architecture Review Board, “Openmp application programming interface,” <https://www.openmp.org/specifications/>.
- [6] D. H. Bailey, E. Barszcz, J. T. Barton, D. S. Browning, R. L. Carter, L. Dagum, R. Fatoohi, P. O. Frederickson, T. A. Lasinski, R. S. Schreiber, H. D. Simon, V. Venkatakrisnan, and S. K. Weeratunga, “The nas parallel benchmarks,” NASA Ames Research Center, Tech. Rep. RNR-94-007, 1994.
- [7] G. Hager and G. Wellein, *Introduction to High Performance Computing for Scientists and Engineers*, 1st ed. Boca Raton, FL, USA: CRC Press, Inc., 2010.
- [8] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.
- [9] S. K. Lam, A. Pitrou, and S. Seibert, “Numba: A LLVM-based Python JIT compiler,” in *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC (LLVM '15)*, 2015.

- [10] L. D. Dalcín, R. R. Paz, P. A. Kler, and A. Cosimo, “Parallel distributed computing using Python,” *Advances in Water Resources*, vol. 34, no. 9, pp. 1124–1139, 2011.
- [11] M. Rocklin, “Dask: Parallel computation with blocked algorithms and task scheduling,” in *Proceedings of the 14th Python in Science Conference (SciPy)*, 2015, pp. 126–132.
- [12] L. Dagum and R. Menon, “OpenMP: An industry standard API for shared-memory programming,” *IEEE Computational Science and Engineering*, vol. 5, no. 1, pp. 46–55, 1998.
- [13] S. Behnel, R. Bradshaw, C. Citro, L. Dalcin, D. S. Seljebotn, and K. Smith, “Cython: The best of both worlds,” *Computing in Science & Engineering*, vol. 13, no. 2, pp. 31–39, 2011.
- [14] T. A. Anderson and T. G. Mattson, “Multithreaded parallel Python through OpenMP support in Numba,” in *Proceedings of the 20th Python in Science Conference (SciPy 2021)*, 2021.
- [15] D. Di Domenico *et al.*, “NPB-Python: NAS Parallel Benchmarks for Python,” <https://github.com/danidomenico/NPB-PYTHON>, 2021.
- [16] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” *Proceedings of the AFIPS Spring Joint Computer Conference*, pp. 483–485, 1967.
- [17] P. Galindo Salgado *et al.*, “Pep 768 – safe external debugger interface for cpython,” <https://peps.python.org/pep-0768/>, 2025.
- [18] Python Software Foundation, “The Python profilers: Statistical sampling profiler (profiling.sampling),” <https://docs.python.org/3.15/library/profiling.sampling.html>, 2026.